1 比赛 1

2 数学 1

3 数据结构 2

4 数值计算 6

5 数论 12

6 组合数学 15

7 图论 16

8 匹配 18

9 几何 23

10 字符串 27

11 杂项 29

比赛 (1)

template.cpp 9 lines

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
ll cdiv(ll a,ll b){return a/b+((a^b)>0&&a%b);} // 向上取整
ll fdiv(ll a,ll b){return a/b-((a^b)<0&&a%b);} // 向下取整
int main(){
  ios::sync_with_stdio(0);cin.tie(0);

}
```

hash.sh 3 lines

```sh
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

1.1 Python 速查

```python
import sys, math
from collections import deque, Counter, defaultdict
from heapq import heappush, heappop, heapify, nsmallest
from bisect import bisect_left, bisect_right, insort
from itertools import accumulate, combinations, product
from functools import lru_cache, cmp_to_key
from fractions import Fraction
from decimal import Decimal, getcontext
input = sys.stdin.readline  # 快速读入，行尾带换行
sys.setrecursionlimit(1 << 20)  # 递归深的题优先改成循环
n, m = map(int, input().split())
a = list(map(int, input().split()))
rest = sys.stdin.read().split()  # 一次读入剩下全部
print(*a)  # 空格分隔输出
sys.stdout.write("\n".join(map(str, a)) + "\n")
x = 10 ** 100  # 整数没有上限
p = 998244353
y = pow(3, x, p)  # 快速幂取模
z = pow(3, -1, p)  # 逆元，需 3.8 以上
g = math.gcd(12, 18); s = math.isqrt(10 ** 20)
c = math.comb(50, 25); l = math.lcm(4, 6)  # lcm 需 3.9 以上
q = -7 // 2  # 向下取整得 -4，和 C++ 不同，% 也是
f = Fraction(1, 3) + Fraction(1, 6)  # 精确分数
getcontext().prec = 50
d = Decimal(2).sqrt()  # 高精度小数
h = int("ff", 16); b = bin(10)[2:]; k = (10).bit_length()
pc = bin(10).count("1"); t = "abc"[::-1]; o = ord("a")
a.sort(key=lambda v: (-v, v % 3))  # 多关键字排序
a.sort(key=cmp_to_key(lambda u, v: u - v))  # 自定义比较
adj = [[] for _ in range(n)]  # 不要写 [[]] * n
dp = [[0] * m for _ in range(n)]
pq = [5, 1, 4]; heapify(pq); heappush(pq, 2)
mn = heappop(pq)  # 小根堆，大根堆存相反数
dq = deque([1, 2]); dq.appendleft(0); dq.pop()
i = bisect_left([1, 3, 5], 3)  # 同 lower_bound
cnt = Counter("abca"); dd = defaultdict(int)
pre = list(accumulate([1, 2, 3], initial=0))  # 前缀和
@lru_cache(maxsize=None)  # 记忆化搜索
def fib(v): return v if v < 2 else fib(v - 1) + fib(v - 2)
INF = float("inf")
# 字符串，不可修改，要改先转成 list(s)
s = "  Hello,World  "
w = s.strip().split(",")  # 去首尾空白，按逗号切
j = "-".join(w); u = s.upper(); lo = s.lower()
pos = s.find("o"); r = s.replace("l", "L")  # 找不到返回 -1
ok = "123".isdigit(); bg = s.strip().startswith("He")
cs = list("abc"); cs[0] = "z"; s2 = "".join(cs)
fs = f"{3.14159:.3f} {42:05d} {255:x}"  # 格式化输出
# 列表
a = [3, 1, 2]; a.append(5); a.insert(0, 9); a.pop()
a.extend([7, 7]); c7 = a.count(7); i1 = a.index(1)
b = sorted(a, reverse=True); a.reverse()
sq = [v * v for v in a if v % 2]  # 列表推导
mx = max(a, key=lambda v: -v); sm = sum(a)
for i, v in enumerate(a): pass
pairs = list(zip([1, 2], "ab")); rv = a[::-1]; sub = a[1:3]
# 集合 / 字典 / 元组
st = {1, 2}; st.add(3); st.discard(9); st2 = st | {4}
inter = st & {2, 3}; diff = st - {1}; has = 2 in st
d = {"x": 1}; d["y"] = 2; gv = d.get("z", 0)
for k, v in sorted(d.items(), key=lambda t: -t[1]): pass
dl = defaultdict(list); dl[1].append(2)
x1, y1 = (4, 5); x1, y1 = y1, x1  # 解包，交换
mc = Counter("aabbbc").most_common(2)  # 出现最多的两个
small = nsmallest(2, [5, 1, 4]); dq2 = deque([1, 2, 3])
dq2.rotate(1)  # 循环右移一位
```

数学 (2)

2.1 方程

$$ax^2+bx+c=0 \Rightarrow x=\frac{-b\pm\sqrt{b^2-4ac}}{2a}$$

极值点为 $x=-b/2a$.

$$\begin{aligned}ax+by&=e\\cx+dy&=f\end{aligned}\Rightarrow\begin{aligned}x&=\frac{ed-bf}{ad-bc}\\y&=\frac{af-ec}{ad-bc}\end{aligned}$$

一般地, 对方程 $Ax=b$, 变量 x_i 的解为

$$x_i=\frac{\det A_i'}{\det A}$$

其中 A_i' 是把 A 的第 i 列换成 b 所得的矩阵.

2.2 递推

若 $a_n=c_1a_{n-1}+\cdots+c_ka_{n-k}$, 且 $r_1,\ldots,r_k$ 是 $x^k-c_1x^{k-1}-\cdots-c_k$ 的互异根, 则存在 $d_1,\ldots,d_k$ 使得

$$a_n=d_1r_1^n+\cdots+d_kr_k^n.$$

重根 r 会乘上多项式因子, 如 $a_n=(d_1n+d_2)r^n$.

2.3 三角函数

$$\sin(v+w)=\sin v\cos w+\cos v\sin w$$
$$\cos(v+w)=\cos v\cos w-\sin v\sin w$$

$$\tan(v+w)=\frac{\tan v+\tan w}{1-\tan v\tan w}$$
$$\sin v+\sin w=2\sin\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$\cos v+\cos w=2\cos\frac{v+w}{2}\cos\frac{v-w}{2}$$
$$(V+W)\tan(v-w)/2=(V-W)\tan(v+w)/2$$

其中 V,W 是角 v,w 的对边长.

$$a\cos x+b\sin x=r\cos(x-\phi)$$
$$a\sin x+b\cos x=r\sin(x+\phi)$$

其中 $r=\sqrt{a^2+b^2},\phi=\operatorname{atan2}(b,a)$.

2.4 几何

2.4.1 三角形

边长: a,b,c

半周长: $p=\frac{a+b+c}{2}$

面积: $A=\sqrt{p(p-a)(p-b)(p-c)}$

外接圆半径: $R=\frac{abc}{4A}$

内切圆半径: $r=\frac{A}{p}$

中线长 (把三角形分成面积相等的两半): $m_a=\frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

角平分线长: $s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$

正弦定理: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

余弦定理: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

正切定理: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

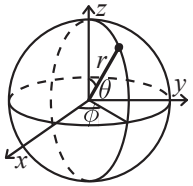
2.4.2 四边形

设边长 a, b, c, d, 对角线 e, f, 对角线夹角 θ, 面积 A, 记 $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

圆内接四边形对角之和为 $180°$, $ef = ac + bd$, 且 $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

2.4.3 球坐标



$$\begin{array}{ll} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y, x) \end{array}$$

2.5 导数/积分

$$\begin{array}{ll} \frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx}\tan x = 1 + \tan^2 x & \frac{d}{dx}\arctan x = \frac{1}{1+x^2} \\ \int \tan ax = -\frac{\ln|\cos ax|}{a} & \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2} \\ \int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1) \end{array}$$

分部积分:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 求和

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

2.7 级数

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

2.8 概率论

设 X 是离散随机变量, 取值 x 的概率为 $p_X(x)$. 其期望 (均值) 为 $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$, 方差为 $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$, 其中 σ 是标准差. 若 X 是连续的, 则有概率密度函数 $f_X(x)$, 上面的求和改为积分, 并把 $p_X(x)$ 换成 $f_X(x)$.

期望是线性的:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

若 X 与 Y 独立,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.9 Markov 链

Markov 链 是一种离散随机过程, 下一状态只依赖当前状态. 设 $X_1, X_2, \ldots$ 是该过程产生的随机变量序列. 则有转移矩阵 $\mathbf{P} = (p_{ij})$, 其中 $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, 且 $\mathbf{p}^{(n)} = \mathbf{P}^n\mathbf{p}^{(0)}$ 是 X_n 的概率分布 (即 $p_i^{(n)} = \Pr(X_n = i)$), 其中 $\mathbf{p}^{(0)}$ 是初始分布.

平稳分布

若 $\pi = \pi\mathbf{P}$, 则称 π 为平稳分布. 若 Markov 链不可约 (任意状态都能到达任意状态), 则 $\pi_i = \frac{1}{\mathbb{E}(T_i)}$, 其中 $\mathbb{E}(T_i)$ 是相邻两次访问状态 i 的期望间隔. π_j/π_i 是相邻两次访问 i 之间访问状态 j 的期望次数.

在连通, 无向, 非二分的图上, 若等概率走向各邻居, 则 π_i 与点 i 的度数成正比.

遍历性

若极限分布与初始分布无关, 则称 Markov 链是遍历的. 有限 Markov 链遍历当且仅当它不可约且非周期 (即各环长的 gcd 为 1). $\lim_{k\to\infty}\mathbf{P}^k = \mathbf{1}\pi$.

吸收

若状态可分成两个集合 $\mathbf{A}$ 和 $\mathbf{G}$, 使 $\mathbf{A}$ 中的状态都是吸收态 ($p_{ii} = 1$), 且 $\mathbf{G}$ 中的状态都能到达 $\mathbf{A}$ 中的吸收态, 则称其为 A-链. 从 j 出发, 被状态 $i \in \mathbf{A}$ 吸收的概率为 $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. 从 i 出发, 到被吸收的期望时间为 $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$.

2.10 期望与概率技巧

- **期望线性性**: 不要求独立. 求个数的期望就拆成指示变量, $\mathbb{E}[\text{个数}] = \sum_i \Pr[\text{事件 } i]$.
- **尾和**: X 取非负整数时 $\mathbb{E}[X] = \sum_{k\geq 1}\Pr[X \geq k]$. 独立同分布的最大值: $\Pr[\max \leq k] = F(k)^n$.
- **期望 DP 倒推**: $E_i = c_i + \sum_j p_{ij}E_j$, 终点 $E = 0$, 按拓扑序从后往前算. 只有自环时移项: $E_i = (c_i + \sum_{j\neq i} p_{ij}E_j)/(1-p_{ii})$.
- **有环, 高斯消元**: 把 $E_i - \sum_j p_{ij}E_j = c_i$ 列成方程组, 用 `SolveLinear`, $O(n^3)$, n 约到 500. 取模时把除法换成乘逆元. 吸收概率同理.
- **只有一个未知量**: 设起点期望为 x, 其他状态都写成 $a_i + b_ix$ 一路推下去, 最后解 $x = a + bx$, 不用高斯消元.
- **二分答案**: 可以随时重来的题, 答案 x 出现在自己的转移里, 即 $E_i = \min(\text{继续}, x)$. 对猜的 x 做一遍 DP 得到 $f(x)$; 若 $f(x) < x$ 则真答案 $< x$, 否则 $\geq x$. 二分约 100 次.
- **方差**: DP 里同时维护 $\mathbb{E}[X]$ 和 $\mathbb{E}[X^2]$, 如 $\mathbb{E}[(X+1)^2] = \mathbb{E}[X^2] + 2\mathbb{E}[X] + 1$.
- **min-max 容斥**: $\mathbb{E}[\max S] = \sum_{\emptyset\neq T\subseteq S}(-1)^{|T|+1}\mathbb{E}[\min T]$. 用于全部出现的期望时间.
- **常用结论**: 成功率 p 的试验, 期望 $1/p$ 次成功一次. 集齐 n 种卡片: nH_n. n 个 $U[0,1]$ 中第 k 小的期望: $k/(n+1)$. 随机排列: 逆序对期望 $n(n-1)/4$, 不动点期望 1, 环数期望 H_n.
- **随机游走**: 树上从 u 走到父亲的期望步数是 $2 \cdot size(u) - 1$. 0 处反射, 从 0 走到 n 期望 n^2 步. 公平赌徒从 i 出发: 先到 n 的概率 i/n, 期望步数 $i(n-i)$.

- **精度**: 期望很大时用 `long double`; 方程太大时可以反复迭代 $E \leftarrow c + PE$ 直到收敛.

数据结构 (3)

OrderStatisticTree.h

Description: pb_ds 平衡树 ordered_set: 有序集合, 比 set 多两个操作, 求排名 order_of_key 和求第 k 小 find_by_order, 都是 $O(\log n)$. 元素互不相同, 重复插入无效. 其余接口同 set: insert, erase, find, lower_bound, upper_bound, size, clear, begin, end. s.order_of_key(x): 返回严格小于 x 的元素个数, x 不必在集合里. s.find_by_order(k): 返回第 k 小的迭代器, k 从 0 开始, k 不小于 size 时为 end. 换元素类型: 改 typedef 里的两个 int, 例如换成 ll. 当 map 用: 把 null_type 换成值类型. 要可重: 元素换成 pair, first 存值, second 存互不相同的编号, 如插入时间, 此时 x 的排名为 order_of_key({x, 0}), 要求编号都大于 0. 注意: 需要 GCC; 常数比 set 大约 2 到 3 倍.

Usage:
```
ordered_set s; // 多组数据: s.clear();
s.insert(5); s.insert(1); s.insert(9);
int r = s.order_of_key(5); // 1, 严格小于 5 的有 1 个
int v = *s.find_by_order(2); // 9, 即第 2 小, 从 0 开始数
if (s.find(1) != s.end()) s.erase(1);
// 值在 [3, 10) 内的个数:
int cnt = s.order_of_key(10) - s.order_of_key(3);
```

Time: 每次操作 $\mathcal{O}(\log N)$

d3016d, 4 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
typedef tree<int,null_type,less<int>,rb_tree_tag,
  tree_order_statistics_node_update>ordered_set;
```

HashMap.h

Description: pb_ds 哈希表 h, 用法同 unordered_map, 但快约 3 倍. 键为 ll, 值为 int; 要别的类型就改 gp_hash_table 的前两个参数, 键必须是整数类型, 因为 chash 只会哈希整数. chash 让高位也参与哈希; RND 是运行时的随机数, 防 CodeForces 上的 hack. 初始容量 2^{16}, 必须是 2 的幂. h[key] 在键不存在时会插入值 0. 只判断存在请用 h.find(key) != h.end(), 不要用 h[key], 否则会插入. 注意: 内存约为 unordered_map 的 1.5 倍; 遍历顺序是随机的.

Usage:
```
h.clear(); // 多组数据清空
h[key] = val; h[key]++; // 插入或修改
auto it = h.find(key);
if (it != h.end()) cout << it->second; // 存在则输出值
h.erase(key); int sz = h.size();
for (auto [k, v] : h) ...; // 遍历, 顺序随机
```

Time: 均摊 $\mathcal{O}(1)$

cb991d, 11 lines

```
#include <bits/extc++.h>
const unsigned long long RND=
  chrono::steady_clock::now().time_since_epoch().count();
struct chash{
  unsigned long long operator()(ll x)const{
    // C 为大奇数
    const unsigned long long C=(ll)(4e18*acos(0))|71;
    return __builtin_bswap64((x^RND)*C);
  }
};
__gnu_pbds::gp_hash_table<ll,int,chash>h({},{},{},{},{1<<16});
```

LazySegmentTree.h

Description: 懒标记线段树, 维护序列 a[1..n], 默认支持区间加, 查询区间和与区间最大值. 输入: 全局 n 和 a[1..n], 要求 $n \le N$. 节点 p 管 [l, r], 儿子为 2p 和 2p+1. build(1, 1, n): 用 a 建树, 同时清空标记. update(1, 1, n, x, y, v): a[x..y] 每个数加 v, 要求 $1 \le x \le y \le n$. query(1, 1, n, x, y): 返回 Node, sum 为 a[x..y] 的和, mx 为最大值. 换用途只改 Node 的字段, operator+, build 的叶子, 标记数组, addtag 和 pushdown, 写法见下方懒标记写法. query 只合并非空的两半, 所以不需要单位元. 注意: 数组开 4N; 和可能超过 int, 用 ll; 多组数据重新 build 即可.

Usage:
```
cin >> n >> q;
for (int i = 1; i <= n; i++) cin >> a[i];
build(1, 1, n); // 建树, 多组数据每组都调用
update(1, 1, n, l, r, v); // a[l..r] 都加 v
Node res = query(1, 1, n, l, r);
cout << res.sum << ' ' << res.mx; // 区间和, 区间最大值
```

Time: 建树 $\mathcal{O}(N)$, 修改和查询 $\mathcal{O}(\log N)$

13a98c, 52 lines

```
const int N=2e5+5;
int n;
ll a[N];
struct Node{ll sum,mx;};
Node t[N<<2];
ll lz[N<<2]; // 整段待加的值
Node operator+(Node x,Node y){ // 合并左右儿子
  return {x.sum+y.sum,max(x.mx,y.mx)};
}
void pushup(int p){t[p]=t[p<<1]+t[p<<1|1];}
void addtag(int p,int l,int r,ll v){ // 整段 [l, r] 都加 v
  t[p].sum+=v*(r-l+1);
  t[p].mx+=v;
  lz[p]+=v;
}
void pushdown(int p,int l,int r){
  if(!lz[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m,lz[p]);
  addtag(p<<1|1,m+1,r,lz[p]);
  lz[p]=0;
}
void build(int p,int l,int r){
  lz[p]=0;
  if(l==r){
    t[p]={a[l],a[l]};
    return;
  }
  int m=(l+r)>>1;
  build(p<<1,l,m);
  build(p<<1|1,m+1,r);
  pushup(p);
}
void update(int p,int l,int r,int x,int y,ll v){ // 加 v
  if(x<=l&&r<=y){
    addtag(p,l,r,v);
    return;
  }
  pushdown(p,l,r);
  int m=(l+r)>>1;
  if(x<=m)update(p<<1,l,m,x,y,v);
  if(y>m)update(p<<1|1,m+1,r,x,y,v);
  pushup(p);
}
Node query(int p,int l,int r,int x,int y){ // [x, y]
  if(x<=l&&r<=y)return t[p];
  pushdown(p,l,r);
  int m=(l+r)>>1;
  if(y<=m)return query(p<<1,l,m,x,y);
  if(x>m)return query(p<<1|1,m+1,r,x,y);
  return query(p<<1,l,m,x,y)+query(p<<1|1,m+1,r,x,y);
}
```

懒标记写法

换用途只改: Node 的字段和合并两个儿子的 operator+, build 里叶子的初值, 标记数组, addtag 和 pushdown. addtag(p, l, r, v): 给整段 $[l, r]$ 打上标记 v, 要立刻更新 t[p], 并把 v 合并进 p 的旧标记, 旧标记先做, v 后做. update 命中整段时调用它, pushdown 给两个儿子时也调用它. 新加的标记数组都要在 build 里设成空标记, 否则多组数据会残留. 没列出的函数 (pushup, build 的递归部分, update, query) 保持模板原样. 区间加就是上面的模板.

- **区间赋值, 求和与最大值**: update(1, 1, n, l, r, v) 把 $a[l..r]$ 都赋成 v. build 里加 tg[p]=0.

```
// 标记: tg[p] 为 1 表示整段待赋成 lz[p]
ll lz[N<<2];
bool tg[N<<2];
void addtag(int p,int l,int r,ll v){ // 整段赋成 v
  t[p].sum=v*(r-l+1);
  t[p].mx=v;
  lz[p]=v; // 新赋值直接覆盖旧标记
  tg[p]=1;
}
void pushdown(int p,int l,int r){
  if(!tg[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m,lz[p]);
  addtag(p<<1|1,m+1,r,lz[p]);
  tg[p]=0;
}
```

- **赋值和加混用**: update 多一个参数 op, op 为 1 时把 $a[l..r]$ 赋成 v, 为 0 时都加 v. build 里把 sv, ad, tg 都清 0.

```
// 标记: 若 tg[p] 先整段赋成 sv[p], 再整段加 ad[p]
ll sv[N<<2],ad[N<<2];
bool tg[N<<2];
void setv(int p,int l,int r,ll v){ // 整段赋成 v
  t[p].sum=v*(r-l+1);
  t[p].mx=v;
  sv[p]=v;
  tg[p]=1;
  ad[p]=0; // 赋值清掉之前的加
}
void addv(int p,int l,int r,ll v){ // 整段加 v
  t[p].sum+=v*(r-l+1);
  t[p].mx+=v;
  ad[p]+=v;
}
void pushdown(int p,int l,int r){
  int m=(l+r)>>1;
  if(tg[p]){ // 先下传赋值
    setv(p<<1,l,m,sv[p]);
    setv(p<<1|1,m+1,r,sv[p]);
    tg[p]=0;
  }
  if(ad[p]){ // 再下传加
    addv(p<<1,l,m,ad[p]);
    addv(p<<1|1,m+1,r,ad[p]);
    ad[p]=0;
  }
}
void update(int p,int l,int r,int x,int y,int op,ll v){
  if(x<=l&&r<=y){
    if(op)setv(p,l,r,v);
    else addv(p,l,r,v);
    return;
  }
  // 其余同模板, 递归时也传 op
```

- **区间乘再加, 求和取模**: update(1, 1, n, l, r, k, b) 把 $a[l..r]$ 每个数 v 变成 $kv + b$, 要求 $0 \le k, b < M$,

$0 \le a_i < M$. 去掉 mx. build 里 mu[p]=1, ad[p]=0, 叶子 t[p]={a[l]%M}.

```
const ll M=998244353;
struct Node{ll sum;};
Node operator+(Node x,Node y){
  return {(x.sum+y.sum)%M};
}
// 标记：整段每个数先乘 mu[p] 再加 ad[p]
ll mu[N<<2],ad[N<<2];
void addtag(int p,int l,int r,ll k,ll b){
  t[p].sum=(k*t[p].sum+b*(r-l+1))%M;
  mu[p]=mu[p]*k%M;
  ad[p]=(ad[p]*k+b)%M;
}
void pushdown(int p,int l,int r){
  if(mu[p]==1&&!ad[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m,mu[p],ad[p]);
  addtag(p<<1|1,m+1,r,mu[p],ad[p]);
  mu[p]=1;
  ad[p]=0;
}
// update 的参数 v 换成 k, b, 命中整段时 addtag(p,l,r,k,b)
```

- **01 序列区间取反, 数 1 的个数**: $a[i]$ 为 0 或 1, update(1, 1, n, l, r) 把 $a[l..r]$ 中 0 和 1 互换, query(1, 1, n, l, r).cnt 为其中 1 的个数. build 里 rev[p]=0, 叶子 t[p]={(int)a[l]}.

```
struct Node{int cnt;}; // 区间里 1 的个数
Node operator+(Node x,Node y){return {x.cnt+y.cnt};}
bool rev[N<<2]; // 取反标记
void addtag(int p,int l,int r){ // 整段 0 和 1 互换
  t[p].cnt=(r-l+1)-t[p].cnt;
  rev[p]^=1;
}
void pushdown(int p,int l,int r){
  if(!rev[p])return;
  int m=(l+r)>>1;
  addtag(p<<1,l,m);
  addtag(p<<1|1,m+1,r);
  rev[p]=0;
}
// update 去掉参数 v, 命中整段时 addtag(p,l,r)
```

- **最大子段和**: 只有单点赋值 modify(1, 1, n, x, v) 把 $a[x]$ 赋成 v, 不需要标记. query(1, 1, n, l, r).best 为 $a[l..r]$ 的最大非空子段和. 叶子 t[p]={a[l], a[l], a[l], a[l]}.

```
// pre: 最大前缀和, suf: 最大后缀和, best: 答案
struct Node{ll sum,pre,suf,best;};
Node operator+(Node x,Node y){
  Node r;
  r.sum=x.sum+y.sum;
  r.pre=max(x.pre,x.sum+y.pre);
  r.suf=max(y.suf,y.sum+x.suf);
  r.best=max({x.best,y.best,x.suf+y.pre});
  return r;
}
// 删去 lz, addtag, pushdown 及其所有调用
void modify(int p,int l,int r,int x,ll v){ // a[x] 赋成 v
  if(l==r){
    t[p]={v,v,v,v};
    return;
  }
  int m=(l+r)>>1;
  if(x<=m)modify(p<<1,l,m,x,v);
  else modify(p<<1|1,m+1,r,x,v);
  pushup(p);
}
```

PersistentSegmentTree.h

Description: 可持久化线段树, 即主席树. 下标 [1, n], 每个节点存区间和. 每次单点加都产生一个新版本, 即新根, 只新建 $O(\log n)$ 个节点, 旧版本不变, 所以可以在任意历史版本上查询或继续修改. 节点 0 是全 0 的空树, rt[0] = 0. update(pre, 1, n, x, v): 在版本 pre 的基础上 a[x] 加 v, 返回新版本的根. query(p, 1, n, l, r): 版本 p 中 a[l..r] 的和. kth(u, v, 1, n, k): 把版本 v 减去版本 u 看成计数数组, 返回其中第 k 小 所在的下标, k 从 1 开始, 要求 k 不超过总个数. 静态区间第 k 小: 值离散化到 1..m, 版本 i 在 id(a[i]) 处加 1, 则 rt[r] 减 rt[l - 1] 就是 a[l..r] 中各值的出现次数. 注意: 节点数 MX 约为 修改次数 * (log n + 2), 默认 N * 20 够 2e5 次修改; 多组数据只需 tot = 0.

Usage: tot = 0; // 多组数据清空

```
// 例 1: 历史版本区间和
rt[i] = update(rt[j], 1, n, x, v); // 由版本 j 得到版本 i
ll s = query(rt[i], 1, n, l, r); // 版本 i 中 a[l..r] 的和
// 例 2: 静态区间第 k 小, b[1..m] 为排序去重后的值
for (int i = 1; i <= n; i++) {
int id = lower_bound(b + 1, b + m + 1, a[i]) - b;
rt[i] = update(rt[i - 1], 1, m, id, 1);
}
ll ans = b[kth(rt[l - 1], rt[r], 1, m, k)]; // a[l..r] 第 k 小
```

Memory: $\mathcal{O}(N \log N)$

Time: 每次操作 $\mathcal{O}(\log N)$

becafc, 29 lines

```
const int N=2e5+5;
const int MX=N*20; // 节点数, 约修改次数 * (log n + 2)
int ls[MX],rs[MX],tot,rt[N];
ll sum[MX];
int update(int pre,int l,int r,int x,ll v){ // 返回新根
  int p=++tot;
  ls[p]=ls[pre];
  rs[p]=rs[pre];
  sum[p]=sum[pre]+v;
  if(l==r)return p;
  int m=(l+r)>>1;
  if(x<=m)ls[p]=update(ls[pre],l,m,x,v);
  else rs[p]=update(rs[pre],m+1,r,x,v);
  return p;
}
ll query(int p,int l,int r,int x,int y){ // [x, y] 的和
  if(!p||y<l||r<x)return 0;
  if(x<=l&&r<=y)return sum[p];
  int m=(l+r)>>1;
  return query(ls[p],l,m,x,y)+query(rs[p],m+1,r,x,y);
}
// 版本 v 减版本 u 中第 k 小的位置, k 从 1 开始
int kth(int u,int v,int l,int r,ll k){
  if(l==r)return l;
  int m=(l+r)>>1;
  ll c=sum[ls[v]]-sum[ls[u]]; // 左半的个数
  if(k<=c)return kth(ls[u],ls[v],l,m,k);
  return kth(rs[u],rs[v],m+1,r,k-c);
}
```

UnionFindRollback.h

Description: 并查集, 点 1..n, 两个版本互相独立, 只抄需要的. 普通并查集: init(n) 初始化; find(x) 返回 x 所在集合的根; unite(x, y) 合并两个集合, 已在同一集合返回 0, 否则返回 1. 可撤销并查集: 按大小合并, 不压缩路径, 所以 findR 是 $O(\log n)$. initR(n) 初始化; findR(x) 返回根; siz[findR(x)] 为 x 所在集合大小; uniteR(x, y) 同 unite, 每次成功合并压栈一次; time() 返回当前栈大小; rollback(t) 撤销到 time() 等于 t 时的状态, 即撤销 t 之后的所有合并. 常用于线段树分治, 离线动态连通性, 按顺序试加边再撤销. 注意: 撤销版不能路径压缩; 多组数据重新 init / initR 即可.

Usage: init(n); // 普通版

```
unite(x, y);
if (find(x) == find(y)) ...; // 是否连通
initR(n); // 可撤销版
int t = time(); // 记下当前时间
uniteR(x, y); uniteR(y, z);
int s = siz[findR(x)]; // 集合大小
rollback(t); // 撤销上面两次合并
```

Time: 普通 均摊 $\mathcal{O}(\log N)$, 实际接近 $\mathcal{O}(1)$; 可撤销 $\mathcal{O}(\log N)$

dba267, 52 lines

```
const int N=2e5+5;
int fa[N];
void init(int n){
  for(int i=1;i<=n;i++)fa[i]=i;
}
int find(int x){ // 路径压缩：减半写法，不递归，不怕爆栈
  while(fa[x]!=x){
    fa[x]=fa[fa[x]];
    x=fa[x];
  }
  return x;
}
bool unite(int x,int y){ // 已在同一集合返回 0
  x=find(x);
  y=find(y);
  if(x==y)return 0;
  fa[x]=y;
  return 1;
}

int pa[N],siz[N];
int stk[N],top; // stk: 被挂到别人下面的根
void initR(int n){
  top=0;
  for(int i=1;i<=n;i++){
    pa[i]=i;
    siz[i]=1;
  }
}
int findR(int x){
  while(pa[x]!=x)x=pa[x];
  return x;
}
bool uniteR(int x,int y){
  x=findR(x);
  y=findR(y);
  if(x==y)return 0;
  if(siz[x]>siz[y])swap(x,y); // 小的挂到大的下面
  pa[x]=y;
  siz[y]+=siz[x];
  stk[++top]=x;
  return 1;
}
int time(){return top;}
void rollback(int t){ // 撤销到 time() 等于 t 时
  while(top>t){
    int x=stk[top];
    top--;
    siz[pa[x]]-=siz[x];
    pa[x]=x;
  }
}
```

LiChaoTree.h

Description: 李超树: 维护一堆直线或线段, 查询某个整数 x 处的最大 y 值. x 取值于 [lo, hi), 动态开点, 范围大如 $\pm 10^9$ 也不用离散化. init(lo, hi): 设定 x 的范围并清空, 每组数据调用一次. addLine({k, b}): 插入直线 $y = kx + b$, 对所有 x 生效. addSeg({k, b}, x1, x2): 插入只在 $x \in [x_1, x_2)$ 上存在的线段, 左闭右开, 超出 [lo, hi) 的部分自动忽略. query(x): 返回所有覆盖 x 的线中 $kx + b$ 的最大值, 没

有则返回 LLONG_MIN. 求最小值: 插入 $(-k,-b)$, 再把 query 的答案取负. 注意: 要求 $|kx+b|<9\cdot10^{18}$, $hi-lo<9\cdot10^{18}$; 节点数 M: 每条直线最多 1 个, 每条线段最多约 2 log C 个, C = hi - lo.

Usage:
```
init(-1e9, 1e9 + 1); // x 的范围 [-1e9, 1e9]
addLine({2, 3}); // y = 2x + 3
addSeg({-1, 10}, 0, 5); // y = -x + 10, 只在 x = 0..4 上
ll y = query(x); // 若为 LLONG_MIN 表示没有线覆盖 x
// 求最小: addLine({-k, -b}); ll mn = -query(x);
```
Time: 直线和查询 $\mathcal{O}(\log C)$, 线段 $\mathcal{O}(\log^2 C)$

cf9a94, 58 lines

```
const int M=4e6+5; // 节点数
struct Line{ll k,b;};
Line ln[M];
int ls[M],rs[M],tot,root;
bool has[M]; // has: 节点上有线
ll lo,hi;
ll calc(Line a,ll x){return a.k*x+a.b;}
void init(ll L,ll R){
  lo=L;
  hi=R;
  tot=0;
  root=0;
}
// 节点 p 管 [l, r), 在 [x, y) 上插入 a
void ins(int&p,ll l,ll r,ll x,ll y,Line a){
  if(y<=l||r<=x)return;
  if(!p){ // 新建节点
    p=++tot;
    ls[p]=0;
    rs[p]=0;
    has[p]=0;
  }
  ll m=l+(r-l)/2;
  if(x<=l&&r<=y){
    if(!has[p]){
      ln[p]=a;
      has[p]=1;
      return;
    }
    if(calc(a,m)>calc(ln[p],m))swap(a,ln[p]);
    // 现在 a 在 m 处不优, 只可能在一侧更优
    if(r-l==1||a.k==ln[p].k)return;
    if(a.k<ln[p].k)ins(ls[p],l,m,x,y,a);
    else ins(rs[p],m,r,x,y,a);
    return;
  }
  ins(ls[p],l,m,x,y,a);
  ins(rs[p],m,r,x,y,a);
}
void addLine(Line a){ins(root,lo,hi,lo,hi,a);}
void addSeg(Line a,ll x1,ll x2){ins(root,lo,hi,x1,x2,a);}
ll query(ll x){
  ll res=LLONG_MIN;
  ll l=lo,r=hi;
  int p=root;
  while(p){
    if(has[p])res=max(res,calc(ln[p],x));
    ll m=l+(r-l)/2;
    if(x<m){
      p=ls[p];
      r=m;
    }else{
      p=rs[p];
      l=m;
    }
  }
  return res;
}
```

FHQTreap.h

Description: FHQ treap: 无旋平衡树, 只靠 split 和 merge, 节点 0 表示空. 两种用法, 不要混在同一棵树上: 用法 1, 按值有序的可重集合, 根为 root: insert(v) 插入一个 v; erase(v) 删除一个 v, 没有则不变; getrank(v) 返回严格小于 v 的个数加 1; kth(k) 返回第 k 小, k 从 1 开始; pre(v) 返回小于 v 的最大值; nxt(v) 返回大于 v 的最小值. 用法 2, 按位置的序列 a[1..n], 支持区间翻转和区间和: 用 root = merge(root, newnode(x)) 依次加到末尾建树; reverse(l, r) 翻转 a[l..r]; rangeSum(l, r) 返回 a[l..r] 的和; dfs(root, o) 按中序把当前序列放进 vector o. 要加别的区间操作: 仿照 reverse, 先 split 出 [l, r] 这棵树 y, 改 y 再 merge 回去. 注意: kth, pre, nxt 要保证答案存在; N 为插入总次数, 删掉的节点不回收; 多组数据令 tot = root = 0.

Usage:
```
tot = root = 0; // 多组数据清空
// 用法 1
insert(5); insert(3); insert(5); erase(5);
int r = getrank(5); // 2
ll v = kth(1), p = pre(5), q = nxt(3); // 3, 3, 5
// 用法 2
for (int i = 1; i <= n; i++)
root = merge(root, newnode(a[i]));
reverse(l, r); ll s = rangeSum(l, r);
vector<ll> o; dfs(root, o); // o 为当前序列
```
Time: 每次操作期望 $\mathcal{O}(\log N)$

f335c2, 129 lines

```
const int N=2e5+5; // 插入总次数
int ls[N],rs[N],siz[N],tot,root;
unsigned rnd[N];
ll val[N],sum[N];
bool rev[N];
mt19937 rng(12345);
int newnode(ll v){
  int p=++tot;
  ls[p]=0;
  rs[p]=0;
  siz[p]=1;
  rev[p]=0;
  rnd[p]=rng();
  val[p]=v;
  sum[p]=v;
  return p;
}
void pushup(int p){
  siz[p]=siz[ls[p]]+siz[rs[p]]+1;
  sum[p]=sum[ls[p]]+sum[rs[p]]+val[p];
}
void pushdown(int p){ // 下传翻转标记
  if(!rev[p])return;
  swap(ls[p],rs[p]);
  if(ls[p])rev[ls[p]]^=1;
  if(rs[p])rev[rs[p]]^=1;
  rev[p]=0;
}
void splitv(int p,ll v,int&x,int&y){ // 值不超过 v 的给 x
  if(!p){
    x=y=0;
    return;
  }
  pushdown(p);
  if(val[p]<=v){
    x=p;
    splitv(rs[p],v,rs[p],y);
  }else{
    y=p;
    splitv(ls[p],v,x,ls[p]);
  }
  pushup(p);
}
void split(int p,int k,int&x,int&y){ // 前 k 个给 x
  if(!p){
    x=y=0;
    return;
  }
  pushdown(p);
  if(siz[ls[p]]<k){
    x=p;
    split(rs[p],k-siz[ls[p]]-1,rs[p],y);
  }else{
    y=p;
    split(ls[p],k,x,ls[p]);
  }
  pushup(p);
}
int merge(int x,int y){ // x 的元素都在 y 前面
  if(!x||!y)return x|y;
  if(rnd[x]<rnd[y]){
    pushdown(x);
    rs[x]=merge(rs[x],y);
    pushup(x);
    return x;
  }
  pushdown(y);
  ls[y]=merge(x,ls[y]);
  pushup(y);
  return y;
}
// ——— 用法 1: 按值的可重集合 ———
void insert(ll v){
  int x,y;
  splitv(root,v,x,y);
  root=merge(merge(x,newnode(v)),y);
}
void erase(ll v){ // 删一个 v, 没有则不变
  int x,y,z;
  splitv(root,v,x,z);
  splitv(x,v-1,x,y); // y 里全是 v
  y=merge(ls[y],rs[y]); // 删掉 y 的根
  root=merge(merge(x,y),z);
}
int getrank(ll v){ // 严格小于 v 的个数 + 1
  int x,y;
  splitv(root,v-1,x,y);
  int r=siz[x]+1;
  root=merge(x,y);
  return r;
}
ll kth(int k){ // 第 k 小, k 从 1 开始
  int p=root;
  while(1){
    if(k<=siz[ls[p]]){
      p=ls[p];
    }else if(k==siz[ls[p]]+1){
      return val[p];
    }else{
      k-=siz[ls[p]]+1;
      p=rs[p];
    }
  }
}
ll pre(ll v){return kth(getrank(v)-1);} // 小于 v 的最大值
ll nxt(ll v){return kth(getrank(v+1));} // 大于 v 的最小值
// ——— 用法 2: 序列, 位置从 1 开始 ———
void reverse(int l,int r){
  int x,y,z;
  split(root,r,x,z);
  split(x,l-1,x,y); // y 为 [l, r]
  rev[y]^=1;
  root=merge(merge(x,y),z);
}
ll rangeSum(int l,int r){
```

```
  int x,y,z;
  split(root,r,x,z);
  split(x,l-1,x,y);
  ll s=sum[y];
  root=merge(merge(x,y),z);
  return s;
}
void dfs(int p,vector<ll>&o){ // 中序遍历
  if(!p)return;
  pushdown(p);
  dfs(ls[p],o);
  o.push_back(val[p]);
  dfs(rs[p],o);
}
```

CartesianTree.h

Description: 笛卡尔树: 对序列 a[1..n] 建二叉树, 下标满足二叉搜索树性质, 即中序遍历为 1, 2, ..., n; 值满足堆性质. 默认小根: 根是最小值. 改大根: 把 build 里 while 的 a[stk[top]]>a[i] 改成 a[stk[top]]<a[i]. 相等: 靠左的相等元素是祖先, 靠右的在它的右子树里. 输入: 全局 n 和 a[1..n], 要求 n 至少为 1. build() 用单调栈 $O(n)$ 建树, 得到: root 为根; ls[i], rs[i] 为左右儿子, 0 表示没有; i 的子树恰好是下标区间 [L[i], R[i]], 也就是 a[i] 作为最小值能向两边 延伸的最大区间: 左边停在第一个不大于 a[i] 的位置, 右边停在第一个 严格小于 a[i] 的位置. 性质: [l, r] 最小值的位置, 相等取最左, 等于 l 和 r 在树上的 LCA. 常见用法: 以 a[i] 为最小值的子数组个数为 (i - L[i] + 1) * (R[i] - i + 1), 每个子数组恰好算一次, 最小值相等时算给最左的; 直方图最大矩形为 a[i] * (R[i] - L[i] + 1) 的最大值; 已排好序的键配随机优先级, 以优先级为 a 建树, 就是 $O(n)$ 建 treap. 注意: 多组数据直接重新 build, 会覆盖 ls, rs, L, R.

Usage: `cin >> n;`

```
for (int i = 1; i <= n; i++) cin >> a[i];
build();
int rt = root; // 最小值的位置, 有多个时取最左
ll s = 0; // 所有子数组的最小值之和
for (int i = 1; i <= n; i++)
s += a[i] * (i - L[i] + 1) * (R[i] - i + 1);
```

Time: $\mathcal{O}(N)$

e93964, 24 lines

```
const int N=2e5+5;
int n,root;
ll a[N];
int ls[N],rs[N],L[N],R[N];
int stk[N];
void build(){
  int top=0;
  for(int i=1;i<=n;i++){
    ls[i]=0;
    rs[i]=0;
    int last=0; // 最后弹出的点, 成为 i 的左儿子
    while(top&&a[stk[top]]>a[i]){
      R[stk[top]]=i-1;
      last=stk[top];
      top--;
    }
    ls[i]=last;
    if(top)rs[stk[top]]=i;
    L[i]=top?stk[top]+1:1;
    stk[++top]=i;
  }
  for(int j=1;j<=top;j++)R[stk[j]]=n;
  root=stk[1];
}
```

FenwickTree.h

Description: 树状数组, 下标 1..n, 全局 n 为长度. 两个版本互相独立, 只抄需要的. 版本 1, 单点加, 前缀和: add(x, v) 令 a[x] 加 v; ask(x) 返回 a[1..x] 的和, a[l..r] 的和为 ask(r) - ask(l - 1); search(s) 返回最小的 x 使 a[1..x] 的和不小于 s, 不存在返回 n + 1, 要求所有 a 非负. 常用于求第 s 小: a[v] 为值 v 的个数, search(s) 即第 s 小的值. 版本 2, 区间加, 区间和: rangeAdd(l, r, v) 令 a[l..r] 都加 v; rangeSum(l, r) 返回 a[l..r] 的和. 原理: 维护差分 d[i] 的两个树状数组, c1 存 d[i], c2 存 i * d[i]. 初值: 版本 1 对每个 i 调用 add(i, a[i]); 版本 2 调用 rangeAdd(i, i, a[i]). 注意: 要求 $n < 2^{21}$; 下标不能为 0; 多组数据把 c 或 c1, c2 的 1..n 清 0.

Usage: `n = ...; // 多组数据先把 c[1..n] 清 0`

```
for (int i = 1; i <= n; i++) add(i, a[i]); // 建树
add(x, v); // a[x] += v
ll s = ask(r) - ask(l - 1); // a[l..r] 的和
int x = search(k); // 前缀和第一次不小于 k 的位置
// 版本 2:  先把 c1[1..n], c2[1..n] 清 0
for (int i = 1; i <= n; i++) rangeAdd(i, i, a[i]);
rangeAdd(l, r, v); ll t = rangeSum(l, r);
```

Time: 每次操作 $\mathcal{O}(\log N)$

17bf60, 39 lines

```
const int N=2e5+5; // 要求 n < 2^21
int n;
ll c[N];
void add(int x,ll v){
  for(;x<=n;x+=x&-x)c[x]+=v;
}
ll ask(int x){ // a[1..x] 的和
  ll r=0;
  for(;x;x-=x&-x)r+=c[x];
  return r;
}
int search(ll s){ // 不存在返回 n + 1
  int x=0;
  for(int i=1<<20;i;i>>=1){
    if(x+i<=n&&c[x+i]<s){
      x+=i;
      s-=c[x];
    }
  }
  return x+1;
}

ll c1[N],c2[N]; // 差分 d[i] 和 i * d[i]
void radd(int x,ll v){
  for(int i=x;i<=n;i+=i&-i){
    c1[i]+=v;
    c2[i]+=v*x;
  }
}
void rangeAdd(int l,int r,ll v){
  radd(l,v);
  radd(r+1,-v);
}
ll rsum(int x){ // a[1..x] 的和
  ll s=0;
  for(int i=x;i;i-=i&-i)s+=(x+1)*c1[i]-c2[i];
  return s;
}
ll rangeSum(int l,int r){return rsum(r)-rsum(l-1);}
```

FenwickTree2d.h

Description: 二维树状数组, 矩阵 a[1..n][1..m], 全局 n, m 为行数和列数. add(x, y, v): a[x][y] 加 v. ask(x, y): 子矩阵 [1..x][1..y] 的和. query(x1, y1, x2, y2): 子矩阵 [x1..x2][y1..y2] 的和, 要求 x1 不超过 x2, y1 不超过 y2. 独立使用: 变量名和一维的 FenwickTree.h 相同, 不要同时抄. 注意: 内存为 $N \cdot M$ 个 ll, 默认 1005 * 1005 约 8MB. 坐标很大但修改点事先已知时, 改用离线: 离散化 + 一维树状数组扫描线. 多组数据把 c[1..n][1..m] 清 0.

Usage: `n = ..., m = ...; // 多组数据先清空 c[1..n][1..m]`

```
add(x, y, v); // a[x][y] += v
ll s = query(x1, y1, x2, y2); // 子矩阵和
```

Time: 每次操作 $\mathcal{O}(\log N \log M)$

0d921f, 18 lines

```
const int N=1005,M=1005;
int n,m;
ll c[N][M];
void add(int x,int y,ll v){
  for(int i=x;i<=n;i+=i&-i){
    for(int j=y;j<=m;j+=j&-j)c[i][j]+=v;
  }
}
ll ask(int x,int y){ // [1..x][1..y] 的和
  ll r=0;
  for(int i=x;i;i-=i&-i){
    for(int j=y;j;j-=j&-j)r+=c[i][j];
  }
  return r;
}
ll query(int x1,int y1,int x2,int y2){
  return ask(x2,y2)-ask(x1-1,y2)-ask(x2,y1-1)+ask(x1-1,y1-1);
}
```

MoQueries.h

Description: 普通莫队: 离线回答 q 个区间询问 a[l..r], 要求能 $O(1)$ 地把一个数加入或移出当前区间, 并维护当前答案 cur. 输入: 全局 n, q, a[1..n]; 第 i 个询问存为 qs[i] = {l, r, i}, 下标从 1 开始. 只需改 add(i) 和 del(i): 把 a[i] 加入或移出当前区间并更新 cur; 下面的 add, del 是例子: 区间不同值个数, 要求 a 已离散化到 1..n. solve() 之后 ans[i] 为第 i 个询问的答案, 顺序和输入一致. 块长 $B = n/\sqrt{q}$, 奇数块按 r 升序, 偶数块按 r 降序, 减少 R 来回跑. 注意: ans 和 cur 是 ll; solve 会打乱 qs 的顺序; 多组数据清空 cnt 等自己维护的数组和 cur.

Usage: `cin >> n >> q;`

```
for (int i = 1; i <= n; i++) cin >> a[i]; // 先离散化
for (int i = 1; i <= q; i++) {
cin >> qs[i].l >> qs[i].r;
qs[i].id = i;
}
solve();
for (int i = 1; i <= q; i++) cout << ans[i] << endl;
```

Time: $\mathcal{O}\left(n\sqrt{q}\right)$ 次 add 和 del

9aee6c, 31 lines

```
const int N=2e5+5;
int n,q,B,a[N];
struct Query{int l,r,id;};
Query qs[N];
ll ans[N],cur;
int cnt[N]; // 例子用: 每个值的出现次数
void add(int i){ // a[i] 进入当前区间
  if(cnt[a[i]]==0)cur++;
  cnt[a[i]]++;
}
void del(int i){ // a[i] 离开当前区间
  cnt[a[i]]--;
  if(cnt[a[i]]==0)cur--;
}
bool cmp(const Query&x,const Query&y){
  if(x.l/B!=y.l/B)return x.l<y.l;
  if((x.l/B)&1)return x.r<y.r;
  return x.r>y.r;
}
void solve(){
  B=max(1,(int)(n/sqrt(q)));
  sort(qs+1,qs+q+1,cmp);
  int L=1,R=0; // 当前区间 [L, R), 开始为空
  for(int i=1;i<=q;i++){
    while(L>qs[i].l)add(--L);
    while(R<qs[i].r)add(++R);
    while(L<qs[i].l)del(L++);
    while(R>qs[i].r)del(R--);
    ans[qs[i].id]=cur;
  }
}
```

数值计算 (4)

4.1 多项式与线性递推

Polynomial.h

Description: 实系数多项式的基本操作. 多项式存成 vector<double> a, 下标从 0 开始, $a[i]$ 为 x^i 的系数, 次数为 a.size()-1. peval(a, x): 用秦九韶 (Horner) 法求 $p(x)$, 返回 double. pderiv(a): 原地求导, a 的长度减 1. pdivroot(a, x0): 原地除以 $(x-x_0)$ (综合除法), a 的长度减 1, 余数 $p(x_0)$ 被丢弃, 所以 x_0 应是根 (用于求出一个根后降次). 注意 a 不能为空; 浮点误差随次数增大.

Usage:
```
vector<double> a={2,-3,1}; // x^2-3x+2
double y=peval(a,0.5); // y=0.75
pdivroot(a,1); // 除以 x-1, a 变为 {-2,1}
pderiv(a); // a 变为 {1}
```

Time: 均为 $\mathcal{O}(n)$, n 为次数

5ee889, 23 lines

```
double peval(const vector<double>&a,double x){
  double res=0;
  for(int i=(int)a.size()-1;i>=0;i--){
    res=res*x+a[i];
  }
  return res;
}
void pderiv(vector<double>&a){
  for(int i=1;i<(int)a.size();i++){
    a[i-1]=i*a[i];
  }
  a.pop_back();
}
void pdivroot(vector<double>&a,double x0){
  double b=a.back();
  a.back()=0;
  for(int i=(int)a.size()-2;i>=0;i--){
    double c=a[i];
    a[i]=a[i+1]*x0+b;
    b=c;
  }
  a.pop_back();
}
```

PolyInterpolate.h

Description: 多项式插值 (牛顿插值): 给定 n 个点 (x_i, y_i), x_i 互不相同, 求唯一的不超过 $n-1$ 次的多项式 p 使 $p(x_i)=y_i$. 输入两个长为 n 的 vector<double> (下标从 0 开始), 返回长为 n 的系数 数组 a, $p(x)=a_0+a_1x+\cdots+a_{n-1}x^{n-1}$ (可用 Polynomial.h 的 peval 求值). 浮点运算, n 大时误差大; 可以自己选点时取 $x_k=c\cos(k\pi/(n-1))$ 更稳定. 模素数下只求一个点的值请用拉格朗日插值公式, 不要用这个.

Usage:
```
vector<double> x={0,1,2},y={1,2,5};
vector<double> a=interpolate(x,y); // a={1,0,1}
// 即 p(x)=1+x^2
```

Time: $\mathcal{O}(n^2)$

d9ae92, 21 lines

```
vector<double>interpolate(vector<double>x,vector<double>y){
  int n=x.size();
  vector<double>res(n),tmp(n);
  // 均差: 结束后 y[k] 为 f[x_0..x_k]
  for(int k=0;k<n-1;k++){
    for(int i=k+1;i<n;i++){
      y[i]=(y[i]-y[k])/(x[i]-x[k]);
    }
  }
  // tmp 为 (x-x_0)...(x-x_{k-1}) 的系数
  double last=0;
  tmp[0]=1;
  for(int k=0;k<n;k++){
    for(int i=0;i<n;i++){
      res[i]+=y[k]*tmp[i];
      swap(last,tmp[i]);
      tmp[i]-=last*x[k];
    }
  }
  return res;
}
```

BerlekampMassey.h

Description: Berlekamp-Massey: 给出数列 $s_0, s_1, \ldots$ 的前若干项 (模素数 MOD), 求最短的线性递推式. 若真实递推阶数为 n, 至少要给前 $2n$ 项. 返回 vector<ll> C (下标从 0 开始, 长度即阶数 L), 满足对所有 $i \geq L$: $s_i=\sum_{j=0}^{L-1} C_j s_{i-j-1} \pmod{MOD}$. 数列全为 0 时返回空. 典型用法: 暴力或 DP 算出前几百项, BM 猜出递推式, 再用 LinearRecurrence 求第 k 项. 输入须在 $[0, MOD)$ 内; MOD 必须是素数 (要求逆元).

Usage:
```
vector<ll> s={0,1,1,3,5,11};
vector<ll> C=BM(s); // C={1,2}
// 即 s[i]=s[i-1]+2*s[i-2]
```

Time: $\mathcal{O}(n^2)$, n 为给出的项数

79eda4, 51 lines

```
const ll MOD=998244353; // 素数模, 可改
ll qpow(ll a,ll b){
  ll r=1;
  a%=MOD;
  while(b){
    if(b&1){
      r=r*a%MOD;
    }
    a=a*a%MOD;
    b>>=1;
  }
  return r;
}
vector<ll>BM(vector<ll>s){
  int n=s.size(),L=0,m=0;
  if(n==0){
    return {};
  }
  vector<ll>C(n),B(n),T;
  C[0]=1;
  B[0]=1;
  ll b=1;
  for(int i=0;i<n;i++){
    m++;
    ll d=s[i]%MOD; // 当前递推式的偏差
    for(int j=1;j<=L;j++){
      d=(d+C[j]*s[i-j])%MOD;
    }
    if(d==0){
      continue;
    }
    T=C;
    ll coef=d*qpow(b,MOD-2)%MOD;
    for(int j=m;j<n;j++){
      C[j]=(C[j]+MOD-coef*B[j-m]%MOD)%MOD;
    }
    if(2*L>i){
      continue;
    }
    L=i+1-L;
    B=T;
    b=d;
    m=0;
  }
  C.resize(L+1);
  C.erase(C.begin());
  for(auto&x:C){
    x=(MOD-x)%MOD;
  }
  return C;
}
```

LinearRecurrence.h

Description: 线性递推求第 k 项: 已知 n 阶递推 $S_i=\sum_{j=0}^{n-1} tr_j S_{i-j-1}$ 和前 n 项 $S_0..S_{n-1}$, 求 $S_k \bmod MOD$ (k 从 0 开始, 可到 10^{18}). 参数: S 至少 n 项 (多给无妨), tr 长为 n (系数顺序与 BM 的返回值相同), 都下标从 0 开始, 值在 $[0, MOD)$ 内. tr 为空时数列全 0, 返回 0. 原理: 求 x^k 模特征多项式, 再与前 n 项做线性组合, 比矩阵快速幂快. MOD 来自 BerlekampMassey.h (单独使用时可删掉 include 自己定义 MOD).

Usage:
```
ll f=linearRec({0,1},{1,1},10); // 第 10 个斐波那契数 55
vector<ll> s=...; // 暴力算出的前若干项
vector<ll> tr=BM(s); // 猜递推式
ll v=linearRec(s,tr,k); // 第 k 项
```

Time: $\mathcal{O}(n^2 \log k)$

"BerlekampMassey.h" 048892, 38 lines

```
// a*b 对特征多项式取模, a, b 长 n+1
vector<ll>combine(const vector<ll>&a,const vector<ll>&b,
    const vector<ll>&tr){
  int n=tr.size();
  vector<ll>res(2*n+1);
  for(int i=0;i<=n;i++){
    for(int j=0;j<=n;j++){
      res[i+j]=(res[i+j]+a[i]*b[j])%MOD;
    }
  }
  for(int i=2*n;i>n;i--){
    for(int j=0;j<n;j++){
      res[i-1-j]=(res[i-1-j]+res[i]*tr[j])%MOD;
    }
  }
  res.resize(n+1);
  return res;
}
ll linearRec(vector<ll>S,vector<ll>tr,ll k){
  int n=tr.size();
  if(n==0){
    return 0;
  }
  vector<ll>pol(n+1),e(n+1);
  pol[0]=1;
  e[1]=1;
  for(k++;k;k>>=1){
    if(k&1){
      pol=combine(pol,e,tr);
    }
    e=combine(e,e,tr);
  }
  ll res=0;
  for(int i=0;i<n;i++){
    res=(res+pol[i+1]*S[i])%MOD;
  }
  return res;
}
```

4.2 最优化与积分

GoldenSectionSearch.h

Description: 黄金分割搜索: 在区间 $[a, b]$ 上求单峰函数 f 的最小值点. 要求 f 在 $[a, b]$ 上只有一个局部极小值且没有局部极大值 (先降后升). 被优化的函数是全局函数 double f(double x), 自己实现 (模板里只有声明). gss(a, b) 返回最小值点 x (误差约 eps $=10^{-7}$), 最小值为 f(x). 求最大值: 把代码中的 $f_1 < f_2$ 改成 $f_1 > f_2$. 比三分法快: 每轮只新算一次 f. 需要更高精度可改小 eps. 整数自变量的版本见 Various 章 TernarySearch.h.

Usage:
```
double f(double x){return 4+x+.3*x*x;} // 自己写
double xm=gss(-1000,1000); // xm 约为 -5/3
double ym=f(xm); // 最小值
```

Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$ 78dea0, 26 lines

```
double f(double x); // 要最小化的函数，在别处自己实现
double gss(double a,double b){
  // r 要足够精确，才能保证 a < x1 < x2 < b
  double r=(sqrt(5.0)-1)/2;
  double eps=1e-7;
  double x1=b-r*(b-a);
  double x2=a+r*(b-a);
  double f1=f(x1);
  double f2=f(x2);
  while(b-a>eps){
    if(f1<f2){ // 求最大值改为 f1 > f2
      b=x2;
      x2=x1;
      f2=f1;
      x1=b-r*(b-a);
      f1=f(x1);
    }else{
      a=x1;
      x1=x2;
      f1=f2;
      x2=a+r*(b-a);
      f2=f(x2);
    }
  }
  return a;
}
```

IntegrateAdaptive.h

Description: 自适应 Simpson 积分: 求 $\int_l^r f(x)\,dx$. 被积函数是全局函数 `double f(double x)`, 自己实现 (模板里只有声明). `integrate(l, r, eps)` 返回积分值, eps 为允许误差 (默认 10^{-8}). 每段先用 Simpson 公式估计, 左右两半之和与整段相差不超过 $15 \cdot eps$ 就停止, 否则递归两半 (eps 减半). 开头强制细分 6 层, 防止在对称函数上一开始误判收敛. 适用于光滑函数; 有尖点或间断时在这些点处把区间拆开分别积分更准. 二重积分: 写 g(x) 对 y 积分, 需要再复制一份本模板并改名.
Usage:
```
double f(double x){return sin(x);} // 自己写
double s=integrate(0,acos(-1)); // s 约为  2
double t=integrate(0,1,1e-10); // 指定精度
```
Time: 与 f 的光滑程度有关, 一般几千次 f 调用 144717, 18 lines

```
double f(double x); // 被积函数，在别处自己实现
double simpson(double l,double r){
  double m=(l+r)/2;
  return (f(l)+4*f(m)+f(r))*(r-l)/6;
}
// S 为整段的 Simpson 值，dep 为至少还要细分的层数
double asr(double l,double r,double eps,double S,int dep){
  double m=(l+r)/2;
  double L=simpson(l,m);
  double R=simpson(m,r);
  if(dep<=0&&(fabs(L+R-S)<=15*eps||r-l<1e-10)){
    return L+R+(L+R-S)/15;
  }
  return asr(l,m,eps/2,L,dep-1)+asr(m,r,eps/2,R,dep-1);
}
double integrate(double l,double r,double eps=1e-8){
  return asr(l,r,eps,simpson(l,r),6);
}
```

Simplex.h

Description: 单纯形法解线性规划: 在约束 $Ax \le b$, $x \ge 0$ 下最大化 $c^T x$. 有 m 个约束, n 个变量, 全部下标从 1 开始: 填全局数组 $A[1..m][1..n]$, $b[1..m]$ (b 可为负), $c[1..n]$, 然后调用 `simplex(m, n)`. 返回值: 无可行解返回 -INF; 目标无上界返回 INF; 否则返回最大值, 一组最优解在 $x[1..n]$. 多组数据直接重新填 A, b, c 即可. 变形: 最小化把 c 取负 (答案再取负); $\ge$ 约束两边乘 -1; 等式拆成 $\le$ 和 $\ge$; 无符号限制的变量拆成 $x' - x''$. 浮点运算, eps $= 10^{-8}$, 不保证数值稳定; 用 Bland 规则防止循环. $b \ge 0$ 时 $x = 0$ 可行, 跳过第一阶段, 更快.

Usage:
```
m=3; n=2; // 约束数，变量数
// 填 A[1..m][1..n], b[1..m], c[1..n]
double v=simplex(m,n);
if(v==-INF) puts("Infeasible");
else if(v==INF) puts("Unbounded");
else ...  // v 为最大值，最优解在 x[1..n]
```
Time: $\mathcal{O}\left(nm \cdot \#pivots\right)$, 最坏指数级, 实际很快 8d715a, 100 lines

```
const int MM=205; // 约束个数上限
const int NN=205; // 变量个数上限
const double eps=1e-8;
const double INF=1e18;
double A[MM][NN],b[MM],c[NN],x[NN]; // 输入输出
int m,n;
int bv[MM],nv[NN]; // 每行的基变量，每列的非基变量
double D[MM][NN]; // 单纯形表，下标从 0 开始
bool lessp(double a,int i,double b,int j){
  return make_pair(a,i)<make_pair(b,j);
}
void pivot(int r,int s){
  double inv=1/D[r][s];
  for(int i=0;i<m+2;i++){
    if(i==r||fabs(D[i][s])<=eps)
      continue;
    double t=D[i][s]*inv;
    for(int j=0;j<n+2;j++)
      D[i][j]-=D[r][j]*t;
    D[i][s]=D[r][s]*t;
  }
  for(int j=0;j<n+2;j++)
    if(j!=s)
      D[r][j]*=inv;
  for(int i=0;i<m+2;i++)
    if(i!=r)
      D[i][s]*=-inv;
  D[r][s]=inv;
  swap(bv[r],nv[s]);
}
bool run(int phase){
  int x=m+phase-1;
  while(1){
    int s=-1; // 入基列 (Bland 规则防循环)
    for(int j=0;j<=n;j++){
      if(nv[j]==-phase)
        continue;
      if(s==-1||lessp(D[x][j],nv[j],D[x][s],nv[s]))
        s=j;
    }
    if(D[x][s]>=-eps)
      return true;
    int r=-1; // 出基行
    for(int i=0;i<m;i++){
      if(D[i][s]<=eps)
        continue;
      if(r==-1||lessp(D[i][n+1]/D[i][s],bv[i],
          D[r][n+1]/D[r][s],bv[r]))
        r=i;
    }
    if(r==-1)
      return false;
    pivot(r,s);
  }
}
double simplex(int _m,int _n){
  m=_m;
  n=_n;
  for(int i=0;i<m+2;i++)
    for(int j=0;j<n+2;j++)
      D[i][j]=0;
  for(int i=0;i<m;i++){
    for(int j=0;j<n;j++)
      D[i][j]=A[i+1][j+1];
    bv[i]=n+i;
    D[i][n]=-1;
    D[i][n+1]=b[i+1];
  }
  for(int j=0;j<n;j++){
    nv[j]=j;
    D[m][j]=-c[j+1];
  }
  nv[n]=-1;
  D[m+1][n]=1;
  int r=0;
  for(int i=1;i<m;i++)
    if(D[i][n+1]<D[r][n+1])
      r=i;
  if(D[r][n+1]<-eps){ // x=0 不可行，先找可行解
    pivot(r,n);
    if(!run(2)||D[m+1][n+1]<-eps)
      return -INF;
    for(int i=0;i<m;i++){
      if(bv[i]!=-1)
        continue;
      int s=0;
      for(int j=1;j<=n;j++)
        if(lessp(D[i][j],nv[j],D[i][s],nv[s]))
          s=j;
      pivot(i,s);
    }
  }
  bool ok=run(1);
  for(int j=1;j<=n;j++)
    x[j]=0;
  for(int i=0;i<m;i++)
    if(bv[i]<n)
      x[bv[i]+1]=D[i][n+1];
  return ok?D[m][n+1]:INF;
}
```

4.3 矩阵

Determinant.h

Description: 求 $n \times n$ 实矩阵的行列式 (高斯消元, 列主元). 填全局数组 $a[1..n][1..n]$ (下标从 1 开始), `det(n)` 返回行列式. 消成上三角, 行列式 = 主对角线之积, 每交换一次两行变号. a 会被破坏; 多组数据重新填即可. 奇异矩阵可能返回很小的数而不是 0. 结果可能很大 (溢出 double 范围时考虑改 long double 或取对数). 整数矩阵求模意义下的行列式 (如矩阵树定理) 用 IntDeterminant.
Usage:
```
// 填 a[1..n][1..n]
double d=det(n);
```
Time: $\mathcal{O}\left(n^3\right)$ 479605, 26 lines

```
const int N=505; // 矩阵大小上限
double a[N][N];
double det(int n){
  double res=1;
  for(int i=1;i<=n;i++){
    int p=i; // 选绝对值最大的主元
    for(int j=i+1;j<=n;j++)
      if(fabs(a[j][i])>fabs(a[p][i]))
        p=j;
    if(a[p][i]==0)
      return 0;
    if(p!=i){
      swap(a[p],a[i]);
      res=-res;
    }
    res*=a[i][i];
```

```
    for(int j=i+1;j<=n;j++){
      double t=a[j][i]/a[i][i];
      if(t==0)
        continue;
      for(int k=i;k<=n;k++)
        a[j][k]-=t*a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: 求整数矩阵的行列式模 p. 填全局数组 $a[1..n][1..n]$ (下标从 1 开始, 可以有负数, 绝对值不超过 long long), detMod(n, p) 返回 $\det \bmod p$, 在 $[0,p)$ 内. 用辗转相除消元: 对第 i 列, 每个下面的行 j 与第 i 行反复做 第 i 行减去 $\lfloor a_{ii}/a_{ji} \rfloor$ 倍的第 j 行再交换, 直到 $a_{ji}=0$ (像求 gcd). 不需要逆元, 所以 p 不必是素数. 要求 p^2 不超过 long long (即 $p<3\cdot 10^9$). a 会被破坏. 常用于矩阵树定理: 填拉普拉斯矩阵去掉一行一列.

Usage: `// 填 a[1..n][1..n]`
```
ll d=detMod(n,1000000007);
```
Time: $\mathcal{O}\left(n^2(n+\log p)\right)$

8fbf91, 23 lines

```
const int N=505; // 矩阵大小上限
ll a[N][N];
ll detMod(int n,ll p){
  ll res=1;
  for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)
      a[i][j]=(a[i][j]%p+p)%p;
  for(int i=1;i<=n;i++){
    for(int j=i+1;j<=n;j++){
      while(a[j][i]){ // 辗转相除, 把 a[j][i] 消成 0
        ll t=a[i][i]/a[j][i];
        for(int k=i;k<=n;k++)
          a[i][k]=(a[i][k]+(p-t)*a[j][k])%p;
        swap(a[i],a[j]);
        res=p-res;
      }
    }
    if(a[i][i]==0)
      return 0;
    res=res*a[i][i]%p;
  }
  return res%p;
}
```

SolveLinear.h

Description: 高斯-约旦消元解实系数线性方程组: n 个方程, m 个未知数. 填全局数组 (下标从 1 开始): $a[i][1..m]$ 为第 i 个方程的系数, $a[i][m+1]$ 为等号右边的常数. 调用 gauss(n, m): 无解返回 -1; 否则返回系数矩阵的秩 r, 一组解在 $x[1..m]$. $r=m$ 时解唯一; $r<m$ 时有无穷多解, 返回的是自由元都取 0 的那组, pos[1..r] 为各主元所在的列 (其余列是自由元). 按列选绝对值最大的主元, 绝对值小于 $\text{eps}=10^{-9}$ 视为 0; 数据范围大时按需调 eps. a 会被破坏, 多组数据重新填即可.

Usage: `// 填 a[1..n][1..m+1]`
```
int r=gauss(n,m);
if(r==-1) puts("No Solution");
else if(r<m) puts("Infinite"); // 仍可用 x[1..m]
else ...  // 唯一解 x[1..m]
```
Time: $\mathcal{O}\left(nm\cdot\min(n,m)\right)$

2e9731, 34 lines

```
const int N=505; // 方程数, 未知数上限
const double eps=1e-9;
double a[N][N],x[N];
int pos[N];
int gauss(int n,int m){
  int r=1; // 当前处理到第 r 行
  for(int c=1;c<=m&&r<=n;c++){
    int p=r; // 选第 c 列绝对值最大的主元
    for(int i=r+1;i<=n;i++)
      if(fabs(a[i][c])>fabs(a[p][c]))
        p=i;
    if(fabs(a[p][c])<eps)
      continue; // 自由元
    swap(a[p],a[r]);
    for(int i=1;i<=n;i++){
      if(i==r||a[i][c]==0)
        continue;
      double t=a[i][c]/a[r][c];
      for(int j=c;j<=m+1;j++)
        a[i][j]-=t*a[r][j];
    }
    pos[r]=c;
    r++;
  }
  r--; // 秩
  for(int i=r+1;i<=n;i++)
    if(fabs(a[i][m+1])>eps)
      return -1; // 出现 0 = 非 0
  for(int j=1;j<=m;j++)
    x[j]=0;
  for(int i=1;i<=r;i++)
    x[pos[i]]=a[i][m+1]/a[i][pos[i]];
  return r;
}
```

SolveLinearBinary.h

Description: 解异或方程组 (模 2 线性方程组): n 个方程, m 个 0/1 未知数, 第 i 个方程为 $a_{i1}x_1\oplus\cdots\oplus a_{im}x_m=a_{i,m+1}$. 填全局 bitset 数组 (下标从 1 开始): a[i] 的第 $1..m$ 位为系数, 第 $m+1$ 位 为右边常数. 调用 gaussXor(n, m): 无解返回 -1; 否则返回秩 r, 一组解在 $x[1..m]$ (自由元取 0), 解的个数为 2^{m-r}. 用 bitset 整行异或, 比普通高斯消元快 64 倍. 需要 $m+1<N$. a 会被破坏; 多组数据先把 a[1..n] 清空 (reset).

Usage: `for(int i=1;i<=n;i++) a[i].reset(); // 清空`
```
a[i][j]=1; // 第 i 个方程含 x_j
a[i][m+1]=1; // 第 i 个方程右边为 1
int r=gaussXor(n,m); // -1 无解, 否则解在 x[1..m]
```
Time: $\mathcal{O}\left(nm\cdot\min(n,m)/64\right)$

bd93d8, 28 lines

```
const int N=1005; // 方程数, 未知数上限
bitset<N>a[N];
int x[N],pos[N];
int gaussXor(int n,int m){
  int r=1; // 当前处理到第 r 行
  for(int c=1;c<=m&&r<=n;c++){
    int p=r;
    while(p<=n&&!a[p][c])
      p++;
    if(p>n)
      continue; // 自由元
    swap(a[p],a[r]);
    for(int i=1;i<=n;i++)
      if(i!=r&&a[i][c])
        a[i]^=a[r];
    pos[r]=c;
    r++;
  }
  r--; // 秩
  for(int i=r+1;i<=n;i++)
    if(a[i][m+1])
      return -1; // 出现 0 = 1
  for(int j=1;j<=m;j++)
    x[j]=0;
  for(int i=1;i<=r;i++)
    x[pos[i]]=a[i][m+1];
  return r;
}
```

MatrixInverse.h

Description: 求 $n\times n$ 实矩阵的逆 (高斯-约旦消元, 列主元). 填全局数组 $a[1..n][1..n]$ (下标从 1 开始), 调用 inverse(n): 可逆返回 true, 且 $a[1..n][1..n]$ 变为逆矩阵; 奇异返回 false (a 被破坏). 做法: 右边拼上单位阵 $a[1..n][n+1..2n]$, 把左半边消成单位阵, 右半边即逆. 主元绝对值小于 10^{-12} 视为奇异 (数值大小差很多时按需调整). 模素数版本: double 改 ll, 主元选任意非 0 元, 除以 v 改为乘 qpow(v, MOD-2), 所有加减乘取模. 模 p^k: 先求模 p 的逆 B, 反复令 $B=B(2I-AB)$, 每次模数的指数 k 翻倍.

Usage: `// 填 a[1..n][1..n]`
```
if(inverse(n)) ...  // 逆矩阵在 a[1..n][1..n]
else puts("No Solution"); // 奇异
```
Time: $\mathcal{O}\left(n^3\right)$

023113, 30 lines

```
const int N=505; // 矩阵大小上限
double a[N][N*2];
bool inverse(int n){
  for(int i=1;i<=n;i++)
    for(int j=n+1;j<=2*n;j++)
      a[i][j]=(j-n==i);
  for(int i=1;i<=n;i++){
    int p=i; // 选绝对值最大的主元
    for(int j=i+1;j<=n;j++)
      if(fabs(a[j][i])>fabs(a[p][i]))
        p=j;
    if(fabs(a[p][i])<1e-12)
      return false;
    swap(a[i],a[p]);
    double t=1/a[i][i];
    for(int j=1;j<=2*n;j++)
      a[i][j]*=t;
    for(int k=1;k<=n;k++){
      if(k==i||a[k][i]==0)
        continue;
      double v=a[k][i];
      for(int j=1;j<=2*n;j++)
        a[k][j]-=v*a[i][j];
    }
  }
  for(int i=1;i<=n;i++)
    for(int j=1;j<=n;j++)
      a[i][j]=a[i][j+n];
  return true;
}
```

4.4 FFT, NTT, FWT

FastFourierTransform.h

Description: FFT 多项式乘法 (迭代版, 自写复数 cp). 系数数组下标从 0 开始, a_i 为 x^i 的系数. conv(a, b): 输入两个 vector<ll> (可为负), 返回 $c_k=\sum_{i+j=k}a_ib_j$, 长度 $|a|+|b|-1$ (任一为空返回空), 结果四舍五入成整数. fft(f, 1) 正变换, fft(f, -1) 逆变换 (已除以 n), f 的长度必须是 2 的幂 (conv 会自动补 0). conv 技巧: 把 a 放实部, b 放虚部, 平方后虚部为 $2ab$, 只需 2 次 FFT. 精度: $(\sum a_i^2+\sum b_i^2)\log_2 n<9\cdot 10^{14}$ 时安全 (结果约 10^{14} 以内; 随机数据可更大). 超出请用 NTT (模 998244353) 或 FFTMod (任意模). 多个数组连乘: 对每个做 fft, 逐点相乘, 再一次 fft(-1).

Usage: `vector<ll> a={1,2},b={1,3}; // 1+2x, 1+3x`
```
vector<ll> c=conv(a,b); // c={1,5,6}
```
Time: $\mathcal{O}\left(n\log n\right)$, $n=|a|+|b|$, $n=2^{22}$ 约 1s

fbd18f, 61 lines

```
struct cp{ // 复数 x+yi
  double x,y;
};
cp operator+(cp a,cp b){return {a.x+b.x,a.y+b.y};}
cp operator-(cp a,cp b){return {a.x-b.x,a.y-b.y};}
cp operator*(cp a,cp b){
  return {a.x*b.x-a.y*b.y,a.x*b.y+a.y*b.x};
}
```

```
const double PI=acos(-1.0);
void fft(vector<cp>&f,int t){ // t=1 正变换, t=-1 逆变换
  int n=f.size();
  vector<int>rev(n); // 位逆序置换
  for(int i=1;i<n;i++){
    rev[i]=rev[i>>1]>>1;
    if(i&1)
      rev[i]|=n>>1;
  }
  for(int i=0;i<n;i++)
    if(i<rev[i])
      swap(f[i],f[rev[i]]);
  for(int len=1;len<n;len<<=1){
    vector<cp>w(len); // 单位根直接用 cos, sin 算, 精度高
    for(int j=0;j<len;j++)
      w[j]={cos(PI*j/len),t*sin(PI*j/len)};
    for(int i=0;i<n;i+=len<<1){
      for(int j=0;j<len;j++){
        cp u=f[i+j];
        cp v=w[j]*f[i+j+len];
        f[i+j]=u+v;
        f[i+j+len]=u-v;
      }
    }
  }
  if(t==-1){
    for(auto&z:f){
      z.x/=n;
      z.y/=n;
    }
  }
}
vector<ll>conv(const vector<ll>&a,const vector<ll>&b){
  if(a.empty()||b.empty())
    return {};
  int m=a.size()+b.size()-1;
  int n=1;
  while(n<m)
    n<<=1;
  vector<cp>f(n);
  for(int i=0;i<(int)a.size();i++)
    f[i].x=a[i];
  for(int i=0;i<(int)b.size();i++)
    f[i].y=b[i];
  fft(f,1);
  for(auto&z:f)
    z=z*z; // (a+bi)^2 的虚部为 2ab
  fft(f,-1);
  vector<ll>c(m);
  for(int i=0;i<m;i++)
    c[i]=llround(f[i].y/2);
  return c;
}
```

FastFourierTransformMod.h

Description: 任意模数多项式乘法 (拆系数 FFT, 共 4 次 FFT, 用 FastFourierTransform.h 的 cp 和 fft). `convMod(a, b)`: 输入两个 vector<ll>, 下标从 0 开始, 值必须在 $[0, MOD)$ 内; 返回 $c_k = \sum_{i+j=k} a_i b_j \bmod MOD$, 长度 $|a| + |b| - 1$. MOD 改成题目的模数 (任意正整数, 须 $< 2^{30}$, 不必是素数). 做法: 拆 $a = a_1 2^{15} + a_0$, b 同理; 令 $P = a_1 + ia_0$, $Q = b_1 + ib_0$, 由 P 的共轭对称分离出 a_1, a_0 的 DFT, 再算 $X = A_1 Q$, $Y = A_0 Q$, 逆变换后实部虚部分别为 $a_1 b_1, a_1 b_0, a_0 b_1, a_0 b_0$, 合并取模. 精度: $n \log_2 n \cdot MOD < 8.6 \cdot 10^{14}$ 时安全, 实际 $n = 10^6$, MOD $= 10^9 + 7$ 没问题.

Usage: `vector<ll> a={1,2},b={3,MOD-1}; // 系数在 [0,MOD)`
`vector<ll> c=convMod(a,b); // c[k]=sum a[i]b[k-i] mod MOD`

Time: $\mathcal{O}(n \log n)$, $n = |a| + |b|$, 约为普通 FFT 的两倍

"FastFourierTransform.h" a83f73, 39 lines

```
const ll MOD=1000000007; // 模数, 按题目修改
vector<ll>convMod(const vector<ll>&a,const vector<ll>&b){
  if(a.empty()||b.empty())
    return {};
  int m=a.size()+b.size()-1;
  int n=1;
  while(n<m)
    n<<=1;
  vector<cp>P(n),Q(n),X(n),Y(n);
  // P = a1 + i a0, Q = b1 + i b0
  for(int i=0;i<(int)a.size();i++)
    P[i]={double(a[i]>>15),double(a[i]&32767)};
  for(int i=0;i<(int)b.size();i++)
    Q[i]={double(b[i]>>15),double(b[i]&32767)};
  fft(P,1);
  fft(Q,1);
  for(int i=0;i<n;i++){
    int j=(n-i)&(n-1);
    cp cj={P[j].x,-P[j].y}; // P[n-i] 的共轭
    cp s=P[i]+cj; // 2 FFT(a1)
    cp d=P[i]-cj; // 2i FFT(a0)
    cp A1={s.x/2,s.y/2};
    cp A0={d.y/2,-d.x/2};
    X[i]=A1*Q[i];
    Y[i]=A0*Q[i];
  }
  fft(X,-1);
  fft(Y,-1);
  // X = a1b1 + i a1b0, Y = a0b1 + i a0b0
  vector<ll>c(m);
  for(int i=0;i<m;i++){
    ll v11=llround(X[i].x)%MOD;
    ll v10=llround(X[i].y)%MOD;
    ll v01=llround(Y[i].x)%MOD;
    ll v00=llround(Y[i].y)%MOD;
    c[i]=((v11<<30)%MOD+((v10+v01)<<15)+v00)%MOD;
  }
  return c;
}
```

NumberTheoreticTransform.h

Description: NTT 多项式乘法, 模 $998244353 = 119 \cdot 2^{23} + 1$, 原根 $g = 3$, 不依赖其他文件. `nttConv(a, b)`: 输入两个 vector<ll>, 下标从 0 开始, 值必须在 $[0, MOD)$ 内 (负数先加 MOD); 返回 $c_k = \sum_{i+j=k} a_i b_j \bmod MOD$, 长度 $|a| + |b| - 1$ (须 $\leq 2^{23}$). 任一为空返回空. `ntt(f, 1)` 正变换, `ntt(f, -1)` 逆变换 (已乘 $1/n$), f 长度须为 2 的幂. 换模数只改 MOD 和 G, 其他 NTT 模数 (原根都是 3): 469762049, 167772161, 1004535809. 要精确整数结果用 FFT; 任意模数 (如 $10^9 + 7$) 用 FFTMod.

Usage: `vector<ll> a={1,2},b={1,3}; // 1+2x, 1+3x`
`vector<ll> c=nttConv(a,b); // c={1,5,6}`

Time: $\mathcal{O}(n \log n)$, $n = |a| + |b|$

8cdaa6, 62 lines

```
const ll MOD=998244353;
const ll G=3; // 原根
ll qpow(ll a,ll b){
  ll r=1;
  a%=MOD;
  while(b){
    if(b&1)
      r=r*a%MOD;
    a=a*a%MOD;
    b>>=1;
  }
  return r;
}
void ntt(vector<ll>&f,int t){ // t=1 正变换, t=-1 逆变换
  int n=f.size();
  vector<int>rev(n); // 位逆序置换
  for(int i=1;i<n;i++){
    rev[i]=rev[i>>1]>>1;
    if(i&1)
      rev[i]|=n>>1;
  }
  for(int i=0;i<n;i++)
    if(i<rev[i])
      swap(f[i],f[rev[i]]);
  for(int len=1;len<n;len<<=1){
    ll wn=qpow(G,(MOD-1)/(len<<1)); // 2len 次单位根
    if(t==-1)
      wn=qpow(wn,MOD-2);
    for(int i=0;i<n;i+=len<<1){
      ll w=1;
      for(int j=0;j<len;j++){
        ll u=f[i+j];
        ll v=w*f[i+j+len]%MOD;
        f[i+j]=(u+v)%MOD;
        f[i+j+len]=(u-v+MOD)%MOD;
        w=w*wn%MOD;
      }
    }
  }
  if(t==-1){
    ll inv=qpow(n,MOD-2);
    for(auto&v:f)
      v=v*inv%MOD;
  }
}
vector<ll>nttConv(vector<ll>a,vector<ll>b){
  if(a.empty()||b.empty())
    return {};
  int m=a.size()+b.size()-1;
  int n=1;
  while(n<m)
    n<<=1;
  a.resize(n);
  b.resize(n);
  ntt(a,1);
  ntt(b,1);
  for(int i=0;i<n;i++)
    a[i]=a[i]*b[i]%MOD;
  ntt(a,-1);
  a.resize(m);
  return a;
}
```

FWT.h

Description: 快速沃尔什变换 (FWT), 求位运算卷积 $c_k = \sum_{i \oplus j = k} a_i b_j$, $\oplus$ 为 or, and, xor. 数组为 vector<ll>, 下标从 0 开始, 两个数组等长, 长度 n 必须是 2 的幂 (值域 $[0, 2^K)$ 就开 $n = 2^K$, 不足补 0). 值用 ll 精确计算, 不取模 (注意结果别超过 long long). `orConv / andConv / xorConv(a, b)`: 返回卷积结果, 长度 n, 不改原数组. `fwtOr / fwtAnd / fwtXor(a, t)`: 原地变换, $t = 1$ 正变换, $t = -1$ 逆变换. 含义: or 正变换即子集和 $\hat{a}_k = \sum_{i \subseteq k} a_i$; and 正变换即 超集和 $\hat{a}_k = \sum_{i \supseteq k} a_i$; xor 正变换 $\hat{a}_k = \sum_i (-1)^{|i \cap k|} a_i$. 正变换后逐点相乘再逆变换, 就是对应的卷积. 取模版本: 每次加减后取模 (减法加 MOD), xor 逆变换的除以 2 改为乘 $inv2 = (MOD + 1)/2$.

Usage: `int n=1<<K; // 值域 [0,2^K)`
`vector<ll> a(n),b(n); // 填 a, b`
`vector<ll> c=xorConv(a,b); // c[k]=sum a[i]b[j], i xor j=k`
`vector<ll> d=orConv(a,b); // i or j=k`
`vector<ll> e=andConv(a,b); // i and j=k`
`fwtOr(a,1); // a 原地变为子集和, fwtOr(a,-1) 变回`

Time: $\mathcal{O}(n \log n)$

d9ce04, 55 lines

```
void fwtOr(vector<ll>&a,int t){
  int n=a.size();
  for(int len=1;len<n;len<<=1)
    for(int i=0;i<n;i+=len<<1)
      for(int j=i;j<i+len;j++)
```

```
        a[j+len]+=t*a[j];
}
void fwtAnd(vector<ll>&a,int t){
  int n=a.size();
  for(int len=1;len<n;len<<=1)
    for(int i=0;i<n;i+=len<<1)
      for(int j=i;j<i+len;j++)
        a[j]+=t*a[j+len];
}
void fwtXor(vector<ll>&a,int t){
  int n=a.size();
  for(int len=1;len<n;len<<=1){
    for(int i=0;i<n;i+=len<<1){
      for(int j=i;j<i+len;j++){
        ll u=a[j];
        ll v=a[j+len];
        a[j]=u+v;
        a[j+len]=u-v;
        if(t==-1){
          a[j]/=2;
          a[j+len]/=2;
        }
      }
    }
  }
}
vector<ll>orConv(vector<ll>a,vector<ll>b){
  fwtOr(a,1);
  fwtOr(b,1);
  for(int i=0;i<(int)a.size();i++)
    a[i]*=b[i];
  fwtOr(a,-1);
  return a;
}
vector<ll>andConv(vector<ll>a,vector<ll>b){
  fwtAnd(a,1);
  fwtAnd(b,1);
  for(int i=0;i<(int)a.size();i++)
    a[i]*=b[i];
  fwtAnd(a,-1);
  return a;
}
vector<ll>xorConv(vector<ll>a,vector<ll>b){
  fwtXor(a,1);
  fwtXor(b,1);
  for(int i=0;i<(int)a.size();i++)
    a[i]*=b[i];
  fwtXor(a,-1);
  return a;
}
```

MaxPlusConvolution.h

Description: $(\max,+)$ 卷积: 给定 $a_0..a_{n-1}$ (任意) 和 $b_0..b_{m-1}$ (凹, 即差分 $b_{j+1}-b_j$ 不增), 求 $c_k=\max_{i+j=k}(a_i+b_j)$, $k=0..n+m-2$. 一般的 $(\max,+)$ 卷积只能 $O(nm)$; b 凹时最优决策单调, 可以分治. **决策单调**: 对 k, 合法的 i 满足 $\max(0,k-m+1)\le i\le\min(k,n-1)$; 记 $opt(k)$ 为取到最大值的 i (有多个时取最小的). 设 $k<k'$, $i<i'$, 下标 $k-i'<k-i, k'-i'<k'-i$ 且两边和相等, 由凹性 $b_{k-i'}+b_{k'-i}\le b_{k-i}+b_{k'-i'}$, 所以 $(a_{i'}+b_{k'-i'})-(a_i+b_{k'-i})\ge(a_{i'}+b_{k-i'})-(a_i+b_{k-i})$: 若 i' 在 k 处严格优于 i, 则在 k' 处也严格优于 i. 故 $opt(k)$ 随 k 不降. **分治**: mpdc(l, r, optl, optr) 求 $c_l..c_r$, 已知它们的 opt 都在 $[optl,optr]$. 取 $mid=(l+r)/2$, 在 $[optl,optr]$ 与合法范围的交里扫 i 求 c_{mid} 和 $opt(mid)$ (严格大于才更新, 即取最小的 i), 再递归 mpdc(l, mid-1, optl, opt) 和 mpdc(mid+1, r, opt, optr). 每层总扫描量 $O(n+m)$, 共 $O(\log)$ 层. 若凹的是 a, 交换两个参数即可. 求 $(\min,+)$ 卷积要求 b 凸, 全部取负即可. **两个都凹**时更简单, $O(n+m)$: $c_0=a_0+b_0$, 把两者的差分数组按从大到小 归并 (同闵可夫斯基和), 依次累加得到 $c_1,c_2,\ldots$.

Usage: `vector<ll> a={1,5,2},b={0,3,4}; // b 凹, a, b 非空`
`vector<ll> c=maxPlus(a,b); // c={1,5,8,9,6}`

Time: $\mathcal{O}\left((n+m)\log(n+m)\right)$ c83ddf, 26 lines

```
vector<ll>va,vb,vc;
// 求 c[l..r], 已知最优的 i 在 [optl, optr] 中
void mpdc(int l,int r,int optl,int optr){
  if(l>r)
    return;
  int mid=(l+r)/2;
  int best=-1;
  int lo=max(optl,mid-(int)vb.size()+1); // 合法的 i 范围
  int hi=min({optr,mid,(int)va.size()-1});
  for(int i=lo;i<=hi;i++){ // 取最左的最优 i
    ll v=va[i]+vb[mid-i];
    if(best==-1||v>vc[mid]){
      vc[mid]=v;
      best=i;
    }
  }
  mpdc(l,mid-1,optl,best);
  mpdc(mid+1,r,best,optr);
}
vector<ll>maxPlus(const vector<ll>&a,const vector<ll>&b){
  va=a;
  vb=b;
  vc.assign(a.size()+b.size()-1,0);
  mpdc(0,(int)vc.size()-1,0,(int)a.size()-1);
  return vc;
}
```

卷积用法

下面的代码直接调用上面的 conv (FFT), nttConv (NTT), xorConv 等 (FWT). 下标都从 0 开始.

- **多项式乘法 / 两数组卷积**: $c_k=\sum_i a_ib_{k-i}$, 长度 $|a|+|b|-1$. 差卷积 $\sum_i a_ib_{i+k}$ 把 b 反转.

```
// a[i], b[i] 为 x^i 的系数
vector<ll> c=nttConv(a,b); // 模 998244353, 先取模
vector<ll> d=conv(a,b); // 精确整数 (FFT), 可有负数
// 差卷积 e[k]=sum a[i]*b[i+k], k 从 -(n-1) 到 m-1:
vector<ll> rb(b.rbegin(),b.rend()); // m=b.size()
vector<ll> t=conv(a,rb); // e[k] 即 t[m-1-k]
```

- **大整数乘法**: 低位在前存数字, 卷积后进位.

```
string bigMul(string s,string t){ // 十进制, 无负号
  int n=s.size();
  int m=t.size();
  vector<ll> a(n),b(m); // 低位在前
  for(int i=0;i<n;i++)
    a[i]=s[n-1-i]-'0';
  for(int i=0;i<m;i++)
    b[i]=t[m-1-i]-'0';
  vector<ll> c=conv(a,b);
  string r;
  ll carry=0;
  for(int i=0;i<(int)c.size()||carry;i++){
    if(i<(int)c.size())
      carry+=c[i];
    r+=char('0'+carry%10);
    carry/=10;
  }
  while(r.size()>1&&r.back()=='0') // 去前导 0
    r.pop_back();
  reverse(r.begin(),r.end());
  return r;
}
```

- **两数之和计数**: 求 $v_i+v_j=k$ $(i<j)$ 的对数, 值域 $[0,V]$. 自己卷自己得到有序对且含 $i=j$.

```
vector<ll> pairSum(vector<int>&v,int V){
  vector<ll> cnt(V+1);
  for(int x:v)
    cnt[x]++;
  vector<ll> f=conv(cnt,cnt); // 有序对, 含 i=j
  for(int x:v)
    f[2*x]--; // 去掉 i=j
  for(auto&y:f)
    y/=2; // 变成无序对 i<j
  return f; // f[k] 为对数, 长度 2V+1
}
```

- **带通配符的字符串匹配**: $\star$ 匹配任意字符. 字符值 1..26, 通配符为 0, 位置 i 匹配当且仅当 $\sum_j(p_j-s_{i+j})^2p_js_{i+j}=0$. 展开后是三个卷积 ($p$ 反转). NTT 下和可能恰为模数的倍数, 误判概率极小; 介意可再换一个模数验证.

```
ll wv(char c){return c=='*'?0:c-'a'+1;} // 通配符为 0
vector<int> wildMatch(string s,string p){
  int n=s.size();
  int m=p.size();
  vector<int> res; // 匹配的起点, 下标从 0 开始
  if(m>n)
    return res;
  vector<ll> S[4],P[4]; // S[k][i] 为 s_i 的 k 次方
  for(int k=1;k<=3;k++){
    S[k].resize(n);
    P[k].resize(m);
  }
  for(int i=0;i<n;i++){
    ll v=wv(s[i]);
    S[1][i]=v;
    S[2][i]=v*v;
    S[3][i]=v*v*v;
  }
  for(int j=0;j<m;j++){ // P 为 p 反转
    ll v=wv(p[m-1-j]);
    P[1][j]=v;
    P[2][j]=v*v;
    P[3][j]=v*v*v;
  }
  vector<ll> c1=nttConv(P[3],S[1]);
  vector<ll> c2=nttConv(P[2],S[2]);
  vector<ll> c3=nttConv(P[1],S[3]);
  for(int i=0;i+m<=n;i++){
    int t=i+m-1;
    if((c1[t]-2*c2[t]+c3[t]+2*MOD)%MOD==0)
      res.push_back(i);
  }
  return res;
}
```

- **位运算卷积**: 求 $v_i\oplus v_j=k$ $(i<j)$ 的对数 (值在 $[0,2^K)$), 以及子集和.

```
vector<ll> pairXor(vector<int>&v,int K){
  vector<ll> cnt(1<<K);
  for(int x:v)
    cnt[x]++;
  vector<ll> f=xorConv(cnt,cnt); // 有序对, 含 i=j
  f[0]-=v.size(); // i=j 时 xor 为 0
  for(auto&y:f)
    y/=2;
  return f;
```

```
}
// or 同理: f=orConv(cnt,cnt), 对每个 x 做 f[x]--, 再除 2
// 子集和: g[S]=sum a[T], T 取遍 S 的子集
vector<ll> g=a;
fwtOr(g,1); // fwtOr(g,-1) 还原
// 超集和: fwtAnd(g,1)
```

数论 (5)

5.1 模运算

ModularArithmetic.h

Description: 模质数 MOD 下的基本运算和组合数. qpow(a, b) 返回 $a^b \bmod MOD$ (a 可以为负, $b \geq 0$); inv(a) 用费马小定理返回 a^{MOD-2}, 即 a 的逆元, 要求 MOD 是质数且 a 不是 MOD 的倍数; init(n) 预处理 fac[i] $= i!$ 和 ifac[i] $= (i!)^{-1}$ ($0 \leq i \leq n$, $n < N$ 且 $n <$ MOD); C(n, m) 返回 $\binom{n}{m} \bmod MOD$, $m < 0$ 或 $m > n$ 时返回 0, 要求 n 不超过 init 的参数. 加减法直接写 (a + b) % MOD 和 (a - b + MOD) % MOD. 注意: 本文件的 inv 函数和 ModInverse 的 inv 数组同名, 不要同时复制.

Usage: `init(1000000); // 只需调用一次, 参数取题目里 n 的最大值`
`ll c = C(5, 2); // 10`
`ll x = inv(3); // 3 的逆元`
`ll y = qpow(2, 100); // 2 的 100 次方模 MOD`

Time: init $\mathcal{O}(n)$, C $\mathcal{O}(1)$, qpow 和 inv $\mathcal{O}(\log MOD)$

412f2b, 25 lines

```
const ll MOD=998244353; // 或 1e9+7, 必须是质数
const int N=1e6+5;
ll fac[N],ifac[N];
ll qpow(ll a,ll b){ // a 的 b 次方模 MOD
  ll r=1;
  a%=MOD;
  if(a<0)a+=MOD;
  while(b){
    if(b&1)r=r*a%MOD;
    a=a*a%MOD;
    b>>=1;
  }
  return r;
}
ll inv(ll a){return qpow(a,MOD-2);}
void init(int n){
  fac[0]=1;
  for(int i=1;i<=n;i++)fac[i]=fac[i-1]*i%MOD;
  ifac[n]=inv(fac[n]);
  for(int i=n;i>=1;i--)ifac[i-1]=ifac[i]*i%MOD;
}
ll C(int n,int m){
  if(m<0||m>n)return 0;
  return fac[n]*ifac[m]%MOD*ifac[n-m]%MOD;
}
```

ModInverse.h

Description: 线性时间求 $1, 2, \ldots, n$ 模质数 p 的逆元, 要求 p 是质数且 $n < p$. 原理: 由 $p = \lfloor p/i \rfloor \cdot i + (p \bmod i)$ 得 $i^{-1} = -\lfloor p/i \rfloor \cdot (p \bmod i)^{-1}$. initInv(n, p) 之后 inv[i] 为 i 模 p 的逆元 ($1 \leq i \leq n$, $n < N$). 只求单个数的逆元用快速幂 a^{p-2} (ModPow) 或 exgcd (euclid, 模数可以不是质数). 注意: 数组名 inv 和 ModularArithmetic 的 inv 函数同名.

Usage: `initInv(1000000, 1000000007); // 多组数据不用重新算`
`ll x = inv[5]; // 5 的逆元, 5 * x % p 等于 1`

Time: $\mathcal{O}(n)$

ccdefc, 6 lines

```
const int N=1e6+5;
ll inv[N];
void initInv(int n,ll p){
  inv[1]=1;
  for(int i=2;i<=n;i++)inv[i]=(p-p/i)*inv[p%i]%p;
}
```

ModPow.h

Description: 快速幂, 返回 $a^b \bmod p$, 结果在 $[0, p)$ 内. a 可以为负或大于 p, $b \geq 0$, $p \geq 1$ ($p = 1$ 时返回 0). 乘法用 long long, 要求 p^2 不超过 long long, 即 p 不超过约 $3 \cdot 10^9$; 模数到 10^{18} 时用 ModMulLL 的 powmod. p 是质数时 qpow(a, p - 2, p) 就是 a 的逆元.

Usage: `ll r = qpow(2, 10, 1000); // 24`
`ll iv = qpow(a, p - 2, p); // p 为质数时 a 的逆元`

Time: $\mathcal{O}(\log b)$

3df302, 11 lines

```
ll qpow(ll a,ll b,ll p){
  ll r=1%p;
  a%=p;
  if(a<0)a+=p;
  while(b){
    if(b&1)r=r*a%p;
    a=a*a%p;
    b>>=1;
  }
  return r;
}
```

ModLog.h

Description: 离散对数 (BSGS, 大步小步). 解 $a^x \equiv b \pmod m$, 返回最小的 $x \geq 0$, 无解返回 -1. bsgs(a, b, m, c) 解 $c \cdot a^x \equiv b \pmod m$, 要求 $\gcd(a, m) = 1$, c 默认为 1. 设 $t = \lfloor \sqrt{m} \rfloor + 1$, 把 $b \cdot a^j$ ($0 \leq j < t$) 存进哈希表, 再枚举 $i = 1..t$ 找 $c \cdot a^{it} = b \cdot a^j$, 得 $x = it - j$. exbsgs(a, b, m) 对任意 a, m 解 $a^x \equiv b \pmod m$: 每次取 $d = \gcd(a, m)$, 若 $d \nmid b$ 无解, 否则 b, m 同除以 d, 系数 c 乘上 a/d, x 的偏移 k 加一, 直到 a, m 互质后用 bsgs 求剩下的部分, 答案加 k. 注意 $x = 0$ 时 $a^0 = 1$, 所以 $b \equiv 1$ 时答案是 0. 所有输入取模后计算, a, b 可以大于 m, 但不能为负. m 不超过约 10^9, 否则乘法要改 __int128.

Usage: `ll x = bsgs(3, 13, 17); // gcd(a, m) = 1, 3^x = 13 (mod 17)`
`ll y = exbsgs(a, b, m); // 任意 a, m, 无解为 -1`
`// a 的阶, 即最小的 k 使 a^k = 1, 为 bsgs(a, 1, m, a) + 1`

Time: $\mathcal{O}\left(\sqrt{m}\right)$

e11eef, 41 lines

```
ll bsgs(ll a,ll b,ll m,ll c=1){
  a%=m;
  b%=m;
  c%=m;
  if(c==b)return 0;
  ll t=sqrtl(m)+1;
  ll at=1; // a^t
  unordered_map<ll,ll>h; // b * a^j 对应的最大的 j
  ll cur=b;
  for(ll j=0;j<t;j++){
    h[cur]=j;
    cur=cur*a%m;
    at=at*a%m;
  }
  // 找 c * a^(i t) = b * a^j, 则 x = i t - j
  cur=c;
  for(ll i=1;i<=t;i++){
    cur=cur*at%m;
    if(h.count(cur))return i*t-h[cur];
  }
  return -1;
}
ll exbsgs(ll a,ll b,ll m){
  a%=m;
  b%=m;
  if(b==1%m)return 0;
  ll k=0,c=1;
  while(true){
    ll d=__gcd(a,m);
    if(d==1)break;
    if(b%d)return -1;
    b/=d;
    m/=d;
    k++;
    c=c*(a/d)%m;
    if(c==b)return k; // x = k
  }
  ll x=bsgs(a,b,m,c);
  if(x<0)return -1;
  return x+k;
}
```

ModMulLL.h

Description: 模数 m 很大 (到 long long 上限, 约 $9.2 \cdot 10^{18}$) 时的乘法和快速幂. 直接 a * b % m 会溢出, 这里先转成 __int128 再取模. mulmod(a, b, m) 返回 $ab \bmod m$, 要求 $0 \leq a, b < m$ (负数先调到 $[0, m)$). powmod(a, b, m) 返回 $a^b \bmod m$, 要求 $a \geq 0$, $b \geq 0$, $m \geq 1$. 注意: MillerRabin 里有完全相同的两个函数, 两个文件都复制时删掉一份.

Usage: `ll m = 1000000000000000003; // 大模数`
`ll x = mulmod(a, b, m); // a * b mod m`
`ll y = powmod(a, b, m); // a 的 b 次方 mod m`

Time: mulmod $\mathcal{O}(1)$, powmod $\mathcal{O}(\log b)$

3657ac, 11 lines

```
ll mulmod(ll a,ll b,ll m){return (__int128)a*b%m;}
ll powmod(ll a,ll b,ll m){
  ll r=1%m;
  a%=m;
  while(b){
    if(b&1)r=mulmod(r,a,m);
    a=mulmod(a,a,m);
    b>>=1;
  }
  return r;
}
```

ModSqrt.h

Description: 二次剩余 (Cipolla 算法): 求 x 使 $x^2 \equiv n \pmod p$, p 必须是质数. cipolla(n, p) 无解时返回 -1; 有解时返回两个根 $x, p - x$ 中较小的那个 ($n \equiv 0$ 时返回 0). n 可以为负或大于 p. 判断有无解用欧拉判别法: $n^{(p-1)/2} \equiv 1$ 才有解. 做法: 找 a 使 $w = a^2 - n$ 不是二次剩余, 在 $\mathbb{F}_p[\sqrt{w}]$ 里算 $x = (a + \sqrt{w})^{(p+1)/2}$. cmul 是这个域里的乘法, $(x + y\sqrt{w})(a + b\sqrt{w})$. 全部乘法用 __int128, p 可到 10^{18}.

Usage: `ll x = cipolla(2, 7); // 3, 因为 3 * 3 = 9 = 2 (mod 7)`
`if(x == -1) ... // 无解`
`else { ll y = p - x; } // 另一个根 (x = 0 时只有一个根)`

Time: 期望 $\mathcal{O}(\log p)$

7135a8, 38 lines

```
typedef __int128 L;
ll pw(ll a,ll b,ll p){
  ll r=1%p;
  a%=p;
  while(b){
    if(b&1)r=(L)r*a%p;
    a=(L)a*a%p;
    b>>=1;
  }
  return r;
}
ll W; // w = a^2 - n
// (x + y sqrt w) 乘上 (a + b sqrt w), 结果存回 x, y
void cmul(ll&x,ll&y,ll a,ll b,ll p){
  ll nx=((L)x*a+(L)y*b%p*W)%p;
  ll ny=((L)x*b+(L)y*a)%p;
  x=nx;
  y=ny;
}
ll cipolla(ll n,ll p){
  n%=p;
  if(n<0)n+=p;
```

```
  if(n==0||p==2)return n;
  if(pw(n,(p-1)/2,p)!=1)return -1; // 欧拉判别法
  ll a=0;
  while(true){
    a++;
    W=((L)a*a%p-n+p)%p;
    if(pw(W,(p-1)/2,p)==p-1)break;
  }
  ll x=1,y=0;
  ll bx=a,by=1;
  for(ll e=(p+1)/2;e;e>>=1){
    if(e&1)cmul(x,y,bx,by,p);
    cmul(bx,by,bx,by,p);
  }
  return min(x,p-x);
}
```

5.2 素性测试

FastEratosthenes.h

Description: 线性筛 (欧拉筛). 一次求出 $[1,n]$ 内所有质数, 欧拉函数 φ 和莫比乌斯函数 μ. 每个合数 $x=i\cdot p$ 只被它的最小质因子 p 筛一次, 所以是 $O(n)$. 调用 sieve(n) 后 ($n<N$): pri[1..cnt] 是从小到大的质数, cnt 是个数; $x\geq 2$ 时 vis[x] 为 0 表示 x 是质数 (vis[0], vis[1] 也是 0, 自己特判); phi[x] 和 mu[x] 对 $1\leq x\leq n$ 有效. 其他积性函数也这样筛: 分 $i \bmod p=0$ (p 整除 i) 和 $i \bmod p\neq 0$ (p 与 i 互质) 两种情况. 注意: sieve 只能调用一次 (cnt 不清零), 取题目所有数据的最大 n. $N=10^7$ 时四个数组约 130MB, 不需要的 phi 或 mu 可以删掉.

Usage: `sieve(10000000); // 只调用一次`
`for(int i = 1; i <= cnt; i++) pri[i]; // 第 i 个质数`
`bool isp = x >= 2 && !vis[x]; // x 是不是质数`
`phi[x]; mu[x]; // 欧拉函数, 莫比乌斯函数`

Time: $\mathcal{O}(n)$ b490bf, 25 lines

```
const int N=1e7+5;
int pri[N],cnt,phi[N],mu[N];
bool vis[N];
void sieve(int n){
  phi[1]=1;
  mu[1]=1;
  for(int i=2;i<=n;i++){
    if(!vis[i]){
      pri[++cnt]=i;
      phi[i]=i-1;
      mu[i]=-1;
    }
    for(int j=1;j<=cnt&&(ll)i*pri[j]<=n;j++){
      int x=i*pri[j];
      vis[x]=1;
      if(i%pri[j]==0){ // pri[j] 是 i 的最小质因子
        phi[x]=phi[i]*pri[j];
        mu[x]=0;
        break;
      }
      phi[x]=phi[i]*(pri[j]-1);
      mu[x]=-mu[i];
    }
  }
}
```

MillerRabin.h

Description: 确定性 Miller-Rabin 素性测试, 判断一个 long long 范围内的数是不是质数. isPrime(n) 对所有 $n\leq 9.2\cdot 10^{18}$ 都正确 (用前 12 个质数 2..37 作底); $n<2$ (包括负数) 返回 false. 做法: 写 $n-1=d\cdot 2^s$, 对每个底 a 看 $a^d, a^{2d},\ldots$ 中是否出现 1 或 $n-1$. 乘法用 __int128 (mulmod), 自带 mulmod 和 powmod, 与 ModMulLL 相同, 一起复制时删掉一份. 只判断 10^7 以内的数时用线性筛更快.

Usage: `if(isPrime(1000000007)) ... // true`
`bool b = isPrime(n); // n 为 long long`

Time: 12 次快速幂, $\mathcal{O}(12\log n)$ 46bed6, 32 lines

```
ll mulmod(ll a,ll b,ll m){return (__int128)a*b%m;}
ll powmod(ll a,ll b,ll m){
  ll r=1%m;
  a%=m;
  while(b){
    if(b&1)r=mulmod(r,a,m);
    a=mulmod(a,a,m);
    b>>=1;
  }
  return r;
}
bool isPrime(ll n){
  if(n<3||n%2==0)return n==2;
  ll d=n-1;
  int s=0;
  while(d%2==0){ // n - 1 = d * 2^s
    d/=2;
    s++;
  }
  for(ll a:{2,3,5,7,11,13,17,19,23,29,31,37}){
    if(n==a)return 1;
    ll x=powmod(a,d,n);
    if(x==1||x==n-1)continue;
    bool ok=0;
    for(int i=1;i<s&&!ok;i++){
      x=mulmod(x,x,n);
      if(x==n-1)ok=1;
    }
    if(!ok)return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho 质因数分解, 分解 long long 范围内的 n (到 $9.2\cdot 10^{18}$). factor(n, f) 把 n 的所有质因子 (含重复, 顺序任意) 追加到 f 里; $n=1$ 时不加任何数. rho(n) 对合数 n 返回一个大于 1 的因子 (有时返回 n 本身, factor 会重试): 用 $f(x)=x^2+c$ (c 随机) 倍增找环, 把 $|t-s|$ 连乘起来每 127 步做一次 gcd. 依赖 MillerRabin 的 isPrime 和 mulmod. 随机数生成器叫 prng, 不和 UniformRandom 的 rng 冲突. 只需要分解 10^{12} 以内的数时, 试除到 $\sqrt{n}$ 更简单.

Usage: `vector<ll> f;`
`factor(2299, f); // f = {11, 19, 11}, 顺序任意`
`sort(f.begin(), f.end()); // 排序后相同的质因子挨在一起`

Time: 期望 $\mathcal{O}\left(n^{1/4}\log n\right)$

"MillerRabin.h" d73a15, 32 lines

```
mt19937_64 prng(20261107);
// n 为合数, 返回 n 的一个因子, 可能是 n
ll rho(ll n){
  if(n%2==0)return 2;
  ll c=prng()%(n-1)+1;
  ll s=0,t=0,v=1;
  for(int g=1;;g<<=1){
    for(int k=1;k<=g;k++){
      t=((__int128)t*t+c)%n;
      v=mulmod(v,abs(t-s),n);
      if(k%127==0){
        ll d=__gcd(v,n);
        if(d>1)return d;
      }
    }
    ll d=__gcd(v,n);
    if(d>1)return d;
    s=t;
    v=1;
  }
}
void factor(ll n,vector<ll>&f){
  if(n==1)return;
  if(isPrime(n)){
    f.push_back(n);
    return;
  }
  ll d=n;
  while(d==n)d=rho(n);
  factor(d,f);
  factor(n/d,f);
}
```

5.3 整除

SqrtIntervals.h

Description: 整除分块 (数论分块). 当 i 从 1 到 n 时 $\lfloor n/i\rfloor$ 只有 $O(\sqrt{n})$ 种取值, 把值相同的 i 合成一段一起算. 下取整: 段的左端 l, 值 $q=\lfloor n/l\rfloor$, 右端 $r=\lfloor n/q\rfloor$. 上取整: 值 $q=\lceil n/l\rceil$, $q>1$ 时右端 $r=\lfloor (n-1)/(q-1)\rfloor$, $q=1$ 时 $r=n$. 两个数一起分块 (如求 $\sum\lfloor n/i\rfloor\lfloor m/i\rfloor$): $r=\min(\lfloor n/\lfloor n/l\rfloor\rfloor, \lfloor m/\lfloor m/l\rfloor\rfloor)$, 循环到 $\min(n,m)$. 下面 floorSum(n) 求 $\sum_{i=1}^{n}\lfloor n/i\rfloor g(i)$, ceilSum(n) 求 $\sum_{i=1}^{n}\lceil n/i\rceil g(i)$, 其中 sg(l, r) $=\sum_{i=l}^{r} g(i)$ 要按题目改 (例子里 $g=1$; 莫比乌斯反演时 $g=\mu$, 用前缀和). n 到 10^{12} 左右时注意结果是否超过 long long, 必要时取模.

Usage: `ll s = floorSum(n); // sum floor(n/i) g(i)`
`ll t = ceilSum(n); // sum ceil(n/i) g(i)`
`for(ll l = 1, r; l <= n; l = r + 1) { // 直接写循环`
`r = n / (n / l); // [l, r] 内 n/i 都相同`
`}`

Time: $\mathcal{O}\left(\sqrt{n}\right)$ 次 sg ea83cb, 20 lines

```
ll sg(ll l,ll r){return r-l+1;} // g 的区间和, 这里 g = 1
ll floorSum(ll n){
  ll res=0;
  for(ll l=1,r;l<=n;l=r+1){
    ll q=n/l;
    r=n/q; // [l, r] 内 n/i 都等于 q
    res+=q*sg(l,r);
  }
  return res;
}
ll ceilSum(ll n){
  ll res=0;
  for(ll l=1,r;l<=n;l=r+1){
    ll q=(n+l-1)/l;
    if(q>1)r=(n-1)/(q-1); // [l, r] 内上取整都是 q
    else r=n;
    res+=q*sg(l,r);
  }
  return res;
}
```

euclid.h

Description: 扩展欧几里得. exgcd(a, b, x, y) 求一组整数 x,y 使 $ax+by=\gcd(a,b)$, 返回 $g=\gcd(a,b)$. 要求 $a,b\geq 0$ 且不全为 0. 得到的解满足 $|x|\leq b$, $|y|\leq a$, 中间不会溢出. 只要 gcd 时直接用 __gcd(a, b). 求逆元: $\gcd(a,m)=1$ 时 exgcd(a, m, x, y) 的 x 就是 a 模 m 的逆元 (调成 $[0,m)$), m 不必是质数. 解 $ax+by=c$: 有解当且仅当 g 整除 c; 一组解为 $x_0=x\cdot c/g$, $y_0=y\cdot c/g$ (注意溢出, 可用 __int128); 通解 $x=x_0+k\cdot b/g$, $y=y_0-k\cdot a/g$; 最小非负 x 为 x_0 对 b/g 取模.

Usage: `ll x, y;`
`ll g = exgcd(a, b, x, y); // a * x + b * y = g`
`exgcd(a, m, x, y); // a, m 互质`
`ll iv = (x % m + m) % m; // a 模 m 的逆元`

Time: $\mathcal{O}(\log\min(a,b))$ 80f39b, 10 lines

```
ll exgcd(ll a,ll b,ll&x,ll&y){
  if(b==0){
```

```
    x=1;
    y=0;
    return a;
  }
  ll d=exgcd(b,a%b,y,x);
  y-=a/b*x;
  return d;
}
```

CRT.h

Description: 中国剩余定理, 解同余方程组 $x \equiv a_i \pmod{m_i}$, $i = 1..n$, 数组下标从 1 开始. a_i 可以为负或大于 m_i, 要求 $m_i \geq 1$. 两个函数都返回 $[0, L)$ 内最小的非负解, 其中 $L = \mathrm{lcm}(m_1, \ldots, m_n)$, 所有解为 $x + kL$. 要求 L 不超过 long long (约 $9.2 \cdot 10^{18}$), 中间乘法用 __int128. crt(a, m, n): m_i 必须两两互质, $x = \sum a_i M_i M_i^{-1}$, $M = \prod m_i$, $M_i = M/m_i$, 逆元模 m_i 求. excrt(a, m, n): m_i 任意, 无解返回 -1. 逐个合并: 设前面的解为 $x \bmod M$, 解 $Mt \equiv a_i - x \pmod{m_i}$ (用 exgcd, $g = \gcd(M, m_i)$ 不整除右边则无解), 新解 $x + Mt$, 新模数 $\mathrm{lcm}(M, m_i)$. 不确定是否互质时直接用 excrt. 依赖 euclid.h 的 exgcd.

Usage:
```
ll a[N], m[N]; int n; // 读入 a[1..n], m[1..n]
ll x = crt(a, m, n); // m 两两互质
ll y = excrt(a, m, n); // 任意 m, 无解为 -1
```

Time: $\mathcal{O}(n \log L)$

"euclid.h" 029370, 29 lines

```
ll crt(ll*a,ll*m,int n){
  ll M=1,x=0;
  for(int i=1;i<=n;i++)M*=m[i];
  for(int i=1;i<=n;i++){
    ll Mi=M/m[i];
    ll r=(a[i]%m[i]+m[i])%m[i];
    ll t,y;
    exgcd(Mi%m[i],m[i],t,y); // t 为 Mi 模 m[i] 的逆元
    t=(t%m[i]+m[i])%m[i];
    t=(__int128)t*r%m[i];
    x=(x+(__int128)Mi*t)%M;
  }
  return x;
}
ll excrt(ll*a,ll*m,int n){
  ll x=0,M=1; // 当前的解为 x mod M
  for(int i=1;i<=n;i++){
    ll c=((a[i]-x)%m[i]+m[i])%m[i];
    ll t,y;
    ll g=exgcd(M,m[i],t,y);
    if(c%g)return -1;
    ll mg=m[i]/g;
    t=(t%mg+mg)%mg;
    t=(__int128)t*(c/g)%mg;
    x+=M*t; // 新的 x 小于新的 M
    M*=mg;
  }
  return x;
}
```

ExLucas.h

Description: 扩展 Lucas, 求 $\binom{n}{m} \bmod P$, P 不必是质数. exlucas(n, m, P) 在 $m < 0$ 或 $m > n$ 时返回 0, $P = 1$ 时返回 0. 范围: $0 \leq n, m \leq 10^{18}$, $P \leq 10^{18}$, 但 P 分解后的每个质数幂 p^k 不能太大 (约 10^6 以内), 因为要枚举 $1..p^k$. 做法: 把 P 分解成 p^k 的乘积; 对每个 p^k, facp(n, p, pk) 求 $n!$ 去掉所有因子 p 后模 p^k 的值, Cpk 再乘上 p^e (e 是 $\binom{n}{m}$ 中 p 的次数, 用 Legendre 公式); 与 p 互质的数的逆元用欧拉定理 $a^{\varphi(p^k)-1}$; 最后用 CRT 把各个 p^k 的结果合并 (模数两两互质). 自带快速幂 pw, 不依赖其他文件. P 是质数且较小时直接用 Lucas 定理更快.

Usage:
```
ll c = exlucas(n, m, P); // C(n, m) mod P
ll d = exlucas(10, 3, 1000); // 120
```

Time: $\mathcal{O}\left(\sum p^k \log n\right)$

438ff4, 56 lines

```
ll pw(ll b,ll e,ll m){
  ll r=1%m;
  b%=m;
  while(e){
    if(e&1)r=r*b%m;
    b=b*b%m;
    e>>=1;
  }
  return r;
}
// n! 去掉所有因子 p 后模 pk
ll facp(ll n,ll p,ll pk){
  if(n==0)return 1;
  ll r=1;
  for(ll i=1;i<=pk;i++){
    if(i%p)r=r*i%pk;
  }
  r=pw(r,n/pk,pk);
  for(ll i=1;i<=n%pk;i++){
    if(i%p)r=r*i%pk;
  }
  return r*facp(n/p,p,pk)%pk;
}
// C(n, m) mod pk, pk = p^k
ll Cpk(ll n,ll m,ll p,ll pk){
  ll e=0; // C(n, m) 中因子 p 的次数
  for(ll t=n;t;t/=p)e+=t/p;
  for(ll t=m;t;t/=p)e-=t/p;
  for(ll t=n-m;t;t/=p)e-=t/p;
  ll ph=pk/p*(p-1); // phi(pk)
  ll r=facp(n,p,pk);
  r=r*pw(facp(m,p,pk),ph-1,pk)%pk;
  r=r*pw(facp(n-m,p,pk),ph-1,pk)%pk;
  return r*pw(p,e,pk)%pk;
}
ll exlucas(ll n,ll m,ll P){
  if(m<0||m>n)return 0;
  ll x=0,M=1; // 已合并的解 x mod M
  ll Q=P;
  for(ll p=2;Q>1;p++){
    if(p*p>Q)p=Q;
    if(Q%p)continue;
    ll pk=1;
    while(Q%p==0){
      Q/=p;
      pk*=p;
    }
    ll r=Cpk(n,m,p,pk);
    ll ph=pk/p*(p-1);
    // 求 t 使 x + M t = r (mod pk), M 与 pk 互质
    ll t=(r-x%pk+pk)%pk*pw(M,ph-1,pk)%pk;
    x+=M*t;
    M*=pk;
  }
  return x;
}
```

5.3.1 Bézout 定理

对 $a \neq 0$, $b \neq 0$, $d = gcd(a, b)$ 是使下式有整数解的最小正整数

$$ax + by = d$$

若 (x, y) 是一组解, 则全部解为

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: 欧拉函数 $\varphi(n)$ 为 $1..n$ 中与 n 互质的数的个数. $n = p_1^{k_1} \cdots p_r^{k_r}$ 时 $\varphi(n) = n \prod_{p|n}(1 - 1/p)$; $\varphi(p^k) = (p-1)p^{k-1}$; m, n 互质时 $\varphi(mn) = \varphi(m)\varphi(n)$. $\sum_{d|n} \varphi(d) = n$, $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\varphi(n)/2$ $(n > 1)$.

欧拉定理: a, n 互质 $\Rightarrow a^{\varphi(n)} \equiv 1 \pmod{n}$. **扩展欧拉定理**: $b \geq \varphi(n)$ 时 $a^b \equiv a^{b \bmod \varphi(n) + \varphi(n)} \pmod{n}$ (a, n 任意), 用于指数很大的幂. **费马小定理**: p 为质数 $\Rightarrow a^p \equiv a \pmod{p}$.

getPhi(n) 用试除法求单个 $\varphi(n)$, 要求 $1 \leq n \leq 9 \cdot 10^{18}$, 复杂度 $O(\sqrt{n})$ (n 到 10^{14} 左右还能接受). 要 $1..n$ 所有的 φ 时用 FastEratosthenes 的线性筛.

Usage:
```
ll f = getPhi(12); // 4
ll e = b % getPhi(n) + getPhi(n); // 扩展欧拉定理降幂
```

Time: $\mathcal{O}\left(\sqrt{n}\right)$

38aab2, 10 lines

```
ll getPhi(ll n){
  ll r=n;
  for(ll i=2;i*i<=n;i++){
    if(n%i)continue;
    r=r/i*(i-1);
    while(n%i==0)n/=i;
  }
  if(n>1)r=r/n*(n-1);
  return r;
}
```

5.3.2 欧拉函数公式

$\varphi(n)$ 为 $1 \ldots n$ 中与 n 互质的数的个数.

- 通项: $n = \prod p_i^{k_i}$ 时 $\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$, $\varphi(p^k) = (p-1)p^{k-1}$, $\varphi(1) = 1$.
- 积性: $\gcd(m, n) = 1$ 时 $\varphi(mn) = \varphi(m)\varphi(n)$. 线性筛里: $p \mid i$ 时 $\varphi(ip) = \varphi(i) \cdot p$, 否则 $\varphi(ip) = \varphi(i)(p-1)$.
- $\sum_{d|n} \varphi(d) = n$; $n > 1$ 时与 n 互质的数之和为 $n\varphi(n)/2$.
- $\sum_{i=1}^{n} \gcd(i, n) = \sum_{d|n} d \cdot \varphi(n/d)$; $\#\{1 \leq i, j \leq n : \gcd(i, j) = 1\} = 2\sum_{i=1}^{n} \varphi(i) - 1$.
- 欧拉定理: $\gcd(a, m) = 1$ 时 $a^{\varphi(m)} \equiv 1 \pmod{m}$, 所以 $a^b \equiv a^{b \bmod \varphi(m)}$. 质数 p 时即费马小定理 $a^{p-1} \equiv 1$.
- 扩展欧拉定理 (不要求互质): $b \geq \varphi(m)$ 时 $a^b \equiv a^{b \bmod \varphi(m) + \varphi(m)} \pmod{m}$; $b < \varphi(m)$ 时直接算. 用于指数巨大或幂塔 $a^{a^{a^{\cdots}}}$, 递归 $m \to \varphi(m)$ 约 $O(\log m)$ 层到 1.

5.4 分数

Fractions.h

Description: 分数 (有理数) 的精确运算. Frac 存分子 p 和分母 q, 总是约到最简且 $q > 0$, 0 存成 0/1. mk(p, q) 用任意 p 和 $q \neq 0$ 造一个约分后的分数; 支持 $+, -, \times, /$ (除数不能为 0) 和 $<, ==$. 因为总是最简, 两个分数相等当且仅当 p, q 都相等. 比较大小用 __int128 交叉相乘. 溢出: 加减乘除会算 $p_1 q_2$ 这样的乘积, 要保证分子分母不超过约 $2 \cdot 10^9$; 更大时把 ll 改成 __int128 (gcd 要自己写, 输出时转回 ll).

Usage:
```
Frac a = mk(1, 2), b = mk(1, 3);
Frac c = a + b; // c.p = 5, c.q = 6
Frac d = mk(4, -6); // d.p = -2, d.q = 3
if(d < a) ...; if(a == b) ...;
cout << c.p << '/' << c.q; // 输出 5/6
```

Time: 每次运算 $\mathcal{O}(\log)$

f10f3a, 23 lines

```
struct Frac{
  ll p,q; // p / q, q > 0, 已约分
};
```

```
Frac mk(ll p,ll q){ // q 不能为 0
  if(q<0){
    p=-p;
    q=-q;
  }
  ll g=gcd(p,q); // 非负, p = 0 时为 q
  return {p/g,q/g};
}
Frac operator+(Frac a,Frac b){
  return mk(a.p*b.q+b.p*a.q,a.q*b.q);
}
Frac operator-(Frac a,Frac b){
  return mk(a.p*b.q-b.p*a.q,a.q*b.q);
}
Frac operator*(Frac a,Frac b){return mk(a.p*b.p,a.q*b.q);}
Frac operator/(Frac a,Frac b){return mk(a.p*b.q,a.q*b.p);}
bool operator<(Frac a,Frac b){
  return (__int128)a.p*b.q<(__int128)b.p*a.q;
}
bool operator==(Frac a,Frac b){return a.p==b.p&&a.q==b.q;}
```

5.5 勾股数

勾股数可唯一地表示为

$$a = k \cdot (m^2 - n^2),\ b = k \cdot (2mn),\ c = k \cdot (m^2 + n^2),$$

其中 $m > n > 0$, $k > 0$, $m \perp n$, 且 m 或 n 为偶数.

5.6 素数

$p = 962592769$ 满足 $2^{21} \mid p-1$, 可能有用. 哈希可用 970592641 (31 位), 31443539979727 (45 位), 3006703054056749 (52 位). 小于 1 000 000 的素数有 78498 个.

除 $p = 2, a > 2$ 外, 模任意素数幂 p^a 都有原根, 共 $\phi(\phi(p^a))$ 个. $p = 2, a > 2$ 时, 群 $\mathbb{Z}_{2^a}^{\times}$ 同构于 $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 估计

$\sum_{d|n} d = O(n \log \log n)$.

n 的约数个数最多约为: $n < 5e4$ 时 100, $n < 1e7$ 时 500, $n < 1e10$ 时 2000, $n < 1e19$ 时 200 000.

5.8 Mobius 函数

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius 反演:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

其他常用形式:

$\sum_{d|n} \mu(d) = [n = 1]$ (很有用)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

组合数学 (6)

6.1 排列

6.1.1 阶乘

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

6.1.2 环

设 $g_S(n)$ 为所有环长都属于集合 S 的 n 元排列个数, 则

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n} \right)$$

6.1.3 错排

没有任何元素在原位置上的排列.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

6.1.4 Burnside 引理

给定对称群 G 和集合 X, X 中在对称意义下不同的元素个数为

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

其中 X^g 是被 g 固定的元素 ($g.x = x$).

若 $f(n)$ 统计长度为 n 的某种方案数, 取 $G = \mathbb{Z}_n$ 即可不计旋转对称, 得

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 划分与子集

6.2.1 划分函数

把 n 写成若干正整数之和的方案数, 不计顺序.

$$p(0) = 1,\ p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

6.2.2 Lucas 定理

设 n, m 为非负整数, p 为素数. 写成 $n = n_k p^k + ... + n_1 p + n_0$, $m = m_k p^k + ... + m_1 p + m_0$, 则 $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod{p}$.

6.2.3 二项式系数

multinomial.h

Description: 计算多项式系数 $\binom{k_1 + \cdots + k_n}{k_1, k_2, \ldots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \ldots k_n!}$, 即把 k_1 个 1 号球, k_2 个 2 号球, ... 排成一排的方案数. 输入 vector k, 每个 $k_i \ge 0$, 空的 vector 返回 1. 不取模, 精确计算: 逐个乘组合数, 每步都能整除. 要求结果乘上 $\sum k_i$ 不超过 ll, 否则中间值溢出. 要取模就改用阶乘和逆元.

Usage: `vector<int> k = {2, 1, 1};`
`ll c = multinomial(k); // 4! / (2! 1! 1!) = 12`

Time: $\mathcal{O}(\sum k_i)$ d7168e, 11 lines

```
ll multinomial(const vector<int>&k){
  ll c=1;
  int m=0; // m: 已放入的个数
  for(int x:k){
    for(int j=1;j<=x;j++){
      m++;
      c=c*m/j;
    }
  }
  return c;
}
```

6.3 容斥与二项式反演

6.3.1 容斥原理

$$\left| \bigcup_{i=1}^{n} A_i \right| = \sum_{\emptyset \ne S \subseteq [n]} (-1)^{|S|+1} \left| \bigcap_{i \in S} A_i \right|, \qquad \#\{\text{一个条件都不满足}\} = \sum_{S \subseteq [}$$

空集的交是全集. 套路: 条件 i 是坏事件, 钦定一个集合 S 里的坏事件一定发生、其余随意, 算方案数后按 $|S|$ 的奇偶加减. 只和 $|S|$ 有关时写成 $\sum_k (-1)^k \binom{n}{k} h(k)$.

- 满射 (n 个不同的球放进 k 个不同的盒子且不空): $\sum_{j=0}^{k} (-1)^j \binom{k}{j} (k-j)^n$.
- 带上界的插板: $x_1 + \cdots + x_k = n$, $0 \le x_i \le c$ 的解数 $\sum_j (-1)^j \binom{k}{j} \binom{n-j(c+1)+k-1}{k-1}$.

- 互质计数: $\#\{1 \le a,b \le n : \gcd(a,b)=1\} = \sum_d \mu(d)\lfloor n/d \rfloor^2$, 莫比乌斯函数就是对质因子做容斥.

6.3.2 二项式反演

$$f(n)=\sum_{i=0}^{n}\binom{n}{i}g(i) \iff g(n)=\sum_{i=0}^{n}(-1)^{n-i}\binom{n}{i}f(i)$$
$$f(k)=\sum_{i=k}^{n}\binom{i}{k}g(i) \iff g(k)=\sum_{i=k}^{n}(-1)^{i-k}\binom{i}{k}f(i)$$

第二个是最常用的 至少 转 恰好: 设 $g(k)$ 为恰好满足 k 个条件的方案数, $f(k)$ 为先钦定 k 个条件满足、其余随意的方案数之和 (同一方案会被算 $\binom{i}{k}$ 次), 通常 f 好算, 再反演得 g. 例: 恰好 k 个不动点的排列数, 钦定 i 个不动点 $f(i)=\binom{n}{i}(n-i)!$, 代入上式; $k=0$ 即错排.

6.3.3 子集反演

$$f(S)=\sum_{T\subseteq S}g(T) \iff g(S)=\sum_{T\subseteq S}(-1)^{|S|-|T|}f(T)$$
$$f(S)=\sum_{T\supseteq S}g(T) \iff g(S)=\sum_{T\supseteq S}(-1)^{|T|-|S|}f(T)$$

对状压可以用子集和 (SOS DP) 的逆运算在 $O(2^n n)$ 内求出. 期望里的 min-max 容斥见 期望与概率技巧 一节.

6.4 常用计数数列

6.4.1 Bernoulli 数

Bernoulli 数的 EGF 为 $B(t)=\frac{t}{e^t-1}$ (可用 FFT 求). $B[0,\dots]=[1,-\frac{1}{2},\frac{1}{6},0,-\frac{1}{30},0,\frac{1}{42},\dots]$

幂和:

$$\sum_{i=1}^{n}n^m=\frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k\cdot(n+1)^{m+1-k}$$

无穷和的 Euler-Maclaurin 公式:

$$\sum_{i=m}^{\infty}f(i)=\int_m^{\infty}f(x)dx-\sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$
$$\approx\int_m^{\infty}f(x)dx+\frac{f(m)}{2}-\frac{f'(m)}{12}+\frac{f'''(m)}{720}+O(f^{(5)}(m))$$

6.4.2 第一类 Stirling 数

n 元排列中恰有 k 个环的排列数.

$$c(n,k)=c(n-1,k-1)+(n-1)c(n-1,k),\ c(0,0)=1$$
$$\textstyle\sum_{k=0}^{n}c(n,k)x^k=x(x+1)\dots(x+n-1)$$

$c(8,k)=8,0,5040,13068,13132,6769,1960,322,28,1$
$c(n,2)=0,0,1,3,11,50,274,1764,13068,109584,\dots$

6.4.3 Eulerian 数

恰有 k 个元素大于前一个元素的排列 $\pi\in S_n$ 个数. 等价于: 恰有 k 个 j 使 $\pi(j)>\pi(j+1)$; 恰有 $k+1$ 个 j 使 $\pi(j)\ge j$; 恰有 k 个 j 使 $\pi(j)>j$.

$$E(n,k)=(n-k)E(n-1,k-1)+(k+1)E(n-1,k)$$
$$E(n,0)=E(n,n-1)=1$$
$$E(n,k)=\sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

6.4.4 第二类 Stirling 数

把 n 个不同元素恰好分成 k 组的方案数.

$$S(n,k)=S(n-1,k-1)+kS(n-1,k)$$
$$S(n,1)=S(n,n)=1$$
$$S(n,k)=\frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

6.4.5 Bell 数

n 个不同元素的划分总数. $B(n)=1,1,2,5,15,52,203,877,4140,21147,\dots$.
对素数 p,

$$B(p^m+n)\equiv mB(n)+B(n+1)\pmod p$$

6.4.6 有标号无根树

\# n 个点: n^{n-2}
\# 连接 k 棵已有的树 (大小为 n_i): $n_1n_2\cdots n_kn^{k-2}$
\# 度数为 d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

6.4.7 Catalan 数

$$C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}=\frac{(2n)!}{(n+1)!n!}$$
$$C_0=1,\ C_{n+1}=\frac{2(2n+1)}{n+2}C_n,\ C_{n+1}=\sum C_iC_{n-i}$$

$C_n=1,1,2,5,14,42,132,429,1430,4862,16796,58786,\dots$

- $n\times n$ 网格中不越过对角线的单调路径.
- n 对括号组成的合法括号串.
- 有 $n+1$ 个叶子的二叉树 (每点 0 或 2 个儿子).
- 有 $n+1$ 个点的有序树.
- 连对角线把凸 $n+2$ 边形剖分成三角形的方案.
- 不含长为 3 的递增子序列的 $[n]$ 的排列.

图论 (7)

7.1 基础

FloydWarshall.h

Description: Floyd 全源最短路, 点 1..n, 有向图, 允许负边权. init(n): 令 $d[i][i]=0$, 其余为 INF. 加边时 d[u][v] 取 min (处理重边; 无向图两个方向都加). floyd(n) 之后 $d[i][j]$ 为 i 到 j 的最短距离, 不可达为 INF. 存在负环当且仅当某个 $d[i][i]<0$; 最后一段把能 经过负环的点对标成 $-$INF (不需要可删). 距离为 ll, 中间结果截断在 $-$INF, 不会溢出. N 为点数上限 (约 500).
Usage: `init(n); // 多组数据每组调用`
Time: $\mathcal{O}(n^3)$

```
for(int i=1;i<=m;i++){
cin>>u>>v>>w;
d[u][v]=min(d[u][v],w);
}
floyd(n);
// d[u][v]==INF 不可达, d[u][v]==-INF 无下界
```

355400, 28 lines

```
const int N=505;
const ll INF=0x3f3f3f3f3f3f3f3fLL;
ll d[N][N];
void init(int n){
  for(int i=1;i<=n;i++){
    for(int j=1;j<=n;j++)d[i][j]=i==j?0:INF;
  }
}
void floyd(int n){
  for(int k=1;k<=n;k++){
    for(int i=1;i<=n;i++){
      if(d[i][k]==INF)continue;
      for(int j=1;j<=n;j++){
        if(d[k][j]==INF)continue;
        d[i][j]=min(d[i][j],max(d[i][k]+d[k][j],-INF));
      }
    }
  }
  // 负环: 能经过负环的点对距离为 -INF
  for(int k=1;k<=n;k++){
    if(d[k][k]>=0)continue;
    for(int i=1;i<=n;i++){
      for(int j=1;j<=n;j++){
        if(d[i][k]<INF&&d[k][j]<INF)d[i][j]=-INF;
      }
    }
  }
}
```

TopoSort.h

Description: Kahn 拓扑排序 (BFS), 点 1..n. 输入: 有向边存在 g[u] 中. topo(n): 是 DAG 返回 1, 拓扑序在 ord[1..n] (所有边从前指向后); 有环返回 0, 此时只有 ord[1..cnt] 有效, 环上的点和环能到达的点都 不会出现. 入度 deg 在函数内计算, 不用手动维护. 字典序最小的拓扑序: 把 queue 换成小根堆 priority_queue<int,vector<int>,greater<int>>.
Usage: `// 多组数据: g[1..n] 清空`
Time: $\mathcal{O}(n+m)$

```
for(int i=1;i<=m;i++){
cin>>u>>v;
g[u].push_back(v); // 边 u 到 v
}
if(topo(n)) for(int i=1;i<=n;i++) cout<<ord[i]<<' ';
else ...; // 有环
```

049849, 24 lines

```
const int N=2e5+5;
vector<int>g[N];
int deg[N],ord[N],cnt;
bool topo(int n){
  for(int i=1;i<=n;i++)deg[i]=0;
```

```
  for(int u=1;u<=n;u++){
    for(int v:g[u])deg[v]++;
  }
  queue<int>q;
  cnt=0;
  for(int i=1;i<=n;i++){
    if(!deg[i])q.push(i);
  }
  while(!q.empty()){
    int u=q.front();
    q.pop();
    ord[++cnt]=u;
    for(int v:g[u]){
      deg[v]--;
      if(!deg[v])q.push(v);
    }
  }
  return cnt==n;
}
```

DynamicConnectivity.h

Description: 离线动态图连通性 (线段树分治 + 可撤销并查集), 点 $1..n$, 时刻 $1..q$. 支持加边, 删边, 询问某时刻的连通块个数 (或两点是否 连通), 全部操作先读入. 做法: 每条边在一个时刻区间 $[l,r]$ 内存在, 用 add 挂到线段树上 $O(\log q)$ 个结点; solve 时进入结点加边, 离开时撤销, 到叶子 t 时 并查集恰好是时刻 t 的图. ans[t] 为时刻 t 的连通块个数 cc; 在 solve 的叶子处可以改成回答任意询问 (如 find(x)==find(y)). 时刻: 第 i 个操作就是时刻 i. 边在时刻 a 加入, 时刻 b 删除, 区间为 $[a,b-1]$; 到最后都没删的边区间为 $[a,q]$. 同一条边先把 u,v 排成 $u<v$ 再用 map 记加入时刻. 并查集按大小合并, 不路径压缩 (才能撤销). N 点数, T 时刻数上限.

Usage: `init(n,q); // q 个时刻, 多组数据每组调用`

```
map<pair<int,int>,int> st; // 边的加入时刻
for(int i=1;i<=q;i++){
cin>>op>>u>>v;
if(u>v) swap(u,v);
if(op==1) st[{u,v}]=i; // 加边
if(op==2){ // 删边
add(1,1,q,st[{u,v}],i-1,u,v);
st.erase({u,v});
}
}
for(auto [e,t]:st) add(1,1,q,t,q,e.first,e.second);
solve(1,1,q); // ans[t] 为时刻 t 的连通块数
```

Time: $\mathcal{O}(m \log q \log n)$

66120c, 57 lines

```
const int N=1e5+5,T=1e5+5; // N 点数, T 时刻数
int fa[N],siz[N],stk[N],tp,cc,ans[T];
vector<pair<int,int>>e[T<<2]; // 线段树结点上的边
int find(int x){
  while(x!=fa[x])x=fa[x];
  return x;
}
void merge(int x,int y){
  x=find(x);
  y=find(y);
  if(x==y)return;
  if(siz[x]<siz[y])swap(x,y);
  fa[y]=x;
  siz[x]+=siz[y];
  stk[++tp]=y; // 记录被挂上去的根
  cc--;
}
void undo(int t){ // 撤销到栈大小为 t
  while(tp>t){
    int y=stk[tp--];
    siz[fa[y]]-=siz[y];
    fa[y]=y;
    cc++;
  }
}
void init(int n,int q){
  for(int i=1;i<=n;i++){
    fa[i]=i;
    siz[i]=1;
  }
  for(int i=0;i<=q*4;i++)e[i].clear();
  cc=n;
  tp=0;
}
// 边 (u,v) 在时刻 [ql,qr] 存在
void add(int p,int l,int r,int ql,int qr,int u,int v){
  if(ql>qr)return;
  if(ql<=l&&r<=qr){
    e[p].push_back({u,v});
    return;
  }
  int mid=(l+r)/2;
  if(ql<=mid)add(p*2,l,mid,ql,qr,u,v);
  if(qr>mid)add(p*2+1,mid+1,r,ql,qr,u,v);
}
void solve(int p,int l,int r){
  int t=tp;
  for(auto[u,v]:e[p])merge(u,v);
  if(l==r){
    ans[l]=cc; // 在这里回答时刻 l 的询问
  }else{
    int mid=(l+r)/2;
    solve(p*2,l,mid);
    solve(p*2+1,mid+1,r);
  }
  undo(t);
}
```

7.2 网络流

MinCostMaxFlow.h

Description: 最小费用最大流. 在流量最大的前提下使总费用最小. 做法: 每次用 SPFA 找费用最短的增广路, 沿它推满, 直到 t 不可达. 输入: 点编号 $1..n$, 先 init(n), 再 add(u,v,c,w) 加有向边, 容量 c, 单位流量费用 w (都是 ll). 允许负费用, 但不能有负环. 存图为链式前向星, cnt 从 1 开始, 边 e 与反向边 $e\oplus 1$ 成对, 反向边费用为 $-w$. cap[e] 是剩余容量. 函数: add 返回正向边编号; mcmf(s,t) 返回 (最大流, 最小费用). 输出方案: (1) 每条原边的流量: cap[$e\oplus 1$], 该边费用为流量乘 w. (2) 最小割: 跑完后 dis[v] 小于 INF 的点 (残量网络中 s 能到达) 为 S 侧. (3) 带权二分图匹配: 源点连左部, 右部连汇点, 容量 1 费用 0, 左右边容量 1 费用 w; 左到右满流 (cap[e] 为 0) 的边为匹配边. 求最大费用: 费用全取负, 答案再取负. 只要最小费用流 (不要求最大流) 时, 在 mcmf 里当 dis[t] 不小于 0 就停止. 注意: 无向边要 add 两次. N, M 为点数, 边数上限 (每条边占 2 个位置). 全局变量名与 Dinic 等冲突, 不能同时 include. 多组数据每组先 init(n).

Usage: `init(n); // 每组数据先调用`

```
for(int i=1;i<=m;i++){
cin>>u>>v>>c>>w;
id[i]=add(u,v,c,w); // 记下第 i 条边的编号
}
auto [f,co]=mcmf(s,t); // 最大流, 最小费用
cout<<f<<' '<<co<<endl;
for(int i=1;i<=m;i++) cout<<cap[id[i]^1]<<' '; // 各边流量
for(int v=1;v<=n;v++) if(dis[v]<INF) cout<<v<<' '; // S 侧
// 带权匹配: 左 1..L, 右 L+1..L+R, s=L+R+1, t=L+R+2
init(L+R+2);
for(int i=1;i<=L;i++) add(s,i,1,0);
for(int j=1;j<=R;j++) add(L+j,t,1,0);
for(int k=1;k<=K;k++) id[k]=add(a[k],L+b[k],1,w[k]);
auto [cntMatch,minCost]=mcmf(s,t);
for(int k=1;k<=K;k++) // 满流的边是匹配边
if(!cap[id[k]]) cout<<a[k]<<' '<<b[k]<<endl;
```

Time: $\mathcal{O}(F \cdot VE)$, F 为最大流, 实际远快于此.

7cd2f6, 65 lines

```
const int N=5005,M=1e5+5; // 点数; 边数, 含反向边要开 2 倍
const ll INF=1e18;
int n,cnt=1,head[N],nxt[M],to[M],pre[N]; // pre[v]: 到 v 的边
ll cap[M],cost[M],dis[N],fl[N]; // fl: 路径上的最小容量
bool inq[N];
void init(int _n){
  n=_n;
  cnt=1;
  for(int i=1;i<=n;i++)head[i]=0;
}
void addOne(int u,int v,ll c,ll w){
  cnt++;
  to[cnt]=v;
  cap[cnt]=c;
  cost[cnt]=w;
  nxt[cnt]=head[u];
  head[u]=cnt;
}
int add(int u,int v,ll c,ll w){ // 返回正向边编号
  addOne(u,v,c,w);
  addOne(v,u,0,-w);
  return cnt-1;
}
bool spfa(int s,int t){
  for(int i=1;i<=n;i++){
    dis[i]=INF;
    inq[i]=0;
  }
  queue<int>q;
  q.push(s);
  dis[s]=0;
  fl[s]=INF;
  inq[s]=1;
  while(!q.empty()){
    int u=q.front();
    q.pop();
    inq[u]=0;
    for(int e=head[u];e;e=nxt[e]){
      int v=to[e];
      if(cap[e]&&dis[u]+cost[e]<dis[v]){
        dis[v]=dis[u]+cost[e];
        pre[v]=e;
        fl[v]=min(fl[u],cap[e]);
        if(!inq[v]){
          inq[v]=1;
          q.push(v);
        }
      }
    }
  }
  return dis[t]<INF;
}
pair<ll,ll>mcmf(int s,int t){
  ll flow=0,c=0;
  while(spfa(s,t)){
    ll f=fl[t];
    flow+=f;
    c+=f*dis[t];
    for(int v=t;v!=s;v=to[pre[v]^1]){
      cap[pre[v]]-=f;
      cap[pre[v]^1]+=f;
    }
  }
  return {flow,c};
}
```

Dinic.h

Description: 最大流 Dinic. 求有向图从 s 到 t 的最大流, 并能输出方案. 输入: 点编号 $1..n$, 先 init(n), 再用 add(u,v,c) 加有向边 (c 为 ll 容量). 存图为链式前向星 head/nxt/to/cap, cnt 从 1 开始, 所以第一条边编号为 2, 边 e 与反向边 $e\oplus 1$ 成对.

cap[e] 是剩余容量. 函数: add 返回正向边编号 (要输出方案就记下来); dinic(s,t) 返回最大流. 输出方案: (1) 每条原边的流量: cap[$e \oplus 1$] (反向边的剩余容量); 原容量为 cap[e] + cap[$e \oplus 1$]. (2) 最小割: 跑完后 dep[v] 非 0 的点就是残量网络中 s 能到达的点, 即 S 侧; 割边为 S 侧指向 T 侧的正向边, 一定满流, 容量和等于最大流 (见 MinCut.h). (3) 二分图匹配: 源点连左部, 右部连汇点, 左右之间容量 1; 跑完后左到右的边若满流 (cap[e] 为 0) 就是匹配边. 注意: 无向边 (u, v, c) 要调用 add(u,v,c) 和 add(v,u,c). N 为点数上限, M 为边数上限 (每条边占 2 个位置). 全局变量名 (N, M, n, cnt, head ...) 与 MinCostMaxFlow 等冲突, 不能同时 include. 多组数据每组先 init(n).

Usage:
```
for(int i=1;i<=m;i++){
cin>>u>>v>>c;
id[i]=add(u,v,c); // 记下第 i 条边的编号
}
cout<<dinic(s,t)<<endl; // 最大流
for(int i=1;i<=m;i++) cout<<cap[id[i]^1]<<' '; // 各边流量
for(int v=1;v<=n;v++) if(dep[v]) cout<<v<<' '; // 最小割 S 侧
// 二分图匹配: 左 1..L, 右 L+1..L+R, s=L+R+1, t=L+R+2
init(L+R+2);
for(int i=1;i<=L;i++) add(s,i,1);
for(int j=1;j<=R;j++) add(L+j,t,1);
for(int k=1;k<=K;k++) id[k]=add(a[k],L+b[k],1); // 边 a 到 b
int ans=dinic(s,t);
for(int k=1;k<=K;k++) // 满流的边是匹配边
if(!cap[id[k]]) cout<<a[k]<<' '<<b[k]<<endl;
```

Time: $\mathcal{O}\left(V^2E\right)$, 单位容量 $\mathcal{O}\left(E\sqrt{E}\right)$, 二分图 $\mathcal{O}\left(E\sqrt{V}\right)$

f42e09, 66 lines

```
const int N=1e4+5,M=2e5+5; // 点数; 边数, 含反向边要开 2 倍
const ll INF=1e18;
int n,S,T,cnt=1,head[N],nxt[M],to[M],dep[N],cur[N];
ll cap[M]; // 剩余容量
void init(int _n){
  n=_n;
  cnt=1;
  for(int i=1;i<=n;i++)head[i]=0;
}
void addOne(int u,int v,ll c){
  cnt++;
  to[cnt]=v;
  cap[cnt]=c;
  nxt[cnt]=head[u];
  head[u]=cnt;
}
int add(int u,int v,ll c){ // u 到 v, 容量 c, 返回编号
  addOne(u,v,c);
  addOne(v,u,0);
  return cnt-1;
}
bool bfs(){ // 求出所有 s 可达点的层数, 不提前退出
  for(int i=1;i<=n;i++)dep[i]=0;
  queue<int>q;
  q.push(S);
  dep[S]=1;
  while(!q.empty()){
    int u=q.front();
    q.pop();
    for(int e=head[u];e;e=nxt[e]){
      int v=to[e];
      if(cap[e]&&!dep[v]){
        dep[v]=dep[u]+1;
        q.push(v);
      }
    }
  }
  return dep[T];
}
ll dfs(int u,ll f){
  if(u==T)return f;
  ll res=0;
  for(int&e=cur[u];e;e=nxt[e]){ // 当前弧优化
    int v=to[e];
    if(cap[e]&&dep[v]==dep[u]+1){
      ll d=dfs(v,min(f,cap[e]));
      cap[e]-=d;
      cap[e^1]+=d;
      res+=d;
      f-=d;
      if(!f)break;
    }
  }
  if(!res)dep[u]=0; // 此点已无法增广
  return res;
}
ll dinic(int s,int t){
  S=s;
  T=t;
  ll flow=0;
  while(bfs()){
    for(int i=1;i<=n;i++)cur[i]=head[i];
    flow+=dfs(S,INF);
  }
  return flow;
}
```

MinCut.h

Description: 输出 s-t 最小割方案, 配合 Dinic.h 使用. 最小割 = 最大流. 跑完 dinic(s,t) 后最后一次 bfs 失败, 此时 dep[v] 非 0 的点就是残量网络中从 s 能到达的点, 构成 S 侧, 其余点为 T 侧. 割边为从 S 侧指向 T 侧的原边, 它们都满流. 函数: minCut() 返回所有割边的编号 (即 add 的返回值, 正向边). 对割边 e: 起点 to[$e \oplus 1$], 终点 to[e], 原容量 cap[$e \oplus 1$], 这些容量之和等于最大流. 若要割点 (每个点拆成入点, 出点, 中间连容量 1 的边) 同理读割边. 注意: 必须在 dinic 之后, 下一次 init 之前调用.

Usage:
```
for(int i=1;i<=m;i++) add(u[i],v[i],c[i]);
ll f=dinic(s,t); // 最小割的值
vector<int> cut=minCut();
for(int e:cut) cout<<to[e^1]<<' '<<to[e]<<endl; // 割边 u v
for(int v=1;v<=n;v++) if(dep[v]) cout<<v<<' '; // S 侧的点
```

Time: $\mathcal{O}(E)$

"Dinic.h" 418f33, 7 lines

```
vector<int>minCut(){ // 跑完 dinic 后调用
  vector<int>res;
  for(int e=2;e<=cnt;e+=2){ // 只看正向边
    if(dep[to[e^1]]&&!dep[to[e]])res.push_back(e);
  }
  return res;
}
```

匹配 (8)

HopcroftKarp.h

Description: 二分图最大匹配 Hopcroft-Karp. 输入: 左部点 1..n, 右部点 1..m (左右各自编号, 可以重号), 先 init(n,m), 再对每条边 g[u].push_back(v) (左 u 连右 v). 函数: hk() 返回最大匹配数. 结果: mL[u] 为左部点 u 匹配的右部点, mR[v] 为右部点 v 匹配的左部点, 未匹配为 0. 做法: 每轮 bfs 从左部未匹配点出发给左部点分层 (d[u]), 再 dfs 沿层找一批不相交的最短增广路; 共 $O(\sqrt{V})$ 轮. 注意: N 为左右部点数的上限. dfs 递归深度可达 n. 与其他用全局变量 n, m, N 的模板不能同时 include (MinimumVertexCover.h 已经 include 本文件).

Usage:
```
init(n,m); // 每组数据先调用
for(int i=1;i<=k;i++){
cin>>u>>v; // 左 u, 右 v
g[u].push_back(v);
}
cout<<hk()<<endl; // 最大匹配数
for(int u=1;u<=n;u++) // 输出匹配边
if(mL[u]) cout<<u<<' '<<mL[u]<<endl;
```

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

9d6e7a, 54 lines

```
const int N=1e5+5; // 左右部点数的最大值
int n,m,mL[N],mR[N],d[N];
vector<int>g[N];
void init(int _n,int _m){
  n=_n;
  m=_m;
  for(int i=1;i<=n;i++)g[i].clear();
}
bool dfs(int u){
  int t=d[u]+1;
  d[u]=0; // 本轮不再访问 u
  for(int v:g[u]){
    if(!mR[v]||(d[mR[v]]==t&&dfs(mR[v]))){
      mL[u]=v;
      mR[v]=u;
      return true;
    }
  }
  return false;
}
int hk(){
  for(int i=1;i<=n;i++)mL[i]=0;
  for(int i=1;i<=m;i++)mR[i]=0;
  int res=0;
  while(true){
    // d[u] 为左部点 u 的层数, 0 表示不可用
    queue<int>q;
    bool found=false;
    for(int i=1;i<=n;i++){
      d[i]=0;
      if(!mL[i]){
        d[i]=1;
        q.push(i);
      }
    }
    while(!q.empty()){
      int u=q.front();
      q.pop();
      for(int v:g[u]){
        int w=mR[v];
        if(!w){
          found=true;
        }else if(!d[w]){
          d[w]=d[u]+1;
          q.push(w);
        }
      }
    }
    if(!found)return res;
    for(int i=1;i<=n;i++){
      if(!mL[i]&&dfs(i))res++;
    }
  }
}
```

MinimumVertexCover.h

Description: 二分图最小点覆盖 (König 定理): 选最少的点使每条边 至少有一个端点被选, 大小等于最大匹配数. 补集为最大独立集. 输入: 同 HopcroftKarp.h, 左部 1..n, 右部 1..m, init(n,m) 后 g[u].push_back(v). 不需要先调用 hk(). 函数:

cover(A,B) 内部先跑 hk(), 返回最大匹配数; A 为覆盖中的左部点, B 为覆盖中的右部点, A.size()+B.size() 等于返回值. 做法: 从左部所有未匹配点出发走交错路 (左到右走任意边, 右到左走匹配边), 标记到达的点 visL, visR. 最小点覆盖 = 左部未标记的点 + 右部已标记的点; 最大独立集 = 左部已标记的点 + 右部未标记的点. 跑完后 mL, mR 也是一组最大匹配.

Usage:
```
init(n,m); // 每组数据先调用
for(int i=1;i<=k;i++){
cin>>u>>v; // 左 u, 右 v
g[u].push_back(v);
}
vector<int> A,B;
int sz=cover(A,B); // 最小点覆盖大小
for(int x:A) cout<<'L'<<x<<' '; // 覆盖中的左部点
for(int y:B) cout<<'R'<<y<<' '; // 覆盖中的右部点
// 最大独立集: 左部 visL[u] 为真的点, 右部 visR[v] 为假的点
```

Time: $\mathcal{O}\left(E\sqrt{V}\right)$

"HopcroftKarp.h" 5c97fe, 35 lines

```
bool visL[N],visR[N];
int cover(vector<int>&A,vector<int>&B){
  int res=hk();
  for(int i=1;i<=n;i++)visL[i]=0;
  for(int i=1;i<=m;i++)visR[i]=0;
  vector<int>st;
  for(int i=1;i<=n;i++){
    if(!mL[i]){
      visL[i]=1;
      st.push_back(i);
    }
  }
  while(!st.empty()){
    int u=st.back();
    st.pop_back();
    for(int v:g[u]){
      if(visR[v])continue;
      visR[v]=1;
      int w=mR[v]; // v 一定已匹配
      if(!visL[w]){
        visL[w]=1;
        st.push_back(w);
      }
    }
  }
  A.clear();
  B.clear();
  for(int i=1;i<=n;i++){
    if(!visL[i])A.push_back(i);
  }
  for(int i=1;i<=m;i++){
    if(visR[i])B.push_back(i);
  }
  return res;
}
```

WeightedMatching.h

Description: 带权二分图最优匹配 (KM / 匈牙利算法, $O(n^2m)$ 版). 问题: 左部 n 个点, 右部 m 个点, 要求 $n \le m$ (否则把矩阵转置), 给每个左部点匹配一个不同的右部点, 使代价和最小. 输入: 全局 n, m 和代价矩阵 a[i][j] (左 i 配右 j 的代价, ll, $1 \le i \le n$, $1 \le j \le m$). 没有的边设为很大的代价 (如 10^{12}, 要远小于 INF 除以 n), 答案不小于它说明无完美匹配. 函数: km() 返回最小代价和; ans[i] 为左 i 匹配的右部点, p[j] 为右 j 匹配的左部点 (0 表示未匹配). 变形: 最大权完美匹配: a 取负, 答案再取负. 最大权匹配 (可以不匹配): a[i][j] 取 $-\max(w, 0)$, 并补 0 使 $n = m$, 答案取负; 输出时跳过原图中不存在或权不为正的配对. 注意: N 为 n, m 上限 (a 占 N^2 个 ll). 多组数据直接覆盖 a, n, m. 全局变量名与其他图论模板冲突, 不能同时 include.

Usage:
```
cin>>n>>m; // 要求 n 不超过 m
for(int i=1;i<=n;i++)
for(int j=1;j<=m;j++) cin>>a[i][j];
ll c=km(); // 最小代价和
for(int i=1;i<=n;i++) cout<<i<<' '<<ans[i]<<endl; // 匹配
// 最大权完美匹配: 读入 w 后 a[i][j]=-w; 答案为 -km()
```

Time: $\mathcal{O}\left(n^2m\right)$

3aa533, 55 lines

```
const int N=505; // n, m 的最大值
const ll INF=1e18; // 大于任意可能的代价和
int n,m,p[N],way[N],ans[N]; // p[j]: 右 j 匹配的左部点
ll a[N][N],u[N],v[N],minv[N]; // u, v 为顶标
bool used[N];
ll km(){
  for(int i=0;i<=n;i++)u[i]=0;
  for(int j=0;j<=m;j++){
    v[j]=0;
    p[j]=0;
  }
  for(int i=1;i<=n;i++){
    p[0]=i; // 虚拟右部点 0 先配给 i
    int j0=0;
    for(int j=0;j<=m;j++){
      minv[j]=INF;
      used[j]=0;
    }
    do{ // 找一条到未匹配右部点的最短增广路
      used[j0]=1;
      int i0=p[j0],j1=0;
      ll delta=INF;
      for(int j=1;j<=m;j++){
        if(used[j])continue;
        ll cur=a[i0][j]-u[i0]-v[j];
        if(cur<minv[j]){
          minv[j]=cur;
          way[j]=j0;
        }
        if(minv[j]<delta){
          delta=minv[j];
          j1=j;
        }
      }
      for(int j=0;j<=m;j++){ // 修改顶标
        if(used[j]){
          u[p[j]]+=delta;
          v[j]-=delta;
        }else{
          minv[j]-=delta;
        }
      }
      j0=j1;
    }while(p[j0]);
    do{ // 沿增广路翻转
      int j1=way[j0];
      p[j0]=p[j1];
      j0=j1;
    }while(j0);
  }
  for(int j=1;j<=m;j++){
    if(p[j])ans[p[j]]=j;
  }
  return -v[0];
}
```

GeneralMatching.h

Description: 一般图最大匹配, 带花树 (Edmonds blossom). 问题: 无向图 (不一定是二分图) 中选最多的边, 使它们没有公共端点. 输入: 点编号 1..n, 先 init(n), 再 add(u,v) 加无向边 (只调用一次, 内部存两个方向). 自环和重边都可以. 函数: solve() 返回最大匹配数. 结果: mt[u] 为 u 匹配的点, 未匹配为 0; 输出每条匹配边时只输出 mt[u] 大于 u 的, 避免重复. 做法: 对每个未匹配点 bfs 建交错树找增广路, vis 为 1 表示偶点 (可入队), 为 2 表示奇点; 两个偶点相连说明出现奇环, 用并查集 fa 把它缩成一朵花 (花根为 lca), 花里的奇点也变成偶点. 注意: N 为点数上限, M 为边数上限的 2 倍. 全局变量名 (n, cnt, head, pre, q ...) 与其他图论模板冲突, 不能同时 include.

Usage:
```
init(n); // 每组数据先调用
for(int i=1;i<=m;i++){
cin>>u>>v;
add(u,v); // 无向边, 只加一次
}
cout<<solve()<<endl; // 最大匹配数
for(int u=1;u<=n;u++) // 输出匹配边
if(mt[u]>u) cout<<u<<' '<<mt[u]<<endl;
```

Time: $\mathcal{O}\left(n^3\right)$

30f548, 98 lines

```
const int N=1005,M=2e5+5; // 点数; 边数, 每条边存两次
int n,cnt,head[N],nxt[M],to[M];
int mt[N],pre[N],fa[N],vis[N],dfn[N],tim;
queue<int>q;
void init(int _n){
  n=_n;
  cnt=0;
  tim=0;
  for(int i=0;i<=n;i++){
    head[i]=0;
    mt[i]=0;
    dfn[i]=0;
  }
}
void addOne(int u,int v){
  cnt++;
  to[cnt]=v;
  nxt[cnt]=head[u];
  head[u]=cnt;
}
void add(int u,int v){
  addOne(u,v);
  addOne(v,u);
}
int find(int x){
  if(fa[x]==x)return x;
  return fa[x]=find(fa[x]);
}
int lca(int x,int y){ // 两个偶点在交错树上的最近公共花
  tim++;
  x=find(x);
  y=find(y);
  while(dfn[x]!=tim){
    dfn[x]=tim;
    x=find(pre[mt[x]]);
    if(y)swap(x,y);
  }
  return x;
}
void blossom(int x,int y,int w){ // 把 x 到花根 w 的路径缩进花
  while(find(x)!=w){
    pre[x]=y;
    y=mt[x];
    if(vis[y]==2){ // 花里的奇点变成偶点
      vis[y]=1;
      q.push(y);
    }
    if(find(x)==x)fa[x]=w;
    if(find(y)==y)fa[y]=w;
    x=pre[y];
  }
}
bool aug(int s){ // 从未匹配点 s 找增广路
  for(int i=1;i<=n;i++){
```

```
    fa[i]=i;
    vis[i]=0;
    pre[i]=0;
  }
  while(!q.empty())q.pop();
  q.push(s);
  vis[s]=1;
  while(!q.empty()){
    int u=q.front();
    q.pop();
    for(int e=head[u];e;e=nxt[e]){
      int v=to[e];
      if(find(u)==find(v)||vis[v]==2)continue;
      if(vis[v]==1){ // u, v 都是偶点，出现奇环
        int w=lca(u,v);
        blossom(u,v,w);
        blossom(v,u,w);
        continue;
      }
      vis[v]=2;
      pre[v]=u;
      if(!mt[v]){ // 找到增广路，沿 pre 翻转
        int x=v;
        while(x){
          int y=mt[pre[x]];
          mt[x]=pre[x];
          mt[pre[x]]=x;
          x=y;
        }
        return true;
      }
      vis[mt[v]]=1;
      q.push(mt[v]);
    }
  }
  return false;
}
int solve(){
  int res=0;
  for(int i=1;i<=n;i++){
    if(!mt[i]&&aug(i))res++;
  }
  return res;
}
```

8.1 DFS 算法

AllRoots.h

Description: 换根 DP 模板: 对树上每个点 u 都求以 u 为根时的答案. 例子: 带边权的树, 点 1..n, 同时求 ans[u] $= \sum_v dist(u,v)$ 和 far[u] $= \max_v dist(u,v)$. 输入: n (全局) 和 g[u] 存 {v, w} (无向边两个方向都加). dfs1(1,0): 以 1 为根自底向上, 求子树内的信息: siz (子树大小), f[u] (子树内各点到 u 的距离和), d1/d2 (向下最长/次长链), c1 (最长链经过的儿子). dfs2(1,0): 自顶向下, 由父亲 u 的答案推出儿子 v 的答案: ans[v] $=$ ans[u] $+(n-2\cdot siz[v])w$; up[v] 为 v 往父亲方向 的最长链. 去掉儿子 v 的贡献: 可减的量 (和) 直接减; 不可减的量 (max) 记最大和次大, 或用前缀/后缀合并. 改成别的问题: 换掉 dfs1 的合并和 dfs2 的转移即可. 注意: 递归深度可达 n, 链状树要开大栈.

Usage: // 多组数据: g[1..n] 清空

```
for(int i=1;i<n;i++){
cin>>u>>v>>w;
g[u].push_back({v,w});
g[v].push_back({u,w});
}
dfs1(1,0);
ans[1]=f[1]; // 根的答案
up[1]=0;
dfs2(1,0);
for(int u=1;u<=n;u++) far[u]=max(d1[u],up[u]);
```

Time: $\mathcal{O}(n)$

ae4e14, 35 lines

```
const int N=2e5+5;
int n,c1[N];
vector<pair<int,int>>g[N]; // {v,w}
// siz 子树大小, f 子树内各点到 u 的距离和, ans 全树距离和
// d1/d2 向下最长次长链/, up 向上最长链, far 最远距离
ll siz[N],f[N],ans[N],d1[N],d2[N],up[N],far[N];
void dfs1(int u,int fa){
  siz[u]=1;
  f[u]=d1[u]=d2[u]=0;
  c1[u]=0;
  for(auto[v,w]:g[u]){
    if(v==fa)continue;
    dfs1(v,u);
    siz[u]+=siz[v];
    f[u]+=f[v]+siz[v]*w;
    ll t=d1[v]+w;
    if(t>d1[u]){
      d2[u]=d1[u];
      d1[u]=t;
      c1[u]=v;
    }else if(t>d2[u]){
      d2[u]=t;
    }
  }
}
void dfs2(int u,int fa){
  for(auto[v,w]:g[u]){
    if(v==fa)continue;
    // 根从 u 移到 v: v 子树内的点近 w, 其余点远 w
    ans[v]=ans[u]+(n-2*siz[v])*w;
    // v 向上: 走到 u, 再走 u 的向上链或不经过 v 的向下链
    up[v]=w+max(up[u],c1[u]==v?d2[u]:d1[u]);
    dfs2(v,u);
  }
}
```

SCC.h

Description: Tarjan 求有向图强连通分量 (SCC), 点 1..n. 输入: 有向边存在 g[u] 中 (u 到 v 则 g[u].push_back(v)). 对每个未访问的点 (dfn 为 0) 调用 tarjan(i). 结果: scc[u] 为点 u 所在分量编号 (1..cnt), sz[i] 为分量 i 的 点数, cnt 为分量个数. Tarjan 的编号是逆拓扑序: 缩点后的边总是 从编号大的分量指向编号小的分量, 所以 $cnt, cnt-1, \ldots, 1$ 就是 DAG 的拓扑序, 可以直接按这个顺序 DP. shrink(n) 建缩点后的 DAG, 存在 h[1..cnt] 中 (可能有重边, 需要时去重). 注意: 递归深度可达 n, 链状图要开大栈. N 为点数上限.

Usage: init(n); // 多组数据每组调用, 清空 g, h, dfn 等

```
for(int i=1;i<=m;i++){
cin>>u>>v;
g[u].push_back(v);
}
for(int i=1;i<=n;i++) if(!dfn[i]) tarjan(i);
shrink(n); // 需要 DAG 时调用
for(int i=cnt;i>=1;i--) // 按拓扑序遍历分量
for(int j:h[i]) ...; // 分量 i 到分量 j 的边
```

Time: $\mathcal{O}(n+m)$

ce4d4c, 43 lines

```
const int N=2e5+5;
vector<int>g[N],h[N]; // g 原图, h 缩点后的 DAG
int dfn[N],low[N],stk[N],tp,tim;
int scc[N],sz[N],cnt;
bool ins[N]; // 是否在栈中
void tarjan(int u){
  dfn[u]=low[u]=++tim;
  stk[++tp]=u;
  ins[u]=1;
  for(int v:g[u]){
    if(!dfn[v]){
      tarjan(v);
      low[u]=min(low[u],low[v]);
    }else if(ins[v]){
      low[u]=min(low[u],dfn[v]);
    }
  }
  if(low[u]==dfn[u]){ // u 是分量的根, 弹栈
    cnt++;
    int v;
    do{
      v=stk[tp--];
      ins[v]=0;
      scc[v]=cnt;
      sz[cnt]++;
    }while(v!=u);
  }
}
void shrink(int n){
  for(int u=1;u<=n;u++){
    for(int v:g[u]){
      if(scc[u]!=scc[v])h[scc[u]].push_back(scc[v]);
    }
  }
}
void init(int n){
  for(int i=1;i<=n;i++){
    g[i].clear();
    h[i].clear();
    dfn[i]=sz[i]=0;
  }
  tim=tp=cnt=0;
}
```

BiconnectedComponents.h

Description: 无向图 Tarjan, 点 1..n, 边 1..m, 一次 DFS 同时求: (1) 割点 cut[u] 和点双连通分量 vb[1..vc] (每个点双的点集; 割点属于 多个点双; 孤立点自己成一个点双). 根是割点当且仅当它在 DFS 树上 至少有两个儿子. (2) 桥 bri[id] (按边编号) 和边双连通分量 eb[u] (编号 1..ec). 用边编号判断是否走回父边, 所以重边能正确处理; 自环不影响结果. 输入: g[u] 存 {v, 边编号}, 每条边两个方向都要加, 编号 1..m. 对每个未访问的点调用 tarjan(i,0,i) (0 表示没有父边, 第三个参数为根). buildRS(n): 建圆方树 T, 圆点 1..n, 第 i 个点双为方点 $n+i$, 向点双内每个点连边; 孤立点也有自己的方点 (只连它一个). u 是割点当且仅当它在圆方树上度数至少 2. buildBT(n): 建边双缩点树 E2[1..ec], 每个边双一个点, 每条桥一条边 (图不连通时为森林). 只需要一部分时删掉另一部分即可. 注意: 递归深度可达 n, 要开大栈. N 点数上限, M 边数上限.

Usage: init(n,m); // 多组数据每组调用

```
for(int i=1;i<=m;i++){
cin>>u>>v; // 第 i 条边
g[u].push_back({v,i});
g[v].push_back({u,i});
}
for(int i=1;i<=n;i++) if(!dfn[i]) tarjan(i,0,i);
// cut[u], bri[i], vb[1..vc], eb[u] 已求出
buildRS(n); // 圆方树 T[1..n+vc]
buildBT(n); // 缩点树 E2[1..ec], 点 u 对应 eb[u]
```

Time: $\mathcal{O}(n+m)$

4390bc, 72 lines

```
const int N=2e5+5,M=4e5+5;
vector<pair<int,int>>g[N]; // {v边编号,}
```

```
vector<int>T[N*2],E2[N]; // 圆方树，边双缩点树
int dfn[N],low[N],tim;
int s1[N],t1,s2[N],t2; // 点双的栈，边双的栈
bool cut[N],bri[M];
int vc,ec,eb[N];
vector<int>vb[N];
// in 为进入 u 的边编号，rt 为本次 DFS 的根
void tarjan(int u,int in,int rt){
  dfn[u]=low[u]=++tim;
  s1[++t1]=u;
  s2[++t2]=u;
  int ch=0;
  for(auto[v,id]:g[u]){
    if(!dfn[v]){
      tarjan(v,id,rt);
      low[u]=min(low[u],low[v]);
      if(low[v]>dfn[u])bri[id]=1;
      if(low[v]>=dfn[u]){ // 弹出一个点双
        ch++;
        if(u!=rt||ch>1)cut[u]=1;
        vb[++vc].clear();
        int x;
        do{
          x=s1[t1--];
          vb[vc].push_back(x);
        }while(x!=v);
        vb[vc].push_back(u);
      }
    }else if(id!=in){
      low[u]=min(low[u],dfn[v]);
    }
  }
  if(u==rt){
    if(!ch)vb[++vc]={u}; // 孤立点
    t1=0;
  }
  if(low[u]==dfn[u]){ // 弹出一个边双
    ec++;
    int x;
    do{
      x=s2[t2--];
      eb[x]=ec;
    }while(x!=u);
  }
}
void buildRS(int n){
  for(int i=1;i<=n+vc;i++)T[i].clear();
  for(int i=1;i<=vc;i++){
    for(int x:vb[i]){
      T[n+i].push_back(x);
      T[x].push_back(n+i);
    }
  }
}
void buildBT(int n){ // 每条桥两个方向各遍历一次
  for(int i=1;i<=ec;i++)E2[i].clear();
  for(int u=1;u<=n;u++){
    for(auto[v,id]:g[u]){
      if(bri[id])E2[eb[u]].push_back(eb[v]);
    }
  }
}
void init(int n,int m){
  for(int i=1;i<=n;i++){
    g[i].clear();
    dfn[i]=cut[i]=0;
  }
  for(int i=1;i<=m;i++)bri[i]=0;
  tim=t1=t2=vc=ec=0;
}
```

2sat.h

Description: 2-SAT: n 个布尔变量 (编号 1..n), 若干形如 (x 取 a) 或 (y 取 b) 的子句, 求一组满足所有子句的赋值. 建图: 点 x 表示 x 为真, 点 $x+n$ 表示 x 为假. 子句 $p \vee q$ 连边 $\neg p \to q$ 和 $\neg q \to p$, 再用 Tarjan 求 SCC. add(x,a,y,b): 加子句 (x 取 a) 或 (y 取 b), $a, b \in \{0, 1\}$. 强制 x 取 a: add(x,a,x,a). 两者不能同时为真: add(x,0,y,0). solve(): 无解返回 0 (某个 x 与 $x+n$ 在同一 SCC); 有解返回 1, ans[x] 为 1 表示 x 为真. 取法: 选 scc 编号小的那一边 (Tarjan 编号 是逆拓扑序, 编号小的在拓扑序后面). 注意: N 为变量数上限, 数组开 2N; 递归深度可达 $2n$, 要开大栈.

Usage: `init(n); // 多组数据每组调用`

```
for(int i=1;i<=m;i++){
cin>>x>>a>>y>>b; // x 取 a 或 y 取 b
add(x,a,y,b);
}
bool ok=solve(); // 0 表示无解
if(ok) for(int i=1;i<=n;i++) cout<<ans[i]<<' ';
```

Time: $\mathcal{O}(n+m)$ 9de69c, 50 lines

```
const int N=2e5+5; // 变量数，点数为 2N
int n,dfn[N*2],low[N*2],stk[N*2],tp,tim;
int scc[N*2],cnt,ans[N];
bool ins[N*2];
vector<int>g[N*2];
void tarjan(int u){
  dfn[u]=low[u]=++tim;
  stk[++tp]=u;
  ins[u]=1;
  for(int v:g[u]){
    if(!dfn[v]){
      tarjan(v);
      low[u]=min(low[u],low[v]);
    }else if(ins[v]){
      low[u]=min(low[u],dfn[v]);
    }
  }
  if(low[u]==dfn[u]){
    cnt++;
    int v;
    do{
      v=stk[tp--];
      ins[v]=0;
      scc[v]=cnt;
    }while(v!=u);
  }
}
// 子句：(x 取值 a) 或 (y 取值 b)
void add(int x,int a,int y,int b){
  g[a?x+n:x].push_back(b?y:y+n); // x 不取 a 则 y 取 b
  g[b?y+n:y].push_back(a?x:x+n); // y 不取 b 则 x 取 a
}
bool solve(){
  for(int i=1;i<=2*n;i++){
    if(!dfn[i])tarjan(i);
  }
  for(int i=1;i<=n;i++){
    if(scc[i]==scc[i+n])return 0;
    ans[i]=scc[i]<scc[i+n];
  }
  return 1;
}
void init(int _n){
  n=_n;
  tim=tp=cnt=0;
  for(int i=1;i<=2*n;i++){
    g[i].clear();
    dfn[i]=0;
  }
}
```

EulerWalk.h

Description: Hierholzer 求欧拉路径/回路 (每条边恰好走一次), 点 1..n, 边按加入顺序编号 1..m, 有向图或无向图都可以, 允许重边和自环. init(n,dir): dir 为 1 有向, 0 无向. addE(u,v): 加一条边. euler(): 有欧拉路径返回 1, 点序列在 path[1..m+1], 走过的边编号 依次在 ep[1..m] ($ep[i]$ 连接 $path[i]$ 和 $path[i+1]$). 存在条件: 无向图奇度点为 0 或 2 个 (2 个时从奇度点出发); 有向图所有点 出度等于入度, 或恰有一个点出度比入度大 1 (起点) 且一个点入度比 出度大 1; 并且所有边连通 (跑完后 len 不等于 $m+1$ 即不连通). 不满足时返回 0. 要回路: 再判断 path[1] 等于 path[m+1]. 注意: dfs 递归深度可达 m, 要开大栈. N 点数, M 边数上限.

Usage: `init(n,0); // 无向图，多组数据每组调用`

```
for(int i=1;i<=m;i++){
cin>>u>>v;
addE(u,v); // 第 i 条边
}
if(euler()){
for(int i=1;i<=m+1;i++) cout<<path[i]<<' ';
}
```

Time: $\mathcal{O}(n+m)$ 00abb0, 63 lines

```
const int N=1e5+5,M=2e5+5;
int n,m,dir,cur[N],deg[N];
int path[M],len,ep[M],le;
bool vis[M]; // 边是否走过
vector<pair<int,int>>g[N]; // {v边编号,}
void init(int _n,int d){
  n=_n;
  dir=d;
  m=len=le=0;
  for(int i=1;i<=n;i++){
    g[i].clear();
    cur[i]=deg[i]=0;
  }
}
// 有向图 deg 为出度减入度，无向图 deg 为度数
void addE(int u,int v){
  m++;
  vis[m]=0;
  g[u].push_back({v,m});
  if(dir){
    deg[u]++;
    deg[v]--;
  }else{
    g[v].push_back({u,m});
    deg[u]++;
    deg[v]++;
  }
}
void dfs(int u){
  // cur[u] 为当前弧，保证每条边只扫一次
  while(cur[u]<(int)g[u].size()){
    auto[v,id]=g[u][cur[u]++];
    if(vis[id])continue;
    vis[id]=1;
    dfs(v);
    ep[++le]=id;
  }
  path[++len]=u;
}
bool euler(){
  int s=1,c=0; // s 起点，c 特殊点个数
  for(int i=n;i>=1;i--){
    if(g[i].size())s=i;
  }
  for(int i=1;i<=n;i++){
    if(dir){
      if(deg[i]==1){
        c++;
```

```
      s=i;
    }else if(deg[i]!=0&&deg[i]!=-1){
      return 0;
    }
  }else if(deg[i]%2){
    c++;
    s=i;
  }
}
if(c>(dir?1:2))return 0;
dfs(s);
reverse(path+1,path+len+1);
reverse(ep+1,ep+le+1);
return len==m+1;
}
```

FunctionalGraph.h

Description: 基环树森林 / 内向树 (函数图): 每个点 i 恰有一条出边 指向 $a[i]$, 点 $1..n$, $1 \le a[i] \le n$. 每个连通块恰有一个环, 其余点形成挂在环上的树. solve(n) 非递归, 求出: cyc[1..cnt]: 每个环上的点, 按出边顺序 (cyc[k][j] 的下一个点是 cyc[k][(j+1)%len], len 为环长 cyc[k].size()). rt[u]: 从 u 出发第一个到达的环上点 (环上点为自己); dep[u]: u 到 rt[u] 的步数 (尾巴长度, 环上点为 0); bel[u]: 所在环的编号; pos[u]: rt[u] 在 cyc[bel[u]] 中的下标. 从 u 走 k 步 ($k \ge$ dep[u]) 到达 cyc[bel[u]][(pos[u]+k-dep[u])%len]; $k <$ dep[u] 时要用倍增.

Usage: `for(int i=1;i<=n;i++) cin>>a[i];`
`solve(n); // 多组数据直接重新调用`
`bool onCycle=(dep[u]==0);`
`int len=cyc[bel[u]].size(); // u 所在环的长度`

Time: $\mathcal{O}(n)$

733ae0, 41 lines

```
const int N=2e5+5;
int a[N],vis[N],rt[N],dep[N],bel[N],pos[N],cnt;
int st[N],tp; // 当前路径上的点
vector<int>cyc[N];
void solve(int n){
  cnt=0;
  for(int i=1;i<=n;i++)vis[i]=0;
  // vis: 0 未访问, 1 在当前路径上, 2 已处理完
  for(int i=1;i<=n;i++){
    if(vis[i])continue;
    int u=i;
    tp=0;
    while(!vis[u]){
      vis[u]=1;
      st[++tp]=u;
      u=a[u];
    }
    if(vis[u]==1){ // 找到新环, u 在环上
      cyc[++cnt].clear();
      while(vis[u]==1){
        vis[u]=2;
        rt[u]=u;
        dep[u]=0;
        bel[u]=cnt;
        pos[u]=cyc[cnt].size();
        cyc[cnt].push_back(u);
        u=a[u];
      }
    }
    // 倒着处理路径上不在环上的点
    for(;tp;tp--){
      int v=st[tp];
      if(vis[v]==2)continue;
      vis[v]=2;
      rt[v]=rt[a[v]];
      dep[v]=dep[a[v]]+1;
      bel[v]=bel[a[v]];
      pos[v]=pos[a[v]];
    }
  }
}
```

8.2 树

CompressTree.h

Description: 虚树 (单调栈建法). 多次询问, 每次给出 k 个关键点, 总和 不大时, 只保留关键点, 它们两两的 LCA 和根 1, 建一棵点数不超过 $2k$ 的虚树, 在上面做树形 DP. 需要 (由别的模板提供, 本文件不含): 原树 (根为 1) 的 dfs 序 dfn[] 和 int lca(int x,int y), 例如用 HLD 或倍增. build(h,k): h[1..k] 为互不相同的关键点 (会被按 dfn 排序). 结果: vt[u] 为虚树中 u 的儿子 (父到子), 虚树边 (u, v) 代表原树中一条链, 长度为 dep[v]−dep[u]. 用到的虚树点为 vs[1..vn], 根 1 总在虚树中但不一定是关键点 (自己用数组标记关键点). 每次 build 会清空用到的点的 vt, 不需要全局清空.

Usage: `// 预处理原树的 dfn 和 lca 后, 每次询问:`
`cin>>k;`
`for(int i=1;i<=k;i++){`
`cin>>h[i];`
`key[h[i]]=1;`
`}`
`build(h,k);`
`dp(1); // 在 vt 上做 DP`
`for(int i=1;i<=k;i++) key[h[i]]=0; // 清空标记`

Time: $\mathcal{O}(k \log k)$ 加 $\mathcal{O}(k)$ 次 lca

fbcf27, 32 lines

```
const int N=2e5+5;
extern int dfn[]; // 用户提供
int lca(int x,int y); // 用户提供
vector<int>vt[N];
int stk[N],tp,vs[N],vn;
bool cmp(int x,int y){return dfn[x]<dfn[y];}
void push(int x){ // 新点入栈
  vt[x].clear();
  vs[++vn]=x;
  stk[++tp]=x;
}
void build(int*h,int k){
  sort(h+1,h+k+1,cmp);
  tp=vn=0;
  push(1);
  for(int i=1;i<=k;i++){
    if(h[i]==1)continue;
    int l=lca(h[i],stk[tp]);
    if(l!=stk[tp]){
      // 栈顶不在 h[i] 到根的链上, 弹出并连边
      while(dfn[l]<dfn[stk[tp-1]]){
        vt[stk[tp-1]].push_back(stk[tp]);
        tp--;
      }
      int x=stk[tp--];
      if(l!=stk[tp])push(l);
      vt[l].push_back(x);
    }
    push(h[i]);
  }
  for(int i=1;i<tp;i++)vt[stk[i]].push_back(stk[i+1]);
}
```

HLD.h

Description: 重链剖分, 点 $1..n$, 根 rt. 支持路径加/路径和, 子树加/子树和, 求 LCA, 每次 $O(\log n)$ 段区间. 输入: 无向树边存在 g[u] 中. dfs1(rt,0) 求 fa (父亲), dep (深度, 根为 1), siz (子树大小), son (重儿子, 没有为 0); dfs2(rt,rt) 求 top (所在重链的顶), dfn (新编号, $1..n$), rnk (dfn 的逆, rnk[dfn[u]]= u). 每条重链和每棵子树在 dfn 上都是连续区间: 子树 u 为 [dfn[u], dfn[u]+siz[u]−1]. 区间数据结构由用户提供 (本文件不含): upd(l,r,v) 把 dfn 区间 $[l, r]$ 加 v, qry(l,r) 求区间和, 一般接到线段树上, 线段树第 i 个位置的 初值为 a[rnk[i]]. 维护 max 等只需改 qryPath 里的合并. 边权: 把边权放到深的那个点上, 路径操作的最后一段改为 [dfn[x]+1, dfn[y]] (x 等于 y 时跳过), 即不含 LCA. 注意: dfs 递归深度可达 n, 链状树要开大栈.

Usage: `// 把 upd 和 qry 接到自己的线段树上, 例如:`
`void upd(int l,int r,ll v){update(1,1,n,l,r,v);}`
`ll qry(int l,int r){return query(1,1,n,l,r);}`
`// 多组数据: g[1..n] 清空, tim=0`
`for(int i=1;i<n;i++){`
`cin>>u>>v;`
`g[u].push_back(v);`
`g[v].push_back(u);`
`}`
`dfs1(rt,0);`
`dfs2(rt,rt);`
`for(int i=1;i<=n;i++) b[i]=a[rnk[i]];`
`build(1,1,n); // 用 b 建线段树`
`updPath(x,y,v); ll s=qryPath(x,y); // 路径`
`updSub(x,v); ll t=qrySub(x); // 子树`
`int l=lca(x,y);`

Time: 路径操作 $\mathcal{O}(\log^2 n)$, 子树操作 $\mathcal{O}(\log n)$

e2ff8b, 55 lines

```
const int N=2e5+5;
void upd(int l,int r,ll v); // 用户提供
ll qry(int l,int r); // 用户提供
vector<int>g[N];
int fa[N],dep[N],siz[N],son[N];
int top[N],dfn[N],rnk[N],tim;
void dfs1(int u,int f){
  fa[u]=f;
  dep[u]=dep[f]+1;
  siz[u]=1;
  son[u]=0;
  for(int v:g[u]){
    if(v==f)continue;
    dfs1(v,u);
    siz[u]+=siz[v];
    if(siz[v]>siz[son[u]])son[u]=v;
  }
}
void dfs2(int u,int t){
  top[u]=t;
  dfn[u]=++tim;
  rnk[tim]=u;
  if(son[u])dfs2(son[u],t); // 先走重儿子, 重链连续
  for(int v:g[u]){
    if(v!=fa[u]&&v!=son[u])dfs2(v,v);
  }
}
int lca(int x,int y){
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]])swap(x,y);
    x=fa[top[x]];
  }
  return dep[x]<dep[y]?x:y;
}
void updPath(int x,int y,ll v){
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]])swap(x,y);
    upd(dfn[top[x]],dfn[x],v);
    x=fa[top[x]];
  }
  if(dep[x]>dep[y])swap(x,y);
  upd(dfn[x],dfn[y],v);
}
ll qryPath(int x,int y){
  ll res=0;
  while(top[x]!=top[y]){
    if(dep[top[x]]<dep[top[y]])swap(x,y);
    res+=qry(dfn[top[x]],dfn[x]);
    x=fa[top[x]];
```

```
  }
  if(dep[x]>dep[y])swap(x,y);
  return res+qry(dfn[x],dfn[y]);
}
void updSub(int x,ll v){upd(dfn[x],dfn[x]+siz[x]-1,v);}
ll qrySub(int x){return qry(dfn[x],dfn[x]+siz[x]-1);}
```

LinkCutTree.h

Description: LCT 动态树, 点 1..n, 维护一个森林, 支持连边, 删边, 判连通, 单点改值, 路径求和, 每次均摊 $O(\log n)$. 每条实链用一棵 splay 维护 (中序为深度从小到大); 对 splay 的根, fa[x] 表示虚边指向的父亲. 点权 val[], sum[] 为 splay 子树和. link(x,y): 若不连通则连边 (x, y). cut(x,y): 若有边 (x, y) 则删掉. connected(x,y): 是否连通. findrt(x): x 所在树的根. makert(x): 把 x 设为所在树的根. modify(x,v): val[x] 改为 v. split(x,y): 把 x 到 y 的路径拉成一棵 splay, 根为 y, 之后 sum[y] 即为路径和 (要求 x, y 连通). 维护 max/xor 等只改 pushup; 加区间修改要再加一个标记并在 pushdown 下传. N 为点数上限, 没有递归.

Usage:
```
for(int i=1;i<=n;i++) cin>>val[i];
init(n); // 多组数据每组调用 (先填 val)
link(x,y); // 加边
cut(x,y); // 删边
if(connected(x,y)){
split(x,y);
ll s=sum[y]; // 路径 x 到 y 的点权和
}
modify(x,v);
```

Time: 均摊 $\mathcal{O}(\log n)$

435305, 99 lines

```
const int N=3e5+5;
int ch[N][2],fa[N],rev[N],st[N];
ll val[N],sum[N];
// x 不是所在 splay 的根
bool nrt(int x){return ch[fa[x]][0]==x||ch[fa[x]][1]==x;}
void pushup(int x){sum[x]=sum[ch[x][0]]+sum[ch[x][1]]+val[x];}
void pushr(int x){ // 翻转 x 的子树
  swap(ch[x][0],ch[x][1]);
  rev[x]^=1;
}
void pushdown(int x){
  if(rev[x]){
    if(ch[x][0])pushr(ch[x][0]);
    if(ch[x][1])pushr(ch[x][1]);
    rev[x]=0;
  }
}
void rotate(int x){
  int y=fa[x],z=fa[y];
  int k=ch[y][1]==x,w=ch[x][!k];
  if(nrt(y))ch[z][ch[z][1]==y]=x;
  ch[x][!k]=y;
  ch[y][k]=w;
  if(w)fa[w]=y;
  fa[y]=x;
  fa[x]=z;
  pushup(y);
}
void splay(int x){
  // 先从上到下 pushdown
  int y=x,tp=0;
  st[++tp]=y;
  while(nrt(y)){
    y=fa[y];
    st[++tp]=y;
  }
  while(tp)pushdown(st[tp--]);
  while(nrt(x)){
    y=fa[x];
    int z=fa[y];
    if(nrt(y)){
      if((ch[y][0]==x)^(ch[z][0]==y))rotate(x);
      else rotate(y);
    }
    rotate(x);
  }
  pushup(x);
}
void access(int x){ // 打通根到 x 的实链
  for(int y=0;x;y=x,x=fa[x]){
    splay(x);
    ch[x][1]=y;
    pushup(x);
  }
}
void makert(int x){
  access(x);
  splay(x);
  pushr(x);
}
int findrt(int x){
  access(x);
  splay(x);
  while(ch[x][0]){
    pushdown(x);
    x=ch[x][0];
  }
  splay(x);
  return x;
}
void split(int x,int y){
  makert(x);
  access(y);
  splay(y);
}
bool connected(int x,int y){return findrt(x)==findrt(y);}
void link(int x,int y){
  makert(x);
  if(findrt(y)!=x)fa[x]=y;
}
void cut(int x,int y){
  makert(x);
  if(findrt(y)==x&&fa[y]==x&&!ch[y][0]){
    fa[y]=0;
    ch[x][1]=0;
    pushup(x);
  }
}
void modify(int x,ll v){
  splay(x);
  val[x]=v;
  pushup(x);
}
void init(int n){
  for(int i=1;i<=n;i++){
    ch[i][0]=ch[i][1]=fa[i]=rev[i]=0;
    sum[i]=val[i];
  }
}
```

8.3 数学

8.3.1 生成树计数

矩阵树定理. 建 $n \times n$ 矩阵 mat (初始全 0), 对每条有向边 $a \to b$ 执行 mat[a][b]--, mat[b][b]++, 即对角线放每个点的入度, 非对角线放负的边数 (重边要累加).

- **有向图**: 把第 i 行和第 i 列 (同一个编号) 都删掉, 剩下 $(n-1) \times (n-1)$ 矩阵的行列式, 就是以 i 为根、所有边都从根往外指 (外向树) 的生成树个数. 要算内向树 (边都指向根), 对角线改放出度: 对每条边 $a \to b$ 执行 mat[a][b]--, mat[a][a]++.
- **无向图**: 每条无向边当作两条有向边, 即对角线是度数, $mat[a][b]$ 为负的边数 (拉普拉斯矩阵). 删去任意一个 i 的第 i 行和第 i 列, 行列式就是生成树个数.
- 答案很大时取模, 用 IntDeterminant.h 的 detMod.

8.3.2 Erdős–Gallai 定理

度数为 $d_1 \geq \cdots \geq d_n$ 的简单图存在, 当且仅当 $d_1 + \cdots + d_n$ 为偶数, 且对每个 $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

几何 (9)

9.1 几何基础

Point.h

Description: 平面点 / 向量, 几何部分其余文件都 include 它. 坐标类型为 T: 默认 double; 坐标都是整数且要求精确 (凸包, 面积, 判断位置等) 时改成 typedef ll T, 此时 unit, rotate, 除法以及求交点的函数会被截断, 不要用; cross, dot, len2 是坐标的二次式, 坐标绝对值约 10^9 以内才不溢出. sgn(x) 返回 x 的符号, $|x| \leq eps$ 时为 0. eps 是绝对误差 (默认 1e-9), 坐标很大时要调大. == 和 < (先比 x 再比 y) 也带 eps, 可直接用于 sort, unique, set. cross(a,b) 为 $a \times b$; cross(a,b,c) 为 $(b-a) \times (c-a)$: 正数表示 a, b, c 逆时针 (c 在 a 看向 b 的左侧), 负数为顺时针, 0 为共线. angle 返回极角, 范围 $[-\pi, \pi]$. 多边形统一用 vector<Point> 存, 下标 0..n-1.

Usage:
```
Point a(1,2),b(3,4),c(0,5);
Point s=a+b,d=b-a,m=a*2; // 向量运算
T k=cross(a,b),t=dot(a,b); // 叉积, 点积
double l=len(b-a); // 两点距离
if(sgn(cross(a,b,c))>0){} // a,b,c 逆时针
Point r=rotate(a,PI/2); // 逆时针转 90 度
vector<Point> poly={a,b,c}; // 多边形 poly[0..n-1]
```

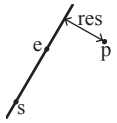
ddb67f, 35 lines

```
typedef double T; // 整数坐标可改为 ll
const double eps=1e-9;
const double PI=acos(-1.0);
int sgn(double x){ // 符号, 绝对值不超过 eps 时为 0
  if(x>eps)return 1;
  if(x<-eps)return -1;
  return 0;
}
struct Point{
  T x,y;
  Point(T x=0,T y=0):x(x),y(y){}
  Point operator+(Point b)const{return Point(x+b.x,y+b.y);}
  Point operator-(Point b)const{return Point(x-b.x,y-b.y);}
  Point operator*(T k)const{return Point(x*k,y*k);}
  Point operator/(T k)const{return Point(x/k,y/k);}
  bool operator==(Point b)const{
    return sgn(x-b.x)==0&&sgn(y-b.y)==0;
  }
  bool operator<(Point b)const{ // 先比 x 再比 y
```

```
    if(sgn(x-b.x)!=0)return x<b.x;
    return sgn(y-b.y)<0;
  }
};
T dot(Point a,Point b){return a.x*b.x+a.y*b.y;}
T cross(Point a,Point b){return a.x*b.y-a.y*b.x;}
// (b-a) 叉乘 (c-a), 大于 0 表示 a,b,c 逆时针
T cross(Point a,Point b,Point c){return cross(b-a,c-a);}
T len2(Point a){return dot(a,a);} // 长度的平方
double len(Point a){return sqrt((double)len2(a));}
double angle(Point a){return atan2(a.y,a.x);} // 极角 [-pi,pi]
Point perp(Point a){return Point(-a.y,a.x);} // 逆时针转 90 度
Point unit(Point a){return a/len(a);} // 单位向量
Point rotate(Point a,double t){ // 绕原点逆时针转 t 弧度
  return Point(a.x*cos(t)-a.y*sin(t),a.x*sin(t)+a.y*cos(t));
}
```

lineDistance.h

Description:



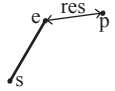
返回点 p 到直线 ab 的有向距离: 从 a 看向 b, p 在左侧为正, 右侧为负, 在直线上为 0. 取绝对值即点到直线的距离. 要求 a 与 b 不重合 (否则得 nan). T 为 ll 时也可用 (返回 double).

Usage:
```
Point a(0,0),b(2,0),p(1,3);
double d=lineDist(a,b,p); // 得 3, 正数表示 p 在左侧
double h=fabs(lineDist(a,b,p)); // 点到直线距离
```
Time: $\mathcal{O}(1)$

"Point.h" 2d2735, 3 lines

```
double lineDist(Point a,Point b,Point p){
  return cross(a,b,p)/len(b-a);
}
```

SegmentDistance.h



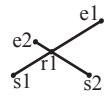
Description:
返回点 p 到线段 se 的最短距离 (端点包含在线段内). s 与 e 可以重合, 此时为两点距离. p 在 se 外侧时取到端点的距离, 否则取到直线的距离. T 为 double 或 ll 均可.

Usage:
```
Point s(0,0),e(2,2),p(1,1),q(3,3);
double d1=segDist(s,e,p); // 得 0, 即 p 在线段上
double d2=segDist(s,e,q); // 得 sqrt(2), 到端点 e
```
Time: $\mathcal{O}(1)$

"Point.h" c6b05b, 6 lines

```
double segDist(Point s,Point e,Point p){
  // 投影落在 s 外侧或 e 外侧时取端点距离
  if(s==e||sgn(dot(p-s,e-s))<0)return len(p-s);
  if(sgn(dot(p-e,s-e))<0)return len(p-e);
  return fabs((double)cross(s,e,p))/len(e-s);
}
```

SegmentIntersection.h



Description:
求线段 s1-e1 与线段 s2-e2 的交点 (端点算在线段上). 返回 vector: 大小为 0 表示不相交; 大小为 1 表示唯一交点 (包括端点相碰); 大小为 2 表示两线段共线且重叠, 返回公共线段的两个端点 (按 x 再按 y 排序). 线段可退化成一个点. T 为 ll 时若交点不是整点则结果错误; 中间有三个坐标的乘积, 注意溢出.

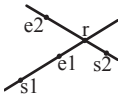
Usage:
```
Point s1(0,0),e1(2,2),s2(0,2),e2(2,0);
vector<Point> v=segInter(s1,e1,s2,e2);
if(v.size()==1){} // 唯一交点 v[0], 这里是 (1,1)
if(v.size()==2){} // 重叠部分为线段 v[0]-v[1]
```
Time: $\mathcal{O}(1)$

"Point.h", "OnSegment.h" 44c49c, 13 lines

```
vector<Point>segInter(Point a,Point b,Point c,Point d){
  T oa=cross(c,d,a),ob=cross(c,d,b);
  T oc=cross(a,b,c),od=cross(a,b,d);
  // 交点唯一且不是端点
  if(sgn(oa)*sgn(ob)<0&&sgn(oc)*sgn(od)<0)
    return {(a*ob-b*oa)/(ob-oa)};
  set<Point>s;
  if(onSegment(c,d,a))s.insert(a);
  if(onSegment(c,d,b))s.insert(b);
  if(onSegment(a,b,c))s.insert(c);
  if(onSegment(a,b,d))s.insert(d);
  return vector<Point>(s.begin(),s.end());
}
```

lineIntersection.h

Description:
求过 s1,e1 的直线与过 s2,e2 的直线的交点 (直线, 不是线段). 返回 {状态, 交点}: 状态 1 表示有唯一交点; 0 表示平行不重合, 无交点; -1 表示两直线重合. 后两种情况交点返回 (0,0). 要求 s1, e1 不重合, s2, e2 不重合. T 为 ll 时若交点不是整点则结果错误; 中间有三个坐标的乘积, 注意溢出.



Usage:
```
Point s1(0,0),e1(1,1),s2(0,2),e2(2,0);
pair<int,Point> r=lineInter(s1,e1,s2,e2);
if(r.first==1){} // 交点为 r.second, 这里是 (1,1)
```
Time: $\mathcal{O}(1)$

"Point.h" 81c471, 10 lines

```
pair<int,Point>lineInter(Point s1,Point e1,Point s2,Point e2){
  T d=cross(e1-s1,e2-s2);
  if(sgn(d)==0){ // 平行, 再判断是否重合
    int same=sgn(cross(s1,e1,s2))==0;
    return {-same,Point(0,0)};
  }
  T p=cross(s2,e1,e2);
  T q=cross(s2,e2,s1);
  return {1,(s1*p+e1*q)/d};
}
```

sideOf.h

Description: 判断点 p 在有向直线 $s \to e$ 的哪一侧: 返回 1 表示左侧 (逆时针方向), 0 表示在直线上, -1 表示右侧. 带参数 lim 的版本按距离判断: p 到直线的距离不超过 lim 时返回 0, 要求 $s \neq e$. T 为 ll 时第一个版本结果精确, 注意乘积溢出.

Usage:
```
Point p1(0,0),p2(1,0),q(5,1);
int s=sideOf(p1,p2,q); // 1, q 在 p1 看向 p2 的左侧
int t=sideOf(p1,p2,q,1e-6); // 距离不超过 1e-6 算在线上
```
Time: $\mathcal{O}(1)$

"Point.h" 60e8a6, 8 lines

```
int sideOf(Point s,Point e,Point p){
  return sgn(cross(s,e,p));
}
int sideOf(Point s,Point e,Point p,double lim){
  double a=cross(s,e,p);
  double l=len(e-s)*lim;
  return (a>l)-(a<-l);
}
```

OnSegment.h

Description: 判断点 p 是否在线段 se 上 (含两个端点), 是则返回 true. 即 p, s, e 共线且 p 在 s 与 e 之间. s 与 e 重合时判断 p 是否就是这个点. 浮点坐标的误差由 sgn 的 eps 处理; T 为 ll 时结果精确, 注意乘积溢出.

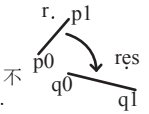
Usage:
```
Point s(0,0),e(4,2);
bool a=onSegment(s,e,Point(2,1)); // true
bool b=onSegment(s,e,Point(6,3)); // false, 在延长线上
```
Time: $\mathcal{O}(1)$

"Point.h" d05f9c, 3 lines

```
bool onSegment(Point s,Point e,Point p){
  return sgn(cross(p,s,e))==0&&sgn(dot(s-p,e-p))<=0;
}
```

linearTransformation.h

Description:
求把 p0 变到 q0, p1 变到 q1 的相似变换 (平移 + 旋转 + 缩放, 不翻转) 作用在点 r 上的结果. 要求 p0 与 p1 不重合, T 为 double.



Usage:
```
Point p0(0,0),p1(1,0),q0(2,2),q1(2,4),r(1,1);
Point r2=linearTransformation(p0,p1,q0,q1,r); // (0,4)
```
Time: $\mathcal{O}(1)$

"Point.h" 8ca2c9, 6 lines

```
Point linearTransformation(Point p0,Point p1,Point q0,
    Point q1,Point r){
  Point dp=p1-p0,dq=q1-q0;
  Point num(cross(dp,dq),dot(dp,dq));
  return q0+Point(cross(r-p0,num),dot(r-p0,num))/len2(dp);
}
```

Angle.h

Description: 精确的极角类, 不用浮点数, 用于极角排序和极角扫描 (双指针). 一个 Angle 是 int 向量 (x, y) 再加上绕原点 t 圈, 表示角度 $\theta + 2\pi t$, 其中 $\theta \in [0, 2\pi)$ 是 (x, y) 的极角 (从 x 轴正方向逆时针量). 比较顺序: 先比 t, 再比 θ; 方向相同的向量视为相等 (不比长度). t90, t180, t360 分别返回加上 $\pi/2$, π, 2π 后的角. segmentAngles(a,b): 两个点 (t=0) 看成从原点出发的方向, 返回覆盖线段 ab 的最小角区间 {first, second}. a+b: 点 a 加向量 b, 圈数取与 a 相差不超过 π 的那个. angleDiff(a,b): 从 a 转到 b 的角 (b 减 a). 坐标绝对值不超过 10^9 (内部用 ll 乘法), 向量不能为 $(0, 0)$.

Usage:
```
vector<Angle> v; // v[0..n-1]
for(int i=0;i<n;i++)v.push_back(Angle(x[i],y[i]));
sort(v.begin(),v.end()); // 极角排序
// 双指针: 把每个角加一圈 (t360) 接在后面
for(int i=0;i<n;i++)v.push_back(v[i].t360());
for(int i=0,j=0;i<n;i++){
while(v[j]<v[i].t180())j++;
// v[i..j-1] 是极角在 [v[i], v[i]+pi) 内的角
}
```
Time: 比较 $\mathcal{O}(1)$, 排序 $\mathcal{O}(n \log n)$

8fe5c3, 34 lines

```
struct Angle{
  int x,y,t; // 向量 (x,y) 再加 t 圈
  Angle(int x,int y,int t=0):x(x),y(y),t(t){}
  Angle operator-(Angle b)const{return Angle(x-b.x,y-b.y,t);}
  int half()const{ // 角在 [pi, 2pi) 内时为 1
    assert(x||y);
    return y<0||(y==0&&x<0);
  }
  Angle t90()const{return Angle(-y,x,t+(half()&&x>=0));}
  Angle t180()const{return Angle(-x,-y,t+half());}
  Angle t360()const{return Angle(x,y,t+1);}
};
bool operator<(Angle a,Angle b){
  // 末尾再加上长度的平方, 可在极角相同时按长度排序
  return make_tuple(a.t,a.half(),a.y*(ll)b.x)<
    make_tuple(b.t,b.half(),a.x*(ll)b.y);
}
// 覆盖线段 ab 的最小角区间, 从 first 逆时针转到 second
pair<Angle,Angle>segmentAngles(Angle a,Angle b){
  if(b<a)swap(a,b);
  if(b<a.t180())return make_pair(a,b);
  return make_pair(b,a.t360());
}
Angle operator+(Angle a,Angle b){ // 点 a 加向量 b
  Angle r(a.x+b.x,a.y+b.y,a.t);
  if(a.t180()<r)r.t--;
  if(r.t180()<a)return r.t360();
  return r;
}
Angle angleDiff(Angle a,Angle b){ // 角 b 减去角 a
  int tu=b.t-a.t;
  a.t=b.t;
  return Angle(a.x*b.x+a.y*b.y,a.x*b.y-a.y*b.x,tu-(b<a));
```

```
}
```

9.2 圆

CircleIntersection.h

Description: 求圆 $(a, r1)$ 与圆 $(b, r2)$ 的交点 (圆周的交点). 有交点时写入 out 并返回 true: 相交时 out.first, out.second 为两个交点 (从 a 看向 b, first 在左侧); 相切时两个交点相同. 两圆相离或一圆在另一圆内部时返回 false. 两圆心重合时返回 false, 此时要求 $r1 \neq r2$ (有 assert, 同心同半径有无穷多交点). 要求 T 为 double.

Usage:
```
Point a(0,0),b(2,0);
pair<Point,Point> out;
if(circleInter(a,b,2,2,out)){
// out.first=(1,sqrt(3)), out.second=(1,-sqrt(3))
}
```

Time: $\mathcal{O}(1)$

"Point.h" 393a65, 17 lines

```
bool circleInter(Point a,Point b,double r1,double r2,
    pair<Point,Point>&out){
  if(a==b){
    assert(r1!=r2);
    return false;
  }
  Point v=b-a;
  double d2=len2(v),sum=r1+r2,dif=r1-r2;
  if(sum*sum<d2||dif*dif>d2)return false;
  // p: 交点连线中点在 ab 上的比例, h2: 半弦长平方的比例
  double p=(d2+r1*r1-r2*r2)/(d2*2);
  double h2=r1*r1-p*p*d2;
  Point mid=a+v*p;
  Point per=perp(v)*sqrt(max(0.0,h2)/d2);
  out={mid+per,mid-per};
  return true;
}
```

CircleTangents.h

Description: 求圆 $(c1, r1)$ 与圆 $(c2, r2)$ 的外公切线; 把 r2 取负则求内公切线; 令 r2 = 0 则求点 c2 到圆 1 的切线. 要求 T 为 double. 返回 vector, 每个元素是一条切线 {圆 1 上的切点, 圆 2 上的切点}, 共 0, 1 或 2 条: 0 条: 一圆在另一圆内部 (求内公切线时还包括两圆相交), 或两圆心重合; 1 条: 两圆相切 (此时 first 与 second 是同一点, 切线垂直于两圆心连线); 2 条: 一般情况. 要四条公切线时分别用 r2 和 -r2 各调用一次.

Usage:
```
Point c1(0,0),c2(5,0);
vector<pair<Point,Point>> v=tangents(c1,1,c2,2); // 外
vector<pair<Point,Point>> w=tangents(c1,1,c2,-2); // 内
// v[i].first 在圆 1 上, v[i].second 在圆 2 上
```

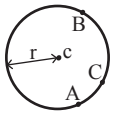
Time: $\mathcal{O}(1)$

"Point.h" 8de4a3, 14 lines

```
vector<pair<Point,Point>>tangents(Point c1,double r1,
    Point c2,double r2){
  Point d=c2-c1;
  double dr=r1-r2,d2=len2(d),h2=d2-dr*dr;
  if(sgn(d2)==0||sgn(h2)<0)return {};
  if(sgn(h2)==0)h2=0; // 相切, 只有一条
  vector<pair<Point,Point>>out;
  for(int s=-1;s<=1;s+=2){
    Point v=(d*dr+perp(d)*(sqrt(h2)*s))/d2;
    out.push_back({c1+v*r1,c2+v*r2});
  }
  if(h2==0)out.pop_back();
  return out;
}
```

circumcircle.h

Description:



三角形的外接圆 (经过三个顶点的圆). ccRadius 返回过 A, B, C 三点的圆的半径, ccCenter 返回该圆的圆心. 要求 T 为 double, 三点不共线 (否则除以 0).

Usage:
```
Point A(0,0),B(2,0),C(0,2);
double r=ccRadius(A,B,C); // sqrt(2)
Point o=ccCenter(A,B,C); // (1,1)
```

Time: $\mathcal{O}(1)$

"Point.h" 1a3baa, 7 lines

```
double ccRadius(Point A,Point B,Point C){
  return len(B-A)*len(C-B)*len(A-C)/fabs(cross(B-A,C-A))/2;
}
Point ccCenter(Point A,Point B,Point C){
  Point b=C-A,c=B-A;
  return A+perp(b*len2(c)-c*len2(b))/cross(b,c)/2;
}
```

MinimumEnclosingCircle.h

Description: 最小圆覆盖 (随机增量法): 求半径最小的圆, 使所有点都在圆内或圆上. 输入点集 vector<Point> (下标 0..n-1, 至少 1 个点, 可有重复点和共线点), 返回 {圆心, 半径}. 要求 T 为 double. 判断点在圆外时用相对误差 1e-8. 函数内部会打乱点的顺序 (传值, 不影响原数组).

Usage:
```
vector<Point> ps={Point(0,0),Point(2,0),Point(1,1)};
pair<Point,double> c=mec(ps);
// 圆心 c.first=(1,0), 半径 c.second=1
```

Time: 期望 $\mathcal{O}(n)$

"circumcircle.h" aa2782, 23 lines

```
pair<Point,double>mec(vector<Point>ps){
  shuffle(ps.begin(),ps.end(),mt19937(time(0)));
  int n=ps.size();
  Point o=ps[0];
  double r=0;
  double E=1+1e-8; // 相对误差
  for(int i=0;i<n;i++){
    if(len(o-ps[i])<=r*E)continue;
    o=ps[i];
    r=0;
    for(int j=0;j<i;j++){
      if(len(o-ps[j])<=r*E)continue;
      o=(ps[i]+ps[j])/2;
      r=len(o-ps[i]);
      for(int k=0;k<j;k++){
        if(len(o-ps[k])<=r*E)continue;
        o=ccCenter(ps[i],ps[j],ps[k]);
        r=len(o-ps[i]);
      }
    }
  }
  return {o,r};
}
```

9.3 多边形

InsidePolygon.h

Description: 射线法判断点 a 是否在多边形 p 内, 在内返回 true. 多边形为 vector<Point> (下标 0..n-1), 顺时针或逆时针均可, 可以是凹多边形 (要求简单多边形). 点在边界上时: strict 为 true 返回 false, strict 为 false 返回 true. 边界判断用 onSegment (带 eps). T 为 ll 时结果精确, 中间有乘积, 注意溢出.

Usage:
```
vector<Point> p={Point(0,0),Point(4,0),Point(0,4)};
bool a=inPolygon(p,Point(1,1)); // true, 严格在内部
bool b=inPolygon(p,Point(2,0),false); // true, 含边界
bool c=inPolygon(p,Point(2,0),true); // false
```

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h" 3a7f00, 10 lines

```
bool inPolygon(vector<Point>&p,Point a,bool strict=true){
  int cnt=0,n=p.size();
  for(int i=0;i<n;i++){
    Point q=p[(i+1)%n];
    if(onSegment(p[i],q,a))return !strict;
    // 射线法: 向右的水平射线与边相交的次数
    cnt^=((a.y<p[i].y)-(a.y<q.y))*cross(a,p[i],q)>0;
  }
  return cnt;
}
```

PolygonArea.h

Description: 返回多边形有向面积的两倍 (鞋带公式). 多边形为 vector<Point> (下标 0..n-1), 顶点逆时针给出时为正, 顺时针为负; 面积为 fabs 后除以 2. 也可用来判断多边形方向. 空多边形返回 0. T 为 ll 时结果精确 (面积两倍是整数), 注意溢出.

Usage:
```
vector<Point> v={Point(0,0),Point(4,0),Point(0,3)};
T a2=polygonArea2(v); // 12, 逆时针为正
double S=fabs(a2)/2; // 面积 6
if(a2<0)reverse(v.begin(),v.end()); // 改成逆时针
```

Time: $\mathcal{O}(n)$

"Point.h" db2374, 8 lines

```
T polygonArea2(vector<Point>&v){
  int n=v.size();
  T a=0;
  for(int i=0;i<n;i++){
    a+=cross(v[i],v[(i+1)%n]);
  }
  return a;
}
```

PolygonCenter.h

Description: 返回多边形 (实心均匀薄板) 的重心 (质心), 不是顶点的平均值. 多边形为 vector<Point> (下标 0..n-1), 顺时针或逆时针均可, 可以是凹多边形; 面积不能为 0. 要求 T 为 double.

Usage:
```
vector<Point>
v={Point(0,0),Point(2,0),Point(2,2),Point(0,2)};
Point c=polygonCenter(v); // (1,1)
```

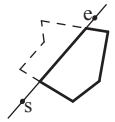
Time: $\mathcal{O}(n)$

"Point.h" 3acb21, 11 lines

```
Point polygonCenter(vector<Point>&v){
  int n=v.size();
  Point res(0,0);
  double A=0; // 面积的两倍
  for(int i=0;i<n;i++){
    Point p=v[i],q=v[(i+1)%n];
    res=res+(p+q)*cross(p,q);
    A+=cross(p,q);
  }
  return res/A/3;
}
```

PolygonCut.h

Description:



用从 s 到 e 的有向直线切割多边形, 切掉直线左侧的部分, 保留右侧 (含直线上的点), 返回剩余部分的顶点, 顺序与原多边形相同. 全部被切掉时返回空. 多边形为 vector<Point> (下标 0..n-1). 对凸多边形结果仍是凸多边形; 凹多边形可能被切成几块, 结果用直线上的边连成一个 (面积仍正确). 要求 T 为 double.

Usage:
```
vector<Point> p; // 正方形 [0,1]x[-1,1], 逆时针
p.push_back(Point(0,-1));
p.push_back(Point(1,-1));
p.push_back(Point(1,1));
p.push_back(Point(0,1));
vector<Point> q=polygonCut(p,Point(0,0),Point(1,0));
// 切掉 x 轴上方, q 为 y 不大于 0 的部分
```

Time: $\mathcal{O}(n)$

"Point.h" 9e72e8, 14 lines

```
vector<Point>polygonCut(vector<Point>&poly,Point s,Point e){
  vector<Point>res;
```

```
  int n=poly.size();
  for(int i=0;i<n;i++){
    Point cur=poly[i];
    Point prev=i?poly[i-1]:poly[n-1];
    double a=cross(s,e,cur),b=cross(s,e,prev);
    if((a<0)!=(b<0)){ // 边与直线相交，加入交点
      res.push_back(cur+(prev-cur)*(a/(a-b)));
    }
    if(a<0)res.push_back(cur);
  }
  return res;
}
```

HalfPlane.h

Description: 半平面交 (极角排序 + 双端队列). 每条有向直线 Line{s, e} 表示它左侧的半平面 (含边界). 返回所有半平面的交 (凸多边形) 的顶点, 逆时针, 下标 0..m-1; 交集为空或面积为 0 时返回空 (面积极小时也可能返回退化的多边形, 按面积判断即可). 要求交集有界: 不确定时先加入一个大矩形的 4 条边 (见用法). 方向相同的直线只保留最靠里的一条. 要求 T 为 double, 判断用 eps.

Usage:
```
vector<Line> L; // 每条直线保留左侧
L.push_back({Point(0,0),Point(1,0)}); // y 不小于 0
double B=1e7; // 逆时针的大矩形，保证交集有界
Point c[4]={Point(-B,-B),Point(B,-B),Point(B,B),Point(-B,B)};
for(int i=0;i<4;i++)L.push_back({c[i],c[(i+1)%4]});
vector<Point> poly=halfPlane(L); // 空表示无交
double S=fabs(polygonArea2(poly))/2; // 交的面积
```

Time: $\mathcal{O}(n \log n)$

"Point.h" 3ab07e, 37 lines

```
struct Line{
  Point s,e; // 有向直线 s 到 e，保留左侧
};
double ang(Line l){return angle(l.e-l.s);}
bool cmpLine(Line a,Line b){ // 按极角排序，相同时靠左的在前
  if(sgn(ang(a)-ang(b))!=0)return ang(a)<ang(b);
  return sgn(cross(a.s,a.e,b.s))<0;
}
// p 不在 l 的严格左侧
bool onRight(Line l,Point p){return sgn(cross(l.s,l.e,p))<=0;}
Point inter(Line a,Line b){ // 两直线交点，要求不平行
  Point u=a.e-a.s,v=b.e-b.s;
  return a.s+u*(cross(v,b.s-a.s)/cross(v,u));
}
vector<Point>halfPlane(vector<Line>L){
  sort(L.begin(),L.end(),cmpLine);
  int n=L.size(),h=0,t=-1;
  vector<Line>q(n); // 队列 q[h..t]
  vector<Point>p(n); // p[i] 为 q[i] 与 q[i+1] 的交点
  for(int i=0;i<n;i++){
    // 同向的只留第一条，即最靠左的
    if(i>0&&sgn(ang(L[i])-ang(L[i-1]))==0)continue;
    while(h<t&&onRight(L[i],p[t-1]))t--;
    while(h<t&&onRight(L[i],p[h]))h++;
    // 与队尾反向平行，交集为空
    if(h<=t&&sgn(cross(L[i].e-L[i].s,q[t].e-q[t].s))==0){
      return {};
    }
    t++;
    q[t]=L[i];
    if(h<t)p[t-1]=inter(q[t-1],q[t]);
  }
  while(h<t&&onRight(q[h],p[t-1]))t--;
  if(t-h<2)return {};
  p[t]=inter(q[t],q[h]);
  return vector<Point>(p.begin()+h,p.begin()+t+1);
}
```

ConvexHull.h



Description:
Andrew 单调链求凸包. 输入点集 vector<Point> (可有重复点, 共线点), 返回凸包顶点, 逆时针, 下标 0..m-1, 从最左 (x 最小, 再 y 最小) 的点开始. 不保留共线点 (凸包边上的点不算顶点), 重复点只算一次. 所有点重合时返回 1 个点, 全部共线时返回 2 个端点. T 可为 ll (结果精确).

Usage:
```
vector<Point> pts; // pts[0..n-1]
for(int i=0;i<n;i++)pts.push_back(Point(x[i],y[i]));
vector<Point> h=convexHull(pts); // h[0..m-1] 逆时针
int m=h.size(); // 边为 h[i] 到 h[(i+1)%m]
```

Time: $\mathcal{O}(n \log n)$

"Point.h" 7b4e27, 21 lines

```
vector<Point>convexHull(vector<Point>p){
  sort(p.begin(),p.end());
  p.erase(unique(p.begin(),p.end()),p.end());
  int n=p.size(),k=0;
  if(n<=1)return p;
  vector<Point>h(2*n);
  // 要保留共线点，两处改成严格小于 0，全部共线时要特判
  for(int i=0;i<n;i++){ // 下凸壳
    while(k>=2&&sgn(cross(h[k-2],h[k-1],p[i]))<=0)k--;
    h[k]=p[i];
    k++;
  }
  int lo=k;
  for(int i=n-2;i>=0;i--){ // 上凸壳
    while(k>lo&&sgn(cross(h[k-2],h[k-1],p[i]))<=0)k--;
    h[k]=p[i];
    k++;
  }
  h.resize(k-1); // 最后一个点是 p[0]，去掉
  return h;
}
```

HullDiameter.h

Description: 旋转卡壳求凸包直径 (点集中距离最远的两点), 返回最远距离的平方. 输入为逆时针凸包 (无重复点和共线点, 即 convexHull 的结果), 可以只有 1 或 2 个点. 求原始点集的最远点对时先求凸包. T 为 ll 时结果精确, 坐标约 10^9 以内.

Usage:
```
vector<Point> h=convexHull(pts);
T d2=hullDiameter(h); // 直径的平方
double d=sqrt((double)d2); // 直径
```

Time: $\mathcal{O}(n)$

"Point.h" e6ead4, 15 lines

```
T hullDiameter(vector<Point>&h){
  int n=h.size();
  if(n==1)return 0;
  if(n==2)return len2(h[0]-h[1]);
  T res=0;
  int j=1;
  for(int i=0;i<n;i++){
    Point a=h[i],b=h[(i+1)%n];
    // h[j+1] 离边 ab 更远时移动 j
    while(sgn(cross(b-a,h[(j+1)%n]-h[j]))>0)j=(j+1)%n;
    res=max(res,len2(a-h[j]));
    res=max(res,len2(b-h[j]));
  }
  return res;
}
```

PointInsideHull.h

Description: 二分判断点 p 是否在凸包 l 内, 在内返回 true. l 为逆时针凸包 (无重复点和共线点, 即 convexHull 的结果, 下标 0..n-1), 可以只有 1 或 2 个点. 点在边界上时: strict 为 true 返回 false, strict 为 false 返回 true. T 为 ll 时结果精确, 注意乘积溢出. 多次询问时每次 $O(\log n)$.

Usage:
```
vector<Point> h=convexHull(pts);
bool a=inHull(h,p); // 严格在内部
bool b=inHull(h,p,false); // 含边界
```

Time: $\mathcal{O}(\log n)$

"Point.h", "sideOf.h", "OnSegment.h" 009cc0, 14 lines

```
bool inHull(vector<Point>&l,Point p,bool strict=true){
  int a=1,b=l.size()-1,r=!strict;
  if(l.size()<3)return r&&onSegment(l[0],l.back(),p);
  if(sideOf(l[0],l[a],l[b])>0)swap(a,b);
  if(sideOf(l[0],l[a],p)>=r||sideOf(l[0],l[b],p)<=-r){
    return false;
  }
  while(abs(a-b)>1){ // 二分 p 所在的扇形
    int c=(a+b)/2;
    if(sideOf(l[0],l[c],p)>0)b=c;
    else a=c;
  }
  return sgn(cross(l[a],l[b],p))<r;
}
```

LineHullIntersection.h

Description: 直线与凸多边形求交. 多边形 poly 须为逆时针且无共线点 (如 convexHull 的结果), 下标 0..n-1. lineHull(a, b, poly) 返回一对下标, 描述直线 ab ($a \neq b$) 与多边形的相交情况: • $(-1,-1)$ 不相交, • $(i,-1)$ 只碰到顶点 i, • (i,i) 沿着边 $(i, i+1)$, • (i,j) 穿过边 $(i, i+1)$ 和 $(j, j+1)$. 最后一种情形中若穿过顶点 i, 视为发生在边 $(i, i+1)$ 上. 返回顺序与沿 a 到 b 方向依次碰到多边形的顺序一致. 下标 $i+1$ 按模 n 计算. extrVertex(poly, dir) 返回在方向 dir 上投影最大 (最远) 的顶点下标. 只用到叉积符号, T 为 ll 时结果精确.

Usage:
```
vector<Point> poly=convexHull(pts);
array<int,2> r=lineHull(a,b,poly);
if(r[0]==-1){} // 不相交
else if(r[1]==-1){} // 只碰到顶点 r[0]
else if(r[0]==r[1]){} // 沿着边 r[0]
else{} // 穿过边 r[0] 和边 r[1]
int i=extrVertex(poly,Point(1,0)); // x 最大的顶点
```

Time: $\mathcal{O}(\log n)$

"Point.h" b1ab15, 49 lines

```
int extrVertex(vector<Point>&poly,Point dir){
  int n=poly.size(),lo=0,hi=n;
  // 比较顶点 i 和 j 在 dir 上的投影，j 更远时为 1
  auto cmp=[&](int i,int j){
    return sgn(cross(perp(dir),poly[i%n]-poly[j%n]));
  };
  // i 是否为最远点
  auto extr=[&](int i){
    return cmp(i+1,i)>=0&&cmp(i,i-1+n)<0;
  };
  if(extr(0))return 0;
  while(lo+1<hi){
    int m=(lo+hi)/2;
    if(extr(m))return m;
    int ls=cmp(lo+1,lo),ms=cmp(m+1,m);
    if(ls<ms||(ls==ms&&ls==cmp(lo,m)))hi=m;
    else lo=m;
  }
  return lo;
}
array<int,2>lineHull(Point a,Point b,vector<Point>&poly){
  int n=poly.size();
  // 顶点 i 在直线 ab 的哪一侧
  auto cmpL=[&](int i){
    return sgn(cross(a,poly[i],b));
  };
  int endA=extrVertex(poly,perp(a-b));
  int endB=extrVertex(poly,perp(b-a));
  if(cmpL(endA)<0||cmpL(endB)>0)return {-1,-1};
  array<int,2>res;
  for(int i=0;i<2;i++){
    int lo=endB,hi=endA;
    while((lo+1)%n!=hi){
      int m=((lo+hi+(lo<hi?0:n))/2)%n;
```

```
      if(cmpL(m)==cmpL(endB))lo=m;
      else hi=m;
    }
    res[i]=(lo+!cmpL(hi))%n;
    swap(endA,endB);
  }
  if(res[0]==res[1])return {res[0],-1};
  if(!cmpL(res[0])&&!cmpL(res[1])){
    switch((res[0]-res[1]+n+1)%n){
      case 0:return {res[0],res[0]};
      case 2:return {res[1],res[1]};
    }
  }
  return res;
}
```

9.4 其他点集问题

ClosestPair.h

Description: 平面最近点对: 按 y 从小到大扫描, 用 set (按 x 排序) 维护 y 坐标在当前答案窗口内的点. 输入 vector<Point> (下标 0..n-1, $n \geq 2$, 可有重复点, 此时答案为 0), 返回距离最近的两个点. T 为 ll 时结果精确, 坐标约 10^9 以内.
Usage: `vector<Point> v; // v[0..n-1], n 至少为 2`
`pair<Point,Point> pq=closest(v);`
`T d2=len2(pq.first-pq.second); // 最近距离的平方`
`double d=sqrt((double)d2); // 最近距离`
Time: $\mathcal{O}(n \log n)$

"Point.h" 805a23, 25 lines

```
bool cmpY(Point a,Point b){return a.y<b.y;}
pair<Point,Point>closest(vector<Point>v){
  assert(v.size()>1);
  set<Point>S; // 按 x 排序
  sort(v.begin(),v.end(),cmpY);
  T best=LLONG_MAX; // 当前最近距离的平方
  pair<Point,Point>ret;
  int j=0;
  for(Point p:v){
    Point d(1+(ll)sqrt((double)best),0); // 当前答案的上界
    while(v[j].y<=p.y-d.x){ // 删去 y 差太大的点
      S.erase(v[j]);
      j++;
    }
    auto lo=S.lower_bound(p-d),hi=S.upper_bound(p+d);
    for(auto it=lo;it!=hi;it++){
      if(len2(*it-p)<best){
        best=len2(*it-p);
        ret={*it,p};
      }
    }
    S.insert(p);
  }
  return ret;
}
```

字符串 (10)

KMP.h

Description: KMP 单模式串匹配. 串均为 1-indexed, 放在 char 数组 s[1..n], t[1..m] 中. getnxt(t,m) 求前缀函数 nxt[1..m]: nxt[i] 为 t[1..i] 的最长真 border 长度, 即既是前缀又是后缀, 且不等于 t[1..i] 本身的最长串的长度. 例: abacaba 得 0 0 1 0 1 2 3. kmp(s,n,t,m) 先调用 getnxt(t,m), 再返回 t 在 s 中所有出现的起点, 按升序, 允许重叠. 常用结论: t 的最小周期为 m-nxt[m]; 若它整除 m 且不等于 m, 则 t 由更短的串重复得到. nxt 只对模式串 t 求, N 为串长上限. 多组数据无需清空, 直接再调用.
Usage: `scanf("%s%s",s+1,t+1);`
`int n=strlen(s+1),m=strlen(t+1);`
`vector<int> occ=kmp(s,n,t,m); // t 每次出现的起点`
`printf("%d",(int)occ.size());`
`// 只要前缀函数时:`
`getnxt(t,m); // 之后读 nxt[1..m]`
`int per=m-nxt[m]; // 最小周期`
Time: $\mathcal{O}(n+m)$

f6a48c, 24 lines

```
const int N=1e6+5; // 串长上限
int nxt[N];
void getnxt(char*t,int m){
  nxt[1]=0;
  for(int i=2,j=0;i<=m;i++){
    while(j&&t[i]!=t[j+1])j=nxt[j];
    if(t[i]==t[j+1])j++;
    nxt[i]=j;
  }
}
// 返回 t 在 s 中所有出现的起点
vector<int>kmp(char*s,int n,char*t,int m){
  getnxt(t,m);
  vector<int>res;
  for(int i=1,j=0;i<=n;i++){
    while(j&&s[i]!=t[j+1])j=nxt[j];
    if(s[i]==t[j+1])j++;
    if(j==m){
      res.push_back(i-m+1);
      j=nxt[j];
    }
  }
  return res;
}
```

Zfunc.h

Description: Z 函数, 即扩展 KMP. 串均为 1-indexed, 放在 char 数组 s[1..n], t[1..m] 中. getz(s,n) 求 z[1..n]: z[i] 为后缀 s[i..n] 与整个 s 的最长公共前缀长度, 约定 z[1] = n. 例: abacaba 得 7 0 1 0 3 0 1. exkmp(s,n,t,m) 求 ext[1..n]: ext[i] 为 s[i..n] 与 t[1..m] 的最长公共前缀长度. 它要用到 t 的 z 数组, 所以必须先调用 getz(t,m), 且之后不能再对别的串调用 getz. ext[i] = m 表示 t 在 s 的位置 i 出现. N 为串长上限, 多组数据无需清空.
Usage: `scanf("%s%s",s+1,t+1);`
`int n=strlen(s+1),m=strlen(t+1);`
`getz(s,n); // z[1..n], s 自己的 z 函数`
`getz(t,m); // 先求 t 的 z`
`exkmp(s,n,t,m); // 得到 ext[1..n]`
`for(int i=1;i<=n;i++)if(ext[i]==m)printf("%d ",i);`
Time: $\mathcal{O}(n+m)$

55c73c, 27 lines

```
const int N=1e6+5; // 串长上限
int z[N],ext[N];
void getz(char*s,int n){
  z[1]=n;
  // [l,r] 为当前右端点最大的匹配段
  for(int i=2,l=0,r=0;i<=n;i++){
    z[i]=i<=r?min(z[i-l+1],r-i+1):0;
    while(i+z[i]<=n&&s[1+z[i]]==s[i+z[i]])z[i]++;
    if(i+z[i]-1>r){
      l=i;
      r=i+z[i]-1;
    }
  }
}
// 需先 getz(t,m)
void exkmp(char*s,int n,char*t,int m){
  for(int i=1,l=0,r=0;i<=n;i++){
    ext[i]=i<=r?min(z[i-l+1],r-i+1):0;
    while(i+ext[i]<=n&&ext[i]<m&&s[i+ext[i]]==t[1+ext[i]]){
      ext[i]++;
    }
    if(i+ext[i]-1>r){
      l=i;
      r=i+ext[i]-1;
    }
  }
}
```

Manacher.h

Description: Manacher 求每个中心的最长回文. 输入串 s[1..n] 为 1-indexed. manacher(s,n) 在字符间和两端插入 #, 得到 a[1..m], $m = 2n + 1$: a[2i] = s[i], 奇数位为 #. 求出 p[1..m]: p[k] 为 a 中以 k 为中心的最长回文半径, 含中心. 返回值为 s 的最长回文子串长度. 读结果: a 中以 k 为中心的回文对应 s 中长 p[k]-1 的回文, 起点为 (k-p[k])/2+1. 以 s[i] 为中心的最长奇回文长 p[2i]-1; 以 s[i], s[i+1] 之间为中心的最长偶回文长 p[2i+1]-1. a[0] 与 a[m+1] 为两个不同的哨兵字符, 它们和 # 都不能在 s 中出现. a, p 开 2N. 多组数据无需清空.
Usage: `scanf("%s",s+1);`
`int n=strlen(s+1);`
`int ans=manacher(s,n); // 最长回文子串长度`
`// 以 s[i] 为中心的最长奇回文`
`int odd=p[2*i]-1;`
`// 以 s[i], s[i+1] 之间为中心的最长偶回文`
`int even=p[2*i+1]-1;`
Time: $\mathcal{O}(n)$

2931f2, 25 lines

```
const int N=1e6+5; // 串长上限
char a[N*2];
int p[N*2];
int manacher(char*s,int n){
  int m=0;
  a[0]='$';
  a[++m]='#';
  for(int i=1;i<=n;i++){
    a[++m]=s[i];
    a[++m]='#';
  }
  a[m+1]='@'; // 两端哨兵不同, 扩展到边界就停
  int ans=0;
  // mid 为右端最远的回文中心, r 为其右端的下一位
  for(int i=1,mid=0,r=0;i<=m;i++){
    p[i]=i<r?min(p[2*mid-i],r-i):1;
    while(a[i+p[i]]==a[i-p[i]])p[i]++;
    if(i+p[i]>r){
      r=i+p[i];
      mid=i;
    }
    ans=max(ans,p[i]-1);
  }
  return ans;
}
```

MinRotation.h

Description: 最小表示法. 输入串 s[1..n] 为 1-indexed, 把它看成环, minrot(s,n) 返回字典序最小的循环同构串的起点 k, 从 1 开始; 若有多个起点都能得到最小串, 返回最小的 k. 最小表示即 s[k..n] 接上 s[1..k-1]. 不需要全局数组, 不需要清空. 求最大表示: 把比较中的两个分支对调即可.
Usage: `scanf("%s",s+1);`
`int n=strlen(s+1);`
`int k=minrot(s,n);`
`for(int i=0;i<n;i++)putchar(s[(k-1+i)%n+1]);`
Time: $\mathcal{O}(n)$

e7936e, 18 lines

```
int minrot(char*s,int n){
  // i, j 为两个候选起点, 从 0 开始; k 为已匹配长度
  int i=0,j=1,k=0;
  while(i<n&&j<n&&k<n){
    int x=s[(i+k)%n+1];
    int y=s[(j+k)%n+1];
    if(x==y){
```

```
      k++;
      continue;
    }
    // 较大的那个起点及其后 k 位都不可能是答案
    if(x>y)i+=k+1;
    else j+=k+1;
    if(i==j)j++;
    k=0;
  }
  return min(i,j)+1;
}
```

SuffixArray.h

Description: 后缀数组, 倍增 + 基数排序. 输入串 s[1..n] 为 1-indexed. getsa(s,n) 求 sa[1..n] 与 rk[1..n]: sa[i] 为排名第 i 小的后缀的起点, rk[i] 为后缀 s[i..n] 的排名, 排名从 1 开始, 有 sa[rk[i]] = i. getheight(s,n) 需在 getsa 之后调用, 求 height[1..n]: height[i] 为后缀 sa[i] 与 sa[i-1] 的最长公共前缀长度, height[1] = 0. 任意两后缀 i, j 的 LCP: 设 $rk_i < rk_j$, 为 height[rk[i]+1..rk[j]] 的最小值, 配 ST 表 O(1) 查询. 本质不同子串数为 $n(n+1)/2 - \sum height_i$. 字符需在 1..127 内, 否则改 m 为字符集大小并把字符映射到 1..m. rk, tp 开 2N. 多组数据无需清空, 函数内部会清.

Usage: `scanf("%s",s+1);`

```
int n=strlen(s+1);
getsa(s,n);
getheight(s,n);
for(int i=1;i<=n;i++)printf("%d ",sa[i]);
ll d=1ll*n*(n+1)/2; // 本质不同子串数
for(int i=1;i<=n;i++)d-=height[i];
```

Time: $\mathcal{O}(n \log n)$

cf6d5c, 53 lines

```
const int N=1e6+5; // 串长上限
int sa[N],rk[N*2],tp[N*2],cnt[N],height[N];
void getsa(char*s,int n){
  int m=127; // 字符集大小
  for(int i=1;i<=2*n;i++){
    rk[i]=0;
    tp[i]=0;
  }
  for(int i=0;i<=m;i++)cnt[i]=0;
  for(int i=1;i<=n;i++){
    rk[i]=s[i];
    cnt[rk[i]]++;
  }
  for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
  for(int i=n;i>=1;i--)sa[cnt[rk[i]]--]=i;
  for(int w=1;;w<<=1){
    // tp: 按第二关键字排好的后缀
    int p=0;
    for(int i=n-w+1;i<=n;i++)tp[++p]=i;
    for(int i=1;i<=n;i++){
      if(sa[i]>w)tp[++p]=sa[i]-w;
    }
    // 按第一关键字 rk 做稳定计数排序
    for(int i=0;i<=m;i++)cnt[i]=0;
    for(int i=1;i<=n;i++)cnt[rk[i]]++;
    for(int i=1;i<=m;i++)cnt[i]+=cnt[i-1];
    for(int i=n;i>=1;i--)sa[cnt[rk[tp[i]]]--]=tp[i];
    // tp 改存旧的 rk, 重新编号
    for(int i=1;i<=n;i++)tp[i]=rk[i];
    p=0;
    for(int i=1;i<=n;i++){
      int x=sa[i];
      int y=sa[i-1];
      bool same=i>1&&tp[x]==tp[y]&&tp[x+w]==tp[y+w];
      rk[x]=same?p:++p;
    }
    if(p==n)break; // 排名已互不相同
    m=p;
  }
}
void getheight(char*s,int n){
  for(int i=1,k=0;i<=n;i++){
    if(rk[i]==1){
      height[1]=0;
      k=0;
      continue;
    }
    if(k)k--;
    int j=sa[rk[i]-1];
    while(i+k<=n&&j+k<=n&&s[i+k]==s[j+k])k++;
    height[rk[i]]=k;
  }
}
```

SuffixAutomaton.h

Description: 后缀自动机 SAM. 字符集为小写字母, 字符以 0..25 传入. 根为 1, 状态数至多 2n-1. 每个状态 v 表示一组 endpos 相同的子串: len[v] 为其中最长串的长度, fa[v] 为后缀链接, ch[v][c] 为转移, 0 表示没有转移. extend(c) 在串末尾加入字符 c. calc() 需在全部 extend 之后调用: 按 len 计数排序得到拓扑序 ord[1..tot], 逆序把 siz 累加到 fa, 使 siz[v] 等于 v 的 endpos 大小, 即 v 中每个串在原串中的出现次数; 返回本质不同子串个数 $\sum_{v>1} len_v - len_{fa_v}$. 查询 t 的出现次数: 从根沿 ch 走 t, 走不动则为 0, 否则为终点的 siz. 空间: ch 为 $2N \times 26$ 个 int, N 取 5e5 时约 104MB, 字符集或 N 大时注意. 多组数据先 init(), 它只清空用过的状态.

Usage: `scanf("%s",s+1);`

```
int n=strlen(s+1);
init(); // 每组数据先清空
for(int i=1;i<=n;i++)extend(s[i]-'a');
ll d=calc(); // 本质不同子串数, 并求出 siz
// 查询 t[1..m] 的出现次数
int u=1;
for(int i=1;i<=m&&u;i++)u=ch[u][t[i]-'a'];
int occ=u?siz[u]:0;
```

Memory: $\mathcal{O}(26n)$

Time: $\mathcal{O}(26n)$

8906cd, 52 lines

```
const int N=5e5+5; // 串长上限
int len[N*2],fa[N*2],ch[N*2][26],siz[N*2];
int tot=1,last=1;
int buc[N],ord[N*2];
void init(){
  for(int i=1;i<=tot;i++){
    len[i]=0;
    fa[i]=0;
    siz[i]=0;
    memset(ch[i],0,sizeof ch[i]);
  }
  tot=1;
  last=1;
}
void extend(int c){
  int p=last;
  int cur=++tot;
  len[cur]=len[p]+1;
  siz[cur]=1;
  for(;p&&!ch[p][c];p=fa[p])ch[p][c]=cur;
  if(!p){
    fa[cur]=1;
  }else{
    int q=ch[p][c];
    if(len[q]==len[p]+1){
      fa[cur]=q;
    }else{
      int cl=++tot; // 复制 q, 其 siz 为 0
      len[cl]=len[p]+1;
      fa[cl]=fa[q];
      memcpy(ch[cl],ch[q],sizeof ch[q]);
      for(;p&&ch[p][c]==q;p=fa[p])ch[p][c]=cl;
      fa[q]=cl;
      fa[cur]=cl;
    }
  }
  last=cur;
}
ll calc(){
  int n=len[last];
  for(int i=0;i<=n;i++)buc[i]=0;
  for(int i=1;i<=tot;i++)buc[len[i]]++;
  for(int i=1;i<=n;i++)buc[i]+=buc[i-1];
  for(int i=tot;i>=1;i--)ord[buc[len[i]]--]=i;
  ll res=0;
  for(int i=tot;i>=2;i--){ // ord[1] 为根
    int v=ord[i];
    siz[fa[v]]+=siz[v];
    res+=len[v]-len[fa[v]];
  }
  return res;
}
```

Hashing.h

Description: 字符串哈希, 双模数. 输入串 s[1..n] 为 1-indexed. build(s,n) 预处理前缀哈希 h1, h2 与幂次 p1, p2. get(l,r) 返回子串 s[l..r] 的哈希, calc(t,m) 返回另一个串 t[1..m] 整体的哈希, 二者可直接比较. 两个模数的结果合成一个 ll 返回, 可直接比较或放进 map, set, unordered_map. 冲突: 双模单次冲突概率约 10^{-18}, 很安全. 单模 10^9+7 比较约 10^5 个串时 就可能因生日悖论冲突; 自然溢出 unsigned long long 最快, 但会被 Thue-Morse 串卡掉. 怕被 hack 时把 B1, B2 换成随机数. 注意: 字符值不能为 0, 例如不要存 s[i]-'a', 否则 a 与 aa 等长度不同的串可能同哈希. 多个串要互相比较子串时, 可拼成一个串再 build, 或分别开数组. 多组数据直接再 build.

Usage: `scanf("%s",s+1);`

```
int n=strlen(s+1);
build(s,n);
// 判断 s[l1..r1] 与 s[l2..r2] 是否相同
bool same=get(l1,r1)==get(l2,r2);
ll ht=calc(t,m); // t[1..m] 的哈希
bool occ=get(1,1+m-1)==ht; // t 是否在位置 l 出现
```

Time: 预处理 $\mathcal{O}(n)$, 查询 $\mathcal{O}(1)$

642520, 31 lines

```
const int N=1e6+5; // 串长上限
const int M1=1e9+7,M2=998244353;
const int B1=131,B2=13331;
int h1[N],h2[N],p1[N],p2[N];
void build(char*s,int n){
  p1[0]=1;
  p2[0]=1;
  h1[0]=0;
  h2[0]=0;
  for(int i=1;i<=n;i++){
    p1[i]=1ll*p1[i-1]*B1%M1;
    p2[i]=1ll*p2[i-1]*B2%M2;
    h1[i]=(1ll*h1[i-1]*B1+s[i])%M1;
    h2[i]=(1ll*h2[i-1]*B2+s[i])%M2;
  }
}
// s[l..r] 的哈希
ll get(int l,int r){
  ll x=(h1[r]-1ll*h1[l-1]*p1[r-l+1]%M1+M1)%M1;
  ll y=(h2[r]-1ll*h2[l-1]*p2[r-l+1]%M2+M2)%M2;
  return x*M2+y;
}
// t[1..m] 整体的哈希
ll calc(char*t,int m){
  ll x=0,y=0;
  for(int i=1;i<=m;i++){
    x=(x*B1+t[i])%M1;
    y=(y*B2+t[i])%M2;
  }
  return x*M2+y;
```

```
}
```

AhoCorasick.h

Description: AC 自动机, 多模式串匹配. 字符集为小写字母, 串均为 1-indexed. 根为 0. insert(s,n,id) 把第 id 个模式串 s[1..n] 插入 Trie, pos[id] 记下它结尾的节点. build() 用 BFS 求 fail, 并把缺失的转移补成 tr[fail[u]][c], 得到 Trie 图; q[1..tot] 为 BFS 序. query(t,m) 在文本 t[1..m] 上跑自动机, 在每个经过的节点 cnt++, 再按 BFS 逆序沿 fail 树累加. 之后节点 v 的 cnt 即以 v 代表的串在 t 中的出现次数, 第 id 个模式串出现 cnt[pos[id]] 次. 允许重复模式串, 它们共享节点; 不允许空串. N 为模式串总长上限; tr 为 $26N$ 个 int, N 取 1e6 时约 104MB. 多组数据先 init(), 它只清空用过的节点. 同一自动机换文本再 query 前, 要把 cnt[0..tot] 清 0.

Usage:
```
init(); // 每组数据先清空
for(int i=1;i<=k;i++){
scanf("%s",p+1);
insert(p,strlen(p+1),i);
}
build();
scanf("%s",t+1);
query(t,strlen(t+1));
for(int i=1;i<=k;i++)printf("%d ",cnt[pos[i]]);
```

Time: 构建 $\mathcal{O}(26L)$, L 为模式串总长; 查询 $\mathcal{O}(m+L)$

f70fcc, 47 lines

```
const int N=1e6+5; // 模式串总长上限
int tr[N][26],fail[N],cnt[N],q[N],tot;
int pos[N]; // pos[id]: 第 id 个模式串结尾的节点
void init(){
  for(int i=0;i<=tot;i++){
    memset(tr[i],0,sizeof tr[i]);
    fail[i]=0;
    cnt[i]=0;
  }
  tot=0;
}
void insert(char*s,int n,int id){
  int u=0;
  for(int i=1;i<=n;i++){
    int c=s[i]-'a';
    if(!tr[u][c])tr[u][c]=++tot;
    u=tr[u][c];
  }
  pos[id]=u;
}
void build(){
  int hd=1,tl=0;
  for(int c=0;c<26;c++){
    if(tr[0][c])q[++tl]=tr[0][c];
  }
  while(hd<=tl){
    int u=q[hd++];
    for(int c=0;c<26;c++){
      int v=tr[u][c];
      if(v){
        fail[v]=tr[fail[u]][c];
        q[++tl]=v;
      }else{
        tr[u][c]=tr[fail[u]][c];
      }
    }
  }
}
void query(char*t,int m){
  int u=0;
  for(int i=1;i<=m;i++){
    u=tr[u][t[i]-'a'];
    cnt[u]++;
  }
  // q[1..tot] 为 BFS 序, 逆序累加到 fail
  for(int i=tot;i>=1;i--)cnt[fail[q[i]]]+=cnt[q[i]];
}
```

```
}
```

杂项 (11)

11.1 区间

IntervalContainer.h

Description: 区间集合: 用 set 维护若干互不相交的整数闭区间 $[l,r]$, 支持加入区间, 删除区间, 查询点. 集合中的区间两两不相交也不相邻, 按左端点排序, 所以 st 里的 (l, r) 就是被覆盖的点组成的极大连续段. add(l, r): 把 $[l,r]$ 内的点全部覆盖, 与它相交或相邻的区间会合并成一个; del(l, r): 把 $[l,r]$ 内的点全部删掉, 跨过边界的区间会被切开 (可能切成两段); getIt(x): 返回包含点 x 的区间的迭代器, 没有被覆盖时返回 st.end(); tot: 当前被覆盖的点数. 要求 $l \le r$, 端点不要用 INT_MIN 或 INT_MAX (会算 $l-1$, $r+1$). 区间是左闭右开 $[L,R)$ 时, 调用 add(L, R - 1) 即可.

Usage:
```
add(1, 5); add(7, 9); add(6, 6); // st = {[1, 9]}
del(3, 4); // st = {[1, 2], [5, 9]}, tot = 7
auto it = getIt(6); // it 指向 (5, 9)
for(auto [l, r] : st) ...  // 按顺序遍历所有区间
st.clear(); tot = 0; // 多组数据清空
```

Time: 每次操作均摊 $\mathcal{O}(\log n)$

e7be0f, 38 lines

```
set<pair<int,int>>st;
ll tot; // 被覆盖的点数
void add(int l,int r){
  auto it=st.lower_bound({l,INT_MIN});
  if(it!=st.begin()&&prev(it)->second>=l-1)it--;
  while(it!=st.end()&&it->first<=r+1){ // 相交或相邻
    l=min(l,it->first);
    r=max(r,it->second);
    tot-=it->second-it->first+1;
    it=st.erase(it);
  }
  st.insert({l,r});
  tot+=r-l+1;
}
void del(int l,int r){
  auto it=st.lower_bound({l,INT_MIN});
  if(it!=st.begin()&&prev(it)->second>=l)it--;
  while(it!=st.end()&&it->first<=r){
    int L=it->first,R=it->second;
    it=st.erase(it);
    tot-=R-L+1;
    if(L<l){ // 左边剩下 [L, l - 1]
      st.insert({L,l-1});
      tot+=l-L;
    }
    if(R>r){ // 右边剩下 [r + 1, R]
      st.insert({r+1,R});
      tot+=R-r;
    }
  }
}
set<pair<int,int>>::iterator getIt(int x){
  auto it=st.upper_bound({x,INT_MAX});
  if(it==st.begin())return st.end();
  it--;
  if(it->second<x)return st.end();
  return it;
}
```

11.2 其他算法

TernarySearch.h

Description: 三分法求单峰函数的最大值点. 整数版 tern(l, r): 在整数区间 $[l,r]$ 上求 f 的最大值点, 返回最大值点中最左的一个. 要求 f 在最大值左边严格上升, 右边不增 (可以有相等); 实际是二分 $f(mid) < f(mid+1)$, 每步算 2 次 f. 如果最大值左边有平台 (相邻两个值相等), 三分会出错, 只能换方法. 求最小值: 把 tern 里的 < 改成 > (要求左边严格下降, 右边不减). 实数版 ternR(l, r): 对 $[l,r]$ 上的单峰函数 F 三分 100 次, 返回最大值点 (精度远小于 10^{-9} 倍区间长). f 和 F 都是全局函数, 按题目改写 (可以用全局数组). 端点可以到 int 范围.

Usage:
```
ll f(int x) { return ...; } // 自己写
int p = tern(1, n); // f(p) 最大
double F(double x) { return ...; } // 实数版
double x = ternR(0, 1e9); // F(x) 最大
```

Time: $\mathcal{O}(\log(r-l))$ 次求值, 实数版 200 次

4d20e9, 19 lines

```
ll f(int x){return -(ll)(x-3)*(x-3);} // 按题目改
int tern(int l,int r){
  while(l<r){
    int mid=((ll)l+r)>>1;
    if(f(mid)<f(mid+1))l=mid+1; // 最大值在 mid 右边
    else r=mid;
  }
  return l;
}
double F(double x){return -(x-1)*(x-1);} // 按题目改
double ternR(double l,double r){
  for(int it=0;it<100;it++){
    double m1=l+(r-l)/3;
    double m2=r-(r-l)/3;
    if(F(m1)<F(m2))l=m1;
    else r=m2;
  }
  return l;
}
```

LIS.h

Description: 最长严格上升子序列 (LIS), 并求出一个具体的最长子序列. 输入全局数组 a[1..n] ($n < N$), lis(n) 返回一个最长严格上升子序列的下标 (从小到大), 长度就是 size(). d[k] 为长度为 k 的上升子序列的最小结尾值, 单调递增, 对每个 a[i] 二分出它能接在哪个长度后面; pos[k] 为 d[k] 对应的下标, pre[i] 为以 i 结尾时前一个元素的下标, 最后从 pos[len] 倒着走回去. 最长不降子序列: 把 lower_bound 改成 upper_bound. 最长下降: 把 a 取相反数. 只要长度不要方案时删掉 pos 和 pre. a 是 ll 时把 a 和 d 都改成 ll. 多组数据不需要清空.

Usage:
```
// 读入 n 和 a[1..n]
vector<int> id = lis(n);
int len = id.size(); // LIS 的长度
for(int i : id) cout << a[i] << ' '; // 输出子序列
```

Time: $\mathcal{O}(n \log n)$

474a7a, 16 lines

```
const int N=2e5+5;
int a[N],d[N],pos[N],pre[N];
vector<int>lis(int n){
  int len=0;
  for(int i=1;i<=n;i++){
    int k=lower_bound(d+1,d+len+1,a[i])-d;
    d[k]=a[i];
    pos[k]=i;
    pre[i]=pos[k-1];
    len=max(len,k);
  }
  vector<int>res;
  for(int i=pos[len];i;i=pre[i])res.push_back(i);
  reverse(res.begin(),res.end());
  return res;
}
```

11.3 动态规划

KnuthDP.h

Description: 区间 DP 的 Knuth 优化 (四边形不等式优化), 把 $O(n^3)$ 的区间 DP 降到 $O(n^2)$. 适用的转移: $f[l][r]=\min_{l\le k<r}(f[l][k]+f[k+1][r])+w(l,r)$, $f[i][i]=0$, 答案 $f[1][n]$. 条件: 对所有 $a\le b\le c\le d$ 有 $w(a,c)+w(b,d)\le w(a,d)+w(b,c)$ (四边形不等式) 且 $w(b,c)\le w(a,d)$ (区间包含单调). 此时最优分割点满足 $opt[l][r-1]\le opt[l][r]\le opt[l+1][r]$, 只在这个范围内枚举 k. 例子是石子合并: $w(l,r)$ 为 $a_l..a_r$ 的和, 用前缀和 s; 按题目改 w 即可. 调用 knuth(n) 后 f[l][r] 为区间 $[l,r]$ 的答案, opt[l][r] 为它的最优分割点 (左半边是 $[l,k]$). 环形问题把数组复制一倍, 求 $\min f[i][i+n-1]$. 求最大值时一般不满足条件, 不要直接改成 max. 多组数据不需要清空 (用到的 f, opt 都会重新算).

Usage:
```
// 读入 n 和 a[1..n]
for(int i = 1; i <= n; i++) s[i] = s[i - 1] + a[i];
ll ans = knuth(n); // f[1][n]
```
Memory: N = 3005 时约 110MB

Time: $\mathcal{O}(n^2)$

378f17, 25 lines

```
const int N=3005;
ll f[N][N],s[N];
int opt[N][N]; // f[l][r] 的最优分割点
ll w(int l,int r){return s[r]-s[l-1];} // 按题目改
ll knuth(int n){
  for(int i=1;i<=n;i++){
    f[i][i]=0;
    opt[i][i]=i;
  }
  for(int len=2;len<=n;len++){
    for(int l=1;l+len-1<=n;l++){
      int r=l+len-1;
      f[l][r]=LLONG_MAX;
      int R=min(r-1,opt[l+1][r]);
      for(int k=opt[l][r-1];k<=R;k++){
        ll v=f[l][k]+f[k+1][r]+w(l,r);
        if(v<f[l][r]){
          f[l][r]=v;
          opt[l][r]=k;
        }
      }
    }
  }
  return f[1][n];
}
```

DivideAndConquerDP.h

Description: 分治优化 DP (决策单调性). 把 $1..n$ 分成恰好 m 段 (每段非空), 段 $[l,r]$ 的代价为 $w(l,r)$, 求最小总代价. 转移: $f_j[i]=\min_{j-1\le k<i} f_{j-1}[k]+w(k+1,i)$, 共 m 层, 每层 $O(n\log n)$. 条件: 最优的 k 随 i 单调不减; w 满足四边形不等式 $w(a,c)+w(b,d)\le w(a,d)+w(b,c)$ $(a\le b\le c\le d)$ 时成立. dc(l, r, kl, kr) 求当前层的 f[l..r], 已知它们的最优 k 都在 $[kl,kr]$ 内: 先暴力求中点 mid 的最优 k (best), 左半边的最优 k 不超过 best, 右半边不小于 best. g 为上一层的结果, f 为当前层. 例子: $w(l,r)$ 为区间和的平方 (前缀和 s), 按题目改 w. 要求 $1\le m\le n$. solveDP(n, m) 返回答案; 结束后 g[i] 为前 i 个分成 m 段的最小代价 ($i\ge m$; $m=1$ 时 g[i] $=w(1,i)$).

Usage:
```
// 读入 n, m, a[1..n]
for(int i = 1; i <= n; i++) s[i] = s[i - 1] + a[i];
ll ans = solveDP(n, m); // 多组数据直接再调用
```
Time: $\mathcal{O}(mn\log n)$

55d175, 30 lines

```
const int N=1e5+5;
ll f[N],g[N],s[N];
ll w(int l,int r){ // 按题目改
  ll x=s[r]-s[l-1];
  return x*x;
}
// 求 f[l..r], 它们的最优 k 在 [kl, kr] 内
void dc(int l,int r,int kl,int kr){
  if(l>r)return;
  int mid=(l+r)/2;
  int best=kl;
  f[mid]=LLONG_MAX;
  for(int k=kl;k<=min(mid-1,kr);k++){
    ll v=g[k]+w(k+1,mid);
    if(v<f[mid]){
      f[mid]=v;
      best=k;
    }
  }
  dc(l,mid-1,kl,best);
  dc(mid+1,r,best,kr);
}
ll solveDP(int n,int m){
  for(int i=1;i<=n;i++)g[i]=w(1,i); // 第 1 层
  for(int j=2;j<=m;j++){
    dc(j,n,j-1,n-1);
    for(int i=j;i<=n;i++)g[i]=f[i];
  }
  return g[n];
}
```

WQS 二分 (Aliens trick)

适用于恰好选 k 个或恰好分 k 段的最优值 $g(k)$, 且 g 关于 k 是凸函数 (求最小值时下凸, 求最大值时上凸). 凸性一般靠打表观察, 或由四边形不等式、费用流模型得到.

- **去掉个数限制**: 每选一个额外加代价 λ, 做不限个数的 DP, 求 $h(\lambda)=\min_k\{g(k)+\lambda k\}$, 同时记录取到最优时选了几个, 记为 $cnt(\lambda)$.
- **打平手时取最少个数**: DP 里值相同的转移一律选个数更少的. 这样 $cnt(\lambda)$ 随 λ 单调不增.
- **二分**: 找最小的 λ 使 $cnt(\lambda)\le k$, 答案是 $g(k)=h(\lambda)-\lambda k$. 注意减去的是 λk 而不是 $\lambda\cdot cnt$, 这样多个 k 共线时也正确.
- **求最大值**: 每选一个减去 λ, $h(\lambda)=\max_k\{g(k)-\lambda k\}$, 同样平手取最少个数, 找最小的 λ 使 $cnt(\lambda)\le k$, 答案 $h(\lambda)+\lambda k$.
- **细节**: g 是整数时 λ 只需整数二分, 范围取相邻差分的最小到最大值 (如 $[-10^{12},10^{12}]$). 总复杂度是 DP 的复杂度乘 $\log V$.
- **常见题**: 恰好 k 段划分, 恰好选 k 个不相邻元素, 恰好 k 条白边的最小生成树.

11.4 Kruskal 重构树

用于 只经过边权不超过 x 的边 能到达哪些点, 以及 两点间所有路径中最大边权的最小值 (最小瓶颈路) 这类问题.

- **建树**: 边按权从小到大排序做 Kruskal. 每合并两个不同的连通块 (并查集的根为 a,b) 时, 新建一个点 z (编号从 $n+1$ 开始), 点权 $val[z]=w$, 让 z 成为 a 和 b 的父亲, 并查集里也把 a,b 并到 z 上. 原图的点全是叶子, 连通图建完共 $2n-1$ 个点, 根是最后新建的点.
- **性质**: 二叉树, 且从叶子往根走点权不降 (祖先的点权不小于子孙).
- **最小瓶颈路**: 原图中 u,v 之间所有路径上最大边权的最小值等于 $val[\mathrm{lca}(u,v)]$. 不连通时两点不在同一棵树里.
- **只走边权不超过 x 的边**: 从 u 往上跳, 跳到点权不超过 x 的最高祖先 a (倍增), 能到达的点就是 a 子树里的所有叶子. 用 dfs 序把子树变成区间, 就能配合主席树等求这些点里的第 k 大等.
- **反过来**: 若关心路径上最小边权的最大值, 改成边权从大到小建树, 上面的不等号全部反过来.

11.5 调试技巧

- `signal(SIGSEGV, [](int) { _Exit(0); });` 把段错误变成 Wrong Answer. 同理可捕获 SIGABRT (断言失败) 和 SIGFPE (除零). `_GLIBCXX_DEBUG` 报错时产生 SIGABRT (gcc 5.4.0 上似乎是 SIGSEGV).
- `feenableexcept(29);` 在遇到 NaN (1), 除零 (4), 无穷 (8) 和非规格化数 (16) 时终止程序.

11.6 优化技巧

`__builtin_ia32_ldmxcsr(40896);` 关闭非规格化数 (它们在最小值附近让浮点运算慢 20 倍).

11.6.1 位运算技巧

- `x&-x` 是 x 的最低位的 1.
- `c=x&-x,r=x+c;(((r^x)>>2)/c)|r` 是比 x 大且 1 的个数相同的下一个数.
- 子集和 (SOS DP):
```
for(int b=0;b<K;b++)
 for(int i=0;i<(1<<K);i++)
  if(i>>b&1)D[i]+=D[i^(1<<b)];
```
- 枚举所有集合的所有子集, 总共 3^n 次. 只要非空子集时内层写成 `for(int t=s;t;t=(t-1)&s):`

```
for(int s=0;s<(1<<n);s++) // 每个集合 s
  for(int t=s;;t=(t-1)&s){ // s 的每个子集 t, 含空集
    // 处理 (s, t)
    if(!t)break;
  }
```

11.6.2 bitset 加速 pragma (放在 bits/stdc++.h 之前)

- `#pragma GCC optimize("O3,unroll-loops")`
- `#include <bits/allocator.h>`
- `#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt")`

UniformRandom.h

Description: 随机数. rng 是 64 位的 mt19937_64, 用当前时间做种子 (每次运行不同, 防止被 hack). rnd(l, r) 返回 $[l,r]$ 内均匀随机的整数 (闭区间, 要求 $l\le r$, 可以到 long long 范围). 不要用 rand(): 在 Windows 上最大只有 32767, 而且质量差. 调试时想固定结果, 把种子改成常数, 如 rng(12345).

Usage: `ll x = rnd(1, 100); // [1, 100] 内的随机整数`
`shuffle(a + 1, a + n + 1, rng); // 随机打乱 a[1..n]`
`double y = uniform_real_distribution<double>(0, 1)(rng);`
`unsigned long long h = rng(); // 64 位随机数, 可做哈希`
Time: $\mathcal{O}(1)$

bb326d, 5 lines

```
mt19937_64 rng(chrono::steady_clock::now()
  .time_since_epoch().count());
ll rnd(ll l,ll r){ // [l, r] 内均匀随机
  return uniform_int_distribution<ll>(l,r)(rng);
}
```

StressTest.sh

24 lines

```
# Usage: bash stress.sh [fast] [slow] [gen] [# iters]
# Example: bash stress.sh main main2 generator 1000
# remove this and add -O2 if you want, cc8b47
flags="-std=c++20 -g -fsanitize=address,undefined"
g++ $flags $1.cpp -o $1
g++ $flags $2.cpp -o $2
g++ $flags $3.cpp -o $3
for ((t=1; t<=$4; t++)) do
  ./$3 > input
  ./$2 < input > outslow
  ./$1 < input > outwrong
  if [[ "$?" -ne 0 ]] || ! (cmp -s "outwrong" "outslow") then
    echo error found
    echo input:
    cat input
    echo wrong output:
    cat outwrong
    echo slow output:
    cat outslow
    exit
  fi
  echo passed $t tests
done
echo passed all tests
```